

Universitatea Națională de Știință și Tehnologie POLITEHNICĂ București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Analiza și implementarea protocoalelor criptografice folosite de IKEv2 și IPsec

Proiect de diplomă

Prezentată ca cerință parțială pentru obținerea

Titlului de *Inginer*

**În domeniul *Electronică, Telecomunicații și Tehnologia Informației* programul
de studii *Rețele software de telecomunicații***

Conducător științific

Conf.Dr.Ing. Octavian CATRINA

Absolvent

Fierea M.M. Cosmin-Andrei

Anul 2026

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **Fierea M.M. Cosmin-Andrei, 441D**

1. Titlul temei: Analiza și implementarea protocoalelor criptografice folosite de IKEv2 și IPsec

2. Descrierea temei:

Proiectul va analiza și implementa variante simplificate ale protocoalelor criptografice utilizate pentru a asigura securitatea comunicațiilor la nivel rețea în Internet: (1) stabilirea cheilor secrete folosind IKEv2, cu criptografie simetrică și asimetrică; (2) criptarea autenticată a pachetelor IP folosind IPsec-ESP. Protocoalele implementate vor conține doar nucleul criptografic al protocoalelor folosite de IKEv2 și IPsec, care asigură îndeplinirea cerințelor fundamentale de securitate, fără alte detalii (de exemplu, fără negocierea algoritmilor criptografici). Infrastructura necesară implementării și testării protocoalelor va fi asigurată de platforma software SJAKE (realizată de conducătorul proiectului), bazată pe bibliotecile criptografice standard din limbajul Java.

3. Contribuția originală:

- Identificarea, descrierea și analiza nucleului criptografic al protocoalelor folosite de IKEv2 pentru stabilirea cheilor secrete și IPsec-ESP pentru criptarea autenticată a pachetelor IP. Protocoalele pentru stabilirea cheilor secrete vor include: (1) autentificare mutuală folosind criptografie simetrică (cod de autentificare și cheie secretă prestabilite, MAC-PSK); (2) autentificare mutuală folosind criptografie asimetrică (semnătură și certificat pentru cheia publică, SIG-PKC); (3) chei secrete generate folosind protocolul Diffie-Hellman (DH); (4) criptare autenticată a identităților și a datelor inițiale.
- Implementarea și testarea protocoalelor folosind SJAKE: stabilirea cheilor de sesiune cu autentificare MAC-PSK și SIG-PKC; transferul datelor cu criptarea autenticată folosind cheile de sesiune stabilite.
- Analiză comparativă a variantelor MAC-PSK și SIG-PKC: securitate, complexitate, scalabilitate, performanțe.
- Identificarea unor potențiale vulnerabilități ale acestor construcții și implementarea folosind SJAKE a unor atacuri care exploatează aceste vulnerabilități.
- Programele de test vor permite: selectarea testului executat (protocoale și atacuri); vizualizarea comunicațiilor și a informației de stare pentru participanții onești și pentru adversari; analiză comparativă a performanțelor pentru diverse variante de protocoale și de configurare criptografică.

4. Resurse și bibliografie:

Proiectul se bazează pe o serie de articole de cercetare, documentația platformei SJAKE și cursul Securitatea rețelilor și serviciilor. Resursele necesare realizării proiectului includ: Java Development Kit; mediu integrat de dezvoltare pentru programe Java (Eclipse); platforma SJAKE pentru implementarea și testarea protocoalelor criptografice.

5. Data înregistrării temei: 2025-12-15 16:24:45

Conducător(i) lucrare,
Conf.Dr.Ing. Octavian CATRINA

Student,
Fierea M.M. Cosmin-Andrei

Director departament,
Conf. dr. ing. Șerban OBREJA

Decan,
Prof. dr. ing. Mihnea UDREA

Cod Validare: **1e22c1463d**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul “ Analiza și implementarea protocoalelor criptografice folosite de IKEv2 și IPsec”, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de Inginer domeniul Inginerie Electronică și Telecomunicații, programul de studii program Rețele și Software de Telecomunicații este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate.

Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare.

Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026

Absolvent: Fierea Cosmin-Andrei

.....

Cuprins

Capitolul 1. Introducere: Motivație și Obiective	1
Capitolul 2. Asigurarea securității comunicațiilor folosind IPsec	4
2.1 Rolul securității la nivel rețea	4
2.2 Security Associations și organizarea comunicației IPsec	5
2.3 Protocoalele IPsec: AH și ESP.....	5
2.4 Protocolul IKEv2	6
2.5 Variante de autentificare în IKEv2.....	7
2.6 Nucleul criptografic	8
Capitolul 3. Utilizarea platformei SJAKE pentru implementarea protocoalelor AKE.....	9
3.1 Considerații generale privind platforma SJAKE	9
3.2 Obiectivele platformei SJAKE	9
3.3 Arhitectura generală și componentele SJAKE.....	10
3.4 Modelul de comunicație și adversarul	11
3.5 Modelul generic al protocoalelor AKE și tipurile de mesaje.....	12
3.6 Automatul de stare pentru protocoalele MAKE	14
3.7 Modelul de testare și trasabilitatea execuției	16
Capitolul 4. Implementarea protocoalelor AKE folosite de IKEv2	19
4.1 Considerații generale privind implementarea	19
4.2 Implementarea protocolului MAKE-IKE-SIG-PKC-DHE 4M	20
4.3 Implementarea protocolului MAKE-IKE-MAC-PSK-DHE 4M.....	24
4.4 Implementarea protocolului MAKE-ESP-MAC-PSK-DHE 2M.....	26
4.5 Testarea funcțională a protocoalelor implementate	29
4.6 Analiza comparativă a variantelor MAC-PSK și SIG-PKC	32
Capitolul 5. Atacuri asupra unor variante vulnerabile ale protocoalelor	34

5.1 Considerații generale privind atacurile ILV implementate	34
5.2 Atacul ILV asupra variantei vulnerabile MAKE-IKE-MAC-PSK-DHE 4M	36
5.3 Atacul ILV asupra variantei vulnerabile MAKE-IKE-SIG-PKC-DHE 4M.....	40
5.4 Testarea atacurilor în SJAKE	43
Concluzii	47
Bibliografie.....	50
Anexe	51
Anexa A1. Cod complet MAKE-IKE-SIG-PKC-DHE 4M.....	51
Anexa A2. Cod complet MAKE-IKE-MAC-PSK-DHE 4M.....	56
Anexa A3. Cod complet MAKE-ESP-MAC-PSK-DHE	61
Anexa A4. Cod complet atac ILV asupra SIG-PKC	65
Anexa A5. Cod complet atac ILV asupra MAC-PSK.....	68

Lista Figurilor

Figura 1.1: Scenariu de comunicație securizată

Figura 2.1 Schema de securitate a comunicațiilor în arhitectura pe niveluri

Figura 2.2: Format Top-Level al unui pachet ESP

Figura 2.3: IKEv2 handshake

Figura 3.1 Arhitectura SJAKE

Figura 3.2 Modelul adversarului care controlează comunicațiile

Figura 3.3 Structura generică a protocoalelor AKE

Figura 3.4 Structura MAKE cu 3 mesaje

Figura 3.5 FSM MAKE 4M

Figura 3.6 FSM MAKE 2M

Figura 4.1 Implementarea Protocolului MAKE-IKE-SIG-PKC-DHE 4M

Figura 4.2 Implementarea Protocolului MAKE-IKE-MAC-PSK-DHE 4M

Figura 4.3 Implementarea Protocolului MAKE-ESP-MAC-PSK-DHE 2M

Figura 5.1 FSM-ul adversarului pentru atacul ILV asupra protocoalelor MAKE-4M vulnerabile

Figura 5.2 Atac ILV asupra MAKE-MAC-PSK-DHE 4M

Figura 5.3 Atac ILV asupra MAKE-SIG-PKC-DHE 4M

Lista Tabelelor

Tabelul 4.1 Rezultatele testelor de performanță pentru protocoalele MAKE-IKE

Lista de Acrônimo

AKE - Authenticated Key Exchange
TLS - Transport Layer Security
IPsec - Internet Protocol Security
IKEv2 - Internet Key Exchange version 2
IP - Internet Protocol
ESP - Encapsulating Security Payload
SA - Security Associations
SIG - Signature
PKC - Public Key Certificates
MAC - Message Authentication Code
PSK - Pre-shared Key
MAKE - Mutual Authenticated Key Exchange
AEAD - Authenticated Encryption with Associated Data
DH – Diffie-Hellman
SPI - Security Parameters Index
VPN - Virtual Private Network
AH - Authentication Header
GRE - Generic Routing Encapsulation
GCM - Galois/Counter Mode
SIGMA - Sign-and-Mac
NAT - Network Address Translation
TCP - Transmission Control Protocol
ECDH - Elliptic Curve Diffie-Hellman
RSA - Rivest-Shamir-Adleman
UAKE - Unilateral Authenticated Key Exchange
FSM - Finite State Machine
ILV - Interleaving

Capitolul 1. Introducere: Motivație și Obiective

În infrastructurile moderne de comunicații, securitatea informațiilor transmise prin rețele deschise reprezintă o cerință fundamentală. Comunicarea prin Internet presupune, în mod obișnuit, existența unui canal nesigur, în care un adversar poate intercepta, modifica, întârzia, reordona sau injecta mesaje. Din acest motiv, simpla transmitere a datelor între două entități nu oferă garanții privind confidențialitatea, integritatea sau originea informației. Pentru ca o comunicație să poată fi considerată securizată, participanții trebuie să stabilească mai întâi o relație criptografică de încredere, iar apoi să utilizeze cheile rezultate pentru protecția efectivă a datelor [1], [2].

Un model uzual de comunicație securizată este împărțit în două etape principale. Prima etapă este stabilirea sesiunii securizate, în care participanții execută un protocol de tip AKE (Authenticated Key Exchange) pentru a se autentifica și pentru a obține chei de sesiune proaspete. A doua etapă este transferul securizat al datelor, în care cheile stabilite anterior sunt utilizate de mecanisme de criptare autenticată pentru a proteja traficul transmis. Astfel de separare este întâlnită în protocoale utilizate pe scară largă, cum sunt TLS (Transport Layer Security) pentru comunicații la nivel transport și IPsec (Internet Protocol Security) pentru securizarea comunicațiilor la nivel rețea [1], [3], [4].

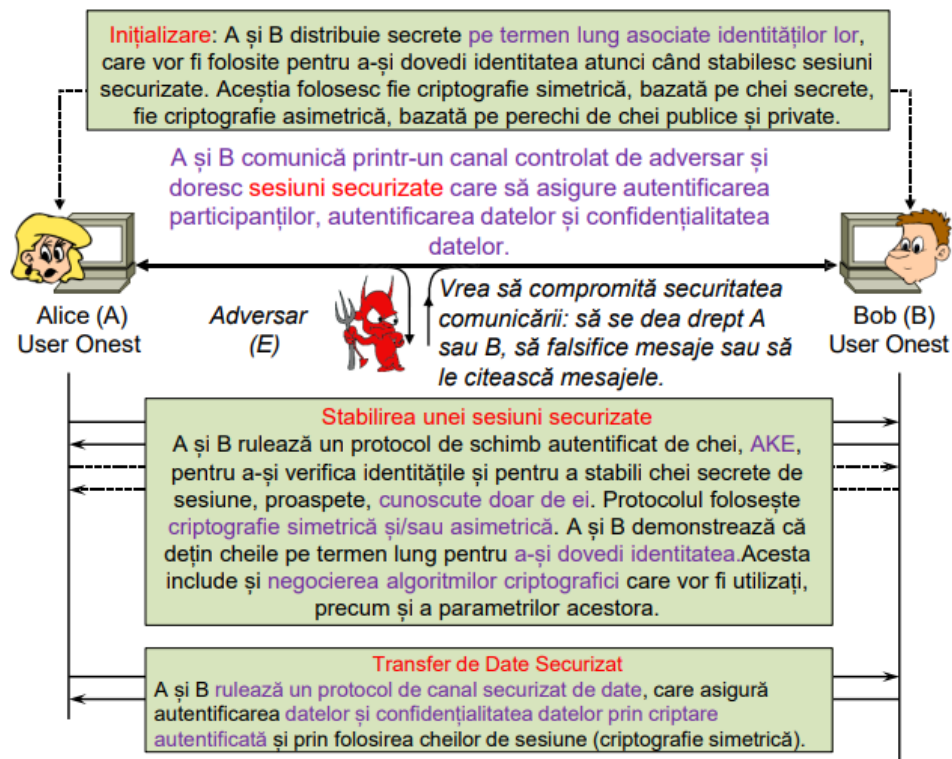


Figura 1.1 Scenariu de comunicație securizată [1]

Suita IPsec este folosită pentru protejarea comunicațiilor la nivel rețea și oferă servicii precum confidențialitate, integritate, autentificarea sursei datelor și controlul accesului. Aceste servicii sunt

realizate prin menținerea unor stări comune între capetele comunicației, denumite Security Associations, care includ informații despre algoritmi criptografici utilizați, cheile folosite și parametrii necesari protecției traficului. Stabilirea manuală a acestor informații nu este scalabilă, motiv pentru care este necesar un protocol de management al asocierilor de securitate. În cadrul suitei IPsec, acest rol este îndeplinit de IKEv2 (Internet Key Exchange version 2), care realizează autentificarea mutuală a participanților și stabilește asocierile de securitate folosite ulterior pentru protejarea pachetelor IP (Internet Protocol) [1], [3].

Componenta ESP (Encapsulating Security Payload) reprezintă mecanismul IPsec orientat către protecția efectivă a pachetelor IP. ESP poate oferi confidențialitate, autentificarea originii datelor, integritate fără conexiune, protecție anti-replay și o formă limitată de confidențialitate a fluxului de trafic. În contextul acestei lucrări, ESP este relevant deoarece reprezintă partea de transfer securizat al datelor, folosind cheile stabilite anterior prin mecanisme de tip IKEv2 [4].

Lucrarea de față are ca temă analiza și implementarea protocoalelor criptografice folosite de IKEv2 și IPsec, cu accent pe nucleul criptografic al acestor protocoale. Nu se urmărește implementarea completă a tuturor detaliilor operaționale din IKEv2 și IPsec, ci construirea unor variante simplificate, suficient de expresive pentru a evidenția mecanismele criptografice esențiale: autentificarea mutuală, schimbul Diffie-Hellman, derivarea cheilor de sesiune, criptarea autentificată a identităților și transferul securizat de date.

Simplificarea protocoalelor a fost făcută cu atenție, astfel încât să fie eliminate detalii funcționale, nu cerințe esențiale de securitate. Un exemplu important este negocierea algoritmilor criptografici. În IKEv2, participanții pot transmite mai multe oferte criptografice, iar una dintre ele este aleasă pentru sesiunea curentă. În implementarea realizată, modelul este redus la o singură configurație transmisă de participant. Totuși, partea importantă din punct de vedere al securității se păstrează: configurația criptografică este inclusă în datele autentificate. Astfel, un adversar nu poate modifica algoritmi negociați fără ca verificarea autenticatorului să eșueze. Prin urmare, negocierea este simplificată ca mecanism, dar rămâne autentificată.

Aceeași idee apare și în tratarea datelor inițiale transmise în mesajele Auth. Într-un protocol IKEv2 complet, aceste mesaje pot transporta informații necesare pentru stabilirea asocierilor de securitate IKE SA (Security Associations) și ESP SA. În această lucrare, aceste informații sunt modelate într-o formă simplificată, prin date inițiale introduse în mesajele de autentificare. Ele nu sunt transmise în clar, ci sunt protejate prin criptare autentificată. Astfel, implementarea păstrează cerința de securitate relevantă: datele importante pentru stabilirea sesiunii sunt protejate înainte de a fi acceptate de participanți [3], [4].

Un alt aspect urmărit este ascunderea identității participanților, cerință importantă în proiectarea IKEv2 [3]. În varianta bazată pe SIG-PKC (Signature-Public Key Certificates), identitatea criptografică este ascunsă prin transmiterea certificatului în partea criptată a dialogului Auth. Astfel, certificatul nu este expus în mesajele inițiale ale protocolului. În varianta bazată pe MAC-PSK (Message Authentication Code – Pre-shared Key), această proprietate nu a putut fi obținută complet, deoarece platforma SJAKE nu separă adresele de rețea de numele participanților. Numele participantului este folosit și pentru rutarea mesajelor în platformă. Această limitare ține de cadrul de implementare, nu de principiul criptografic analizat.

Baza lucrării constă în implementarea și analiza unor protocoale de stabilire a cheilor secrete inspirate din IKEv2, împreună cu un mecanism de transfer securizat al datelor inspirat din IPsec-ESP. Protocoalele analizate utilizează atât criptografie simetrică, prin autentificare bazată pe MAC-PSK, cât și criptografie asimetrică, prin autentificare bazată pe SIG-PKC. În ambele cazuri, cheile de sesiune sunt generate folosind valori Diffie-Hellman efemere, iar datele sensibile transmise în faza de autentificare sunt protejate prin criptare autenticată. Astfel, implementarea nu se limitează la un schimb simplu de chei, ci modelează mecanisme apropiate de nucleul criptografic al IKEv2 [1], [3].

În cadrul proiectului au fost implementate trei construcții principale de protocol: MAKE-IKE-SIG-PKC-DHE 4M, MAKE-IKE-MAC-PSK-DHE 4M și MAKE-ESP-MAC-PSK-DHE 2M. Pe lângă acestea, a fost implementat și mecanismul de transfer securizat al datelor prin criptare autenticată cu date adiționale opționale. De asemenea, lucrarea include analiza și implementarea unor atacuri de tip Interleaving asupra variantelor vulnerabile ale protocoalelor MAKE (Mutual Authenticated Key Exchange) bazate pe MAC-PSK și SIG-PKC. Aceste obiective corespund direcției proiectului, care urmărește implementarea construcțiilor AKE, transferul securizat prin AEAD (Authenticated Encryption with Associated Data), analiza atacurilor Interleaving și comparația performanțelor între variantele MAC-PSK și SIG-PKC [1], [5], [6], [7].

Implementarea și testarea protocoalelor sunt realizate folosind de aseamenea platforma SJAKE, destinată implementării rapide și analizei experimentale a protocoalelor de tip AKE. SJAKE oferă suport pentru construcții de protocoale cu autentificare unilaterală și mutuală, pentru variante bazate pe MAC-PSK, SIG-PKC și Diffie-Hellman, precum și pentru implementarea atacurilor asupra variantelor vulnerabile. Platforma include infrastructură de comunicație, primitive criptografice bazate pe Java Cryptography Architecture, interfețe de testare și suport pentru măsurători de performanță [5], [6], [8].

Un alt obiectiv al lucrării este analiza comparativă a variantelor MAC-PSK și SIG-PKC. Autentificarea prin MAC-PSK este eficientă din punct de vedere computațional, deoarece se bazează pe operații simetrice rapide, dar are limitări de scalabilitate, deoarece presupune existența unor chei secrete prestabilite între participanți. Autentificarea prin SIG-PKC are un cost computațional mai ridicat, deoarece implică semnături digitale și certificate de cheie publică, dar este mai potrivită pentru scenarii distribuite, cu număr mare de entități, unde gestionarea perechilor de chei pre-partajate ar deveni dificilă. Comparația este urmărită atât din punct de vedere conceptual, cât și experimental, prin măsurarea timpilor de execuție pentru configurații criptografice relevante [1], [7].

Lucrarea este structurată în șase capitole. Capitolul 1 introduce contextul general, motivația, obiectivele și structura lucrării. Capitolul 2 prezintă suita IPsec, protocolul IKEv2 și cerințele de securitate urmărite de aceste mecanisme, cu accent pe rolul protocoalelor AKE și al ESP. Capitolul 3 descrie platforma SJAKE și modelul generic de implementare a protocoalelor, incluzând tipurile de mesaje, fluxurile de protocol, automatele de stare și mecanismele de testare. Capitolul 4 prezintă protocoalele implementate, separat pentru variantele SIG-PKC, MAC-PSK și ESP-MAC-PSK, explicând principiile criptografice, structura mesajelor și deciziile de implementare. Capitolul 5 analizează atacurile implementate asupra variantelor vulnerabile, în special atacurile de tip Interleaving, și explică de ce protocoalele corecte nu sunt afectate de aceleași vulnerabilități. Capitolul 6 prezintă concluziile lucrării, rezultatele obținute, observațiile privind implementarea și posibile direcții de extindere.

Capitolul 2. Asigurarea securității comunicațiilor folosind IPsec

2.1 Rolul securității la nivel rețea

Securitatea comunicațiilor poate fi implementată la mai multe niveluri ale arhitecturii de rețea. La nivel aplicație, fiecare serviciu își poate defini propriile mecanisme de protecție, însă această abordare presupune proiectarea unor soluții separate pentru fiecare aplicație. La nivel transport, protocoale precum TLS oferă comunicații securizate între procese, fiind foarte potrivite pentru aplicații client-server. În schimb, securitatea la nivel rețea are avantajul că poate proteja traficul IP într-un mod transparent pentru aplicațiile superioare. Din acest motiv, IPsec este folosit în special pentru protejarea comunicațiilor între gazde, între gateway-uri de securitate sau între o gazdă și un gateway de securitate [1], [3], [4].

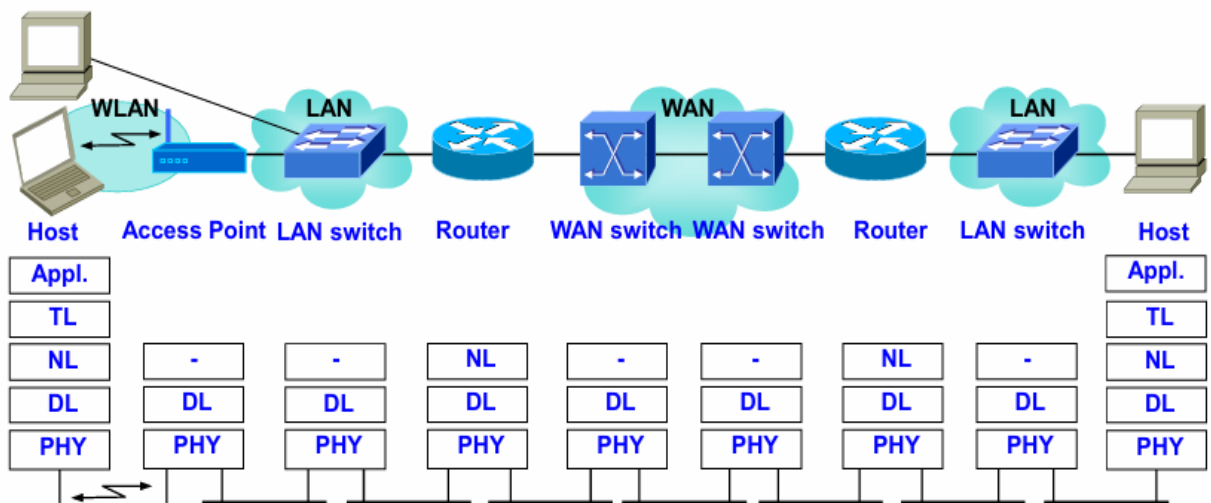


Figura 2.1 Schema de securitate a comunicațiilor în arhitectura pe niveluri [1]

Suita IPsec urmărește să asigure securitatea pachetelor IP transmise printr-o rețea nesigură. Într-un astfel de model, adversarul poate observa traficul, poate încerca să modifice pachetele sau poate injecta pachete false. Pentru a contracara aceste amenințări, IPsec oferă servicii precum confidențialitatea datelor, autentificarea originii pachetelor, integritatea datelor și protecția împotriva reluării unor pachete vechi. Aceste servicii sunt aplicate direct asupra pachetelor IP, fără ca aplicațiile să fie nevoite să implementeze separat mecanisme criptografice [3], [4].

Din punct de vedere funcțional, IPsec poate fi privit ca având două componente mari. Prima componentă este planul de date, responsabil cu protejarea efectivă a traficului IP. A doua componentă este planul de control, responsabil cu stabilirea parametrilor de securitate și a cheilor criptografice necesare pentru protejarea traficului. În această lucrare, planul de date este asociat în principal cu IPsec-ESP, iar planul de control este asociat cu IKEv2 [1], [3], [4].

2.2 Security Associations și organizarea comunicației IPsec

Pentru ca două entități să poată comunica securizat folosind IPsec, acestea trebuie să partajeze anumite informații criptografice și operaționale. Aceste informații sunt organizate sub forma unor SA-uri. O asociere de securitate descrie serviciile aplicate traficului, algoritmi criptografici utilizați, cheile folosite, modul de funcționare și alți parametri necesari procesării pachetelor [3], [4]. O caracteristică importantă este faptul că o asociere de securitate IPsec este, în mod conceptual, unidirecțională. Pentru comunicații bidirecționale sunt necesare asocieri de securitate separate pentru fiecare direcție. De exemplu, dacă două gateway-uri comunică între ele, unul dintre SA-uri protejează traficul de la primul gateway către al doilea, iar celălalt protejează traficul în sens invers [1], [3].

Asocierile de securitate sunt identificate prin parametri precum SPI (Security Parameters Index). Acest identificator permite receptorului să determine ce chei și ce algoritmi trebuie utilizați pentru procesarea unui anumit pachet IPsec. În lipsa unei astfel de stări comune, receptorul nu ar putea ști cum trebuie verificat sau decriptat pachetul primit [3], [4]. Stabilirea manuală a acestor asocieri de securitate ar fi dificilă și nescalabilă, mai ales în rețele mari sau în scenarii VPN (Virtual Private Network) cu multe site-uri. Din acest motiv, IPsec folosește un protocol specializat pentru managementul acestor asocieri, IKEv2. Rolul acestuia este de a autentifica participanții, de a negocia parametrii necesari și de a stabili cheile secrete folosite ulterior de ESP sau AH (Authentication Header) [3].

2.3 Protocoalele IPsec: AH și ESP

Suita IPsec include două mecanisme principale pentru protejarea traficului: AH și ESP. AH este orientat către autentificarea și integritatea pachetelor IP, însă nu oferă confidențialitate. ESP este mai flexibil și poate oferi confidențialitate, integritate, autentificarea originii datelor și protecție anti-replay [1], [4].

ESP poate fi folosit în două moduri principale: transport mode și tunnel mode. În transport mode, ESP protejează conținutul pachetului IP, păstrând antetul IP original vizibil pentru rutare. Acest mod este folosit mai ales pentru comunicații directe între gazde sau pentru protejarea unor tuneluri deja existente, cum ar fi GRE (Generic Routing Encapsulation). În tunnel mode, întregul pachet IP original este încapsulat într-un nou pachet IP, iar ESP protejează pachetul interior. Acest mod este foarte utilizat în VPN-uri site-to-site, unde traficul dintre două rețele private este transportat printr-o rețea publică [1], [4].

În formatul unui pachet ESP apar mai multe câmpuri. Câmpul SPI identifică asocierea de securitate utilizată. Câmpul Sequence Number este folosit pentru protecția anti-replay. Câmpul Payload Data conține datele protejate, iar trailerul ESP include informații necesare pentru aliniere și identificarea protocolului următor. În final, câmpul Integrity Check Value permite verificarea integrității, atunci când serviciul de integritate este activat [4].

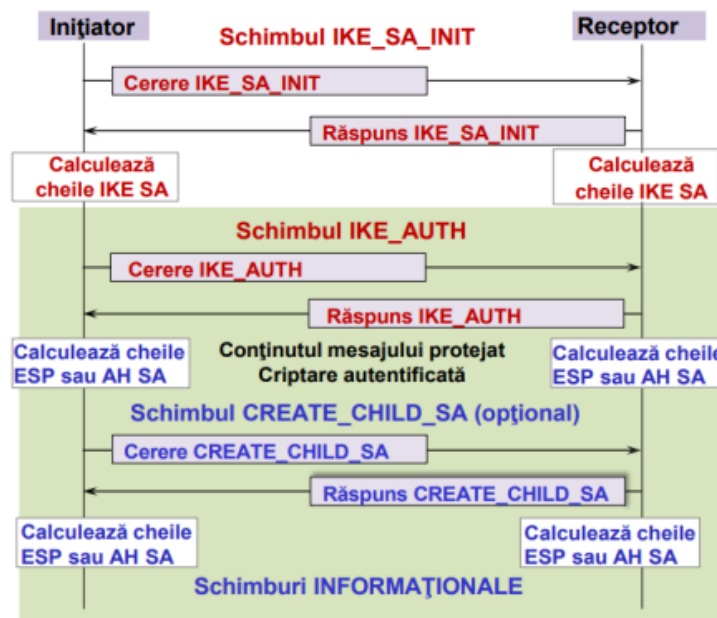


Figura 2.3: IKEv2 handshake [1]

Primul schimb, IKE_SA_INIT, are rolul de a stabili contextul criptografic inițial. În această etapă sunt transmise valorile Diffie-Hellman, nonce-urile și propunerile de algoritmi criptografici. Pe baza valorilor Diffie-Hellman și a nonce-urilor, participanții pot calcula materialul criptografic inițial necesar pentru protejarea mesajelor următoare [3].

Al doilea schimb, IKE_AUTH, are rolul de a autentifica participanții și de a finaliza stabilirea asocierii de securitate. În această etapă sunt transmise identitățile, autentificatorii și informațiile necesare pentru crearea primului Child SA, care va fi folosit de ESP sau AH pentru protejarea traficului de date. Deoarece mesajele din această etapă sunt protejate folosind cheile stabilite după IKE_SA_INIT, identitățile participanților pot fi transmise confidențial. Această proprietate este relevantă pentru mecanismul de identity hiding, adică ascunderea identităților față de observatori pasivi [1], [3], [10].

IKEv2 este inspirat de familia de protocoale SIGMA (Sign-and-Mac), care combină semnăturile digitale cu coduri de autentificare a identității, pentru a evita atacurile de tip identity misbinding și pentru a permite protejarea identităților prin criptare. În mod simplificat, ideea este că semnătura autentifică transcriptul criptografic al sesiunii, iar MAC-ul aplicat identității dovedește legătura dintre identitate și cheia de sesiune. Această structură este esențială pentru protocoalele implementate, deoarece variantele SIG-PKC și MAC-PSK trebuie să lege corect identitățile, nonce-urile, valorile Diffie-Hellman și cheile rezultate [1], [10].

2.5 Variante de autentificare în IKEv2

IKEv2 poate folosi mai multe metode de autentificare. Pentru lucrarea de față sunt relevante două familii principale: autentificarea bazată pe chei pre-partajate și autentificarea bazată pe semnături digitale și certificate de cheie publică, adică SIG-PKC [1], [3].

În varianta MAC-PSK, cele două părți dețin anterior o cheie secretă comună. Autentificarea se realizează prin calculul unor coduri MAC asupra datelor relevante ale sesiunii. Avantajul acestei metode este eficiența: operațiile MAC sunt rapide și au cost redus. Dezavantajul este scalabilitatea redusă, deoarece fiecare pereche de participanți trebuie să aibă o cheie secretă comună sau trebuie să existe o infrastructură suplimentară de distribuție a acestor secrete. În rețele mari, gestionarea cheilor pre-partajate devine dificilă [1], [7].

În varianta SIG-PKC, autentificarea se bazează pe semnături digitale și certificate. Fiecare participant are o pereche de chei publică-privată, iar cheia publică este legată de identitate printr-un certificat. Participantul semnează datele relevante ale sesiunii cu cheia sa privată, iar cealaltă parte verifică semnătura folosind cheia publică certificată. Această metodă are un cost computațional mai mare decât MAC-PSK, dar este mai scalabilă, deoarece nu necesită câte o cheie secretă comună pentru fiecare pereche de participanți [1], [2], [3], [7].

În ambele variante, Diffie-Hellman are rolul de a genera un secret comun proaspăt, dependent de contribuțiile ambelor părți. Utilizarea unor exponenți efemeri permite obținerea proprietății de forward secrecy, deoarece compromiterea ulterioară a cheilor pe termen lung nu permite recalcularea cheilor de sesiune anterioare, dacă valorile efemere au fost eliminate corect [1], [2], [3].

2.6 Nucleul criptografic

Lucrarea nu urmărește implementarea completă a protocolului IKEv2 și nici a întregii suite IPsec. O implementare completă ar trebui să includă numeroase detalii operaționale: retransmisii, tratarea erorilor, ferestre de mesaje, NAT (Network Address Translation) traversal, rekeying, negociere completă de algoritmi, payload-uri multiple și compatibilitate cu implementări reale. Aceste elemente sunt importante în practică, dar depășesc scopul proiectului. În schimb, lucrarea se concentrează pe nucleul criptografic al protocolelor. Acest nucleu include schimbul Diffie-Hellman, generarea nonce-urilor, derivarea cheilor, autentificarea mutuală prin MAC-PSK sau SIG-PKC și criptarea autentificată a datelor sensibile transmise după stabilirea cheilor. Prin această abordare, implementarea rămâne suficient de simplă pentru analiză experimentală, dar păstrează mecanismele esențiale care asigură securitatea comunicației [1], [3], [4].

Pentru partea inspirată de IKEv2 s-a avut în vedere două protocole de stabilire a cheilor: unul cu autentificare bazată pe semnături și certificate, MAKE-IKE-SIG-PKC-DHE 4M, și unul cu autentificare bazată pe MAC și cheie pre-partajată, MAKE-IKE-MAC-PSK-DHE 4M. Ambele folosesc patru mesaje și urmăresc modelul în care participanții schimbă inițial materialul necesar pentru Diffie-Hellman și nonce-urile, iar apoi transmit autentificatori protejați prin criptare autentificată [1], [3].

Pentru partea inspirată de IPsec-ESP s-a avut în vedere un protocol de tip MAKE-ESP-MAC-PSK-DHE 2M, care modelează stabilirea cheilor pentru protejarea traficului de date după existența unei sesiuni IKE. În această etapă, mesajele sunt deja transmise într-un context protejat, iar scopul este obținerea materialului criptografic necesar pentru transferul securizat. Ulterior, datele sunt protejate prin criptare autentificată, similar rolului îndeplinit de ESP în IPsec [1], [4].

Capitolul 3. Utilizarea platformei SJAKE pentru implementarea protocoalelor AKE

3.1 Considerații generale privind platforma SJAKE

Platforma SJAKE reprezintă mediul software utilizat pentru implementarea, testarea și analiza experimentală a protocoalelor de tip AKE. Rolul platformei este de a oferi o infrastructură comună pentru modelarea participanților, transmiterea mesajelor, configurarea algoritmilor criptografici și observarea execuției protocoalelor. În loc ca fiecare protocol să fie implementat complet de la zero, SJAKE pune la dispoziție componente reutilizabile pentru comunicație, mesaje, primitive criptografice, managementul cheilor și testare. Datorita acestei platforme, implementarea se axează pe logica efectivă a protocolului (ce mesaje sunt transmise, ce valori criptografice sunt calculate, ce autentificatori sunt verificați și în ce stare se află fiecare participant).

SJAKE este potrivită, deoarece protocoalele implementate au o structură apropiată de protocoalele AKE folosite în practică, dar nu urmăresc implementarea completă a standardului IKEv2. Scopul lucrării este analiza nucleului criptografic: schimbul Diffie-Hellman, autentificarea mutuală, derivarea cheilor, criptarea autentificată și comportamentul protocolului în prezența unui adversar activ. Pentru aceste obiective, un cadru experimental precum SJAKE este mai potrivit decât o implementare completă IPsec/IKEv2, deoarece permite controlul direct asupra mesajelor, asupra stărilor protocolului și asupra variantelor vulnerabile analizate [5].

Platforma nu este folosită doar ca mediu de rulare, ci ca infrastructură experimentală completă. Protocoalele nu sunt tratate doar ca scheme abstracte, ci ca programe reale, în care mesajele sunt construite, transmise, procesate și verificate. Această abordare prezintă multe vulnerabilități ale protocoalelor criptografice ce apar la nivel de integrare. Un protocol poate folosi algoritmi siguri, dar poate deveni vulnerabil dacă nu autentifică toate datele relevante, dacă nu separă corect cheile sau dacă permite reutilizarea unui mesaj într-un alt context.

3.2 Obiectivele platformei SJAKE

SJAKE este folosită pentru trei direcții principale. Prima direcție este implementarea protocoalelor corecte de stabilire a cheilor secrete. Aceste protocoale sunt variante simplificate, inspirate din mecanismele IKEv2, și includ autentificare mutuală prin semnături și certificate, respectiv prin MAC și chei pre-partajate. A doua direcție este implementarea transferului securizat de date, folosind cheile de sesiune stabilite anterior și mecanisme de criptare autentificată, într-un model apropiat de IP-sec-ESP. A treia direcție este implementarea unor atacuri asupra variantelor vulnerabile ale protocoalelor, pentru a evidenția efectele unor decizii incorecte de proiectare [5], [6]. Testele pot fi selectate și executate în mod controlat. Pentru fiecare execuție se poate urmări schimbul de mesaje dintre participanți, starea internă a protocolului și comportamentul adversarului. Prin această trasabilitate se observă că protocoalele AKE

sunt sensibile la ordinea mesajelor și la legarea corectă a identităților de valorile criptografice schimbate. O vulnerabilitate nu apare neapărat din slăbiciunea unei primitive criptografice, ci din modul în care primitivele sunt combinate în protocol.

Platforma permite și măsurători de performanță, ceea ce face posibilă compararea variantelor bazate pe MAC-PSK și SIG-PKC. Această comparație este relevantă, deoarece cele două modele de autentificare au costuri și avantaje diferite. Autentificarea simetrică este mai rapidă, dar mai greu de scalat, în timp ce autentificarea cu semnături și certificate este mai flexibilă în scenarii distribuite, dar presupune operații criptografice mai costisitoare [7].

3.3 Arhitectura generală și componentele SJAKE

Arhitectura SJAKE este organizată în mai multe componente, fiecare având un rol clar în execuția protocoalelor. La nivel superior se află mecanismul care configurează testele, creează participanții și pornește execuția scenariilor. Fiecare participant onest este modelat ca o aplicație care rulează o instanță de protocol. În scenariile de atac, adversarul este modelat separat și poate intercepta, modifica, retransmite sau construi mesaje, în funcție de logica atacului implementat [5], [6].

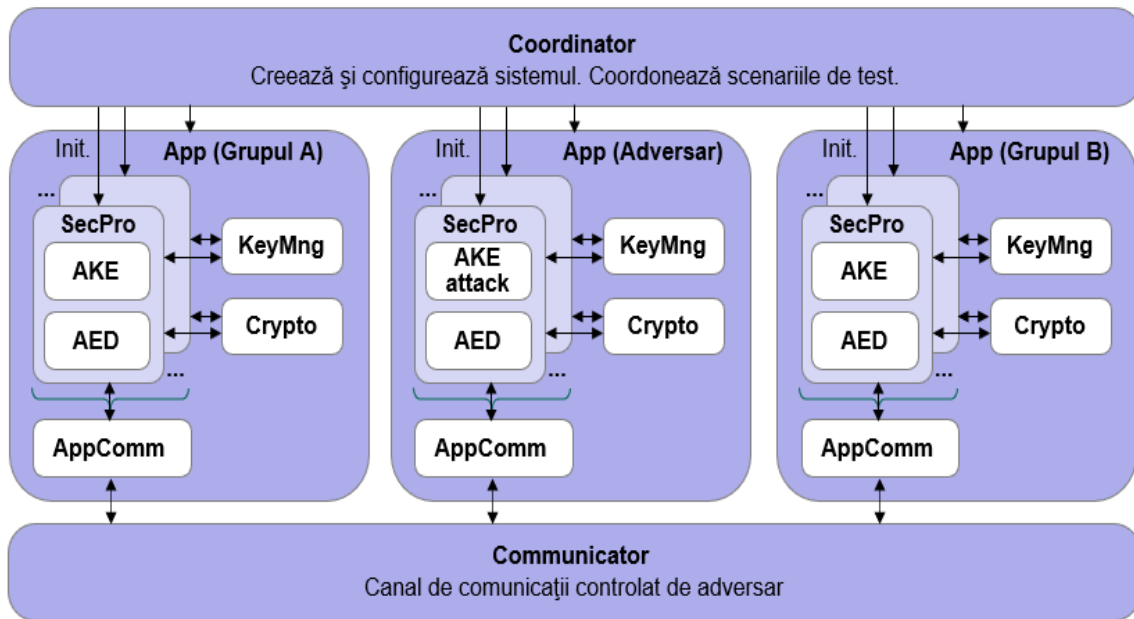


Figura 3.1 Arhitectura SJAKE [5]

Coordinator are rolul de a crea și configura sistemul de testare. Aceasta inițializează participanții, adversarul, componenta de comunicație și configurațiile criptografice necesare. Într-un scenariu experimental, Coordinator-ul stabilește ce protocol se execută, ce participanți sunt implicați, dacă testul este unul normal sau unul de atac și ce parametri criptografici sunt folosiți. Din acest motiv, Coordinator-ul poate fi privit ca punctul de pornire al testului, adică modulul care construiește mediul controlat în care protocolul este rulat [5].

Communicator modelează canalul de comunicație dintre participanți. Într-un sistem real, mesajele ar circula printr-o rețea, unde adversarul ar putea intercepta sau modifica traficul. În SJAKE, această situație este modelată software: mesajele sunt transmise între obiecte Java, iar canalul poate fi controlat de adversar. În varianta simplificată SJAKE, participanții și comunicatorul rulează în același proces, folosind fire de execuție și cozi de mesaje, nu conexiuni TCP (Transmission Control Protocol) reale. Această alegere simplifică testarea și permite observarea clară a execuției protocolului [5].

AppComm reprezintă interfața prin care o aplicație sau o instanță de protocol trimite și primește mesaje. Practic, protocolul nu comunică direct cu Communicator, ci prin AppComm. Această separare este utilă deoarece logica protocolului rămâne independentă de detaliile concrete ale livrării mesajelor. În implementările protocoalelor, obiectul AppComm este transmis către clasa de protocol și este folosit pentru trimiterea mesajelor de tip InitMsg, AuthMsg, InitAuthMsg sau FailMsg [5].

SecPro reprezintă nivelul de protocol de securitate. Aceasta include implementarea logicii AKE și a transferului securizat de date. În cazul protocoalelor implementate în această lucrare, clasele de protocol extind o clasă de bază comună, care conține variabile și metode reutilizabile pentru mai multe construcții AKE. Acest nivel se ocupă de stările protocolului, de procesarea mesajelor primite, de generarea mesajelor transmise și de apelarea componentelor criptografice necesare [5].

KeyMng are rolul de management al cheilor. Ea gestionează cheile pe termen lung, cheile pre-partajate, certificatele și informațiile necesare identificării participanților. În variantele bazate pe MAC-PSK, KeyMng oferă acces la secretul pre-partajat folosit pentru autentificare. În variantele bazate pe SIG-PKC, componenta este legată de accesul la certificate, chei private și chei publice necesare semnării și verificării autentificatorilor. Prin această separare, protocolul nu trebuie să gestioneze direct toate detaliile de stocare și încărcare a materialului criptografic [5].

Crypto oferă primitivele criptografice utilizate de protocoale. Aici intră algoritmi pentru acord Diffie-Hellman sau ECDH (Eliptic Curve Diffie-Hellman), algoritmi MAC, semnăturile digitale, verificarea certificatelor, derivarea cheilor și criptarea autentificată. În SJAKE, aceste operații sunt implementate folosind bibliotecile criptografice standard Java, ceea ce permite testarea unor algoritmi reali, nu doar a unor modele abstracte [5], [8].

AKE este partea din SecPro care realizează efectiv protocolul de stabilire a cheii de sesiune. Ea construiește mesajele de inițializare, procesează valorile Diffie-Hellman, generează nonce-uri, derivează chei și verifică autentificatorii. După finalizarea cu succes a protocolului AKE, cheile rezultate sunt folosite de componenta de transfer securizat de date, asociată în arhitectură cu AEAD, adică mecanismul de criptare autentificată a datelor [5].

3.4 Modelul de comunicație și adversarul

Un aspect notabil al platformei este faptul că execuția protocoalelor are loc într-un model de comunicație controlat de adversar. Acest model corespunde ipotezelor obișnuite din analiza protocoalelor criptografice: adversarul poate citi mesajele, le poate întârzia, modifica, șterge, retransmite sau poate introduce mesaje noi. Participanții onești rulează protocolul conform specificației, în timp ce

adversarul poate devia de la protocol pentru a încerca impersonarea unei entități sau stabilirea unei chei incorect asociate unei identități [1], [6].

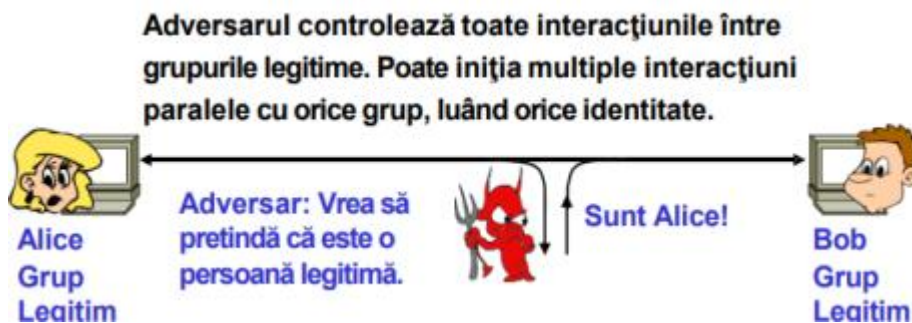


Figura 3.2 Modelul adversarului care controlează comunicațiile [1]

În SJAKE, acest model este implementat prin componenta Communicator și prin aplicația adversarului. Communicator este canalul prin care trec mesajele, iar adversarul poate fi plasat logic între participanți, pentru a observa sau manipula fluxul protocolului. Spre deosebire de o rețea reală, unde analiza traficului ar fi mai dificilă, în SJAKE fiecare mesaj este vizibil în trace, împreună cu tipul său, expeditorul, destinatarul, identificatorul de sesiune și conținutul relevant.

Într-un astfel de model, securitatea protocolului nu poate fi redusă la securitatea algoritmilor criptografici individuali. Chiar dacă Diffie-Hellman, MAC-ul, semnătura digitală sau criptarea autenticată sunt corecte ca primitive, protocolul poate rămâne vulnerabil dacă autentificatorii nu includ toate datele relevante sau dacă identitățile nu sunt legate corect de transcriptul sesiunii. Din acest motiv, SJAKE este utilă nu doar pentru implementarea protocolelor corecte, ci și pentru construirea variantelor vulnerabile și a atacurilor care le exploatează.

3.5 Modelul generic al protocolelor AKE și tipurile de mesaje

În acest model, participanții urmăresc autentificarea entităților și stabilirea unei chei de sesiune secrete. Autentificarea asigură faptul că fiecare participant obține garanții privind identitatea celuilalt, iar stabilirea cheii permite obținerea unui secret comun care poate fi folosit ulterior pentru protejarea datelor [1], [2].

În cadrul platformei SJAKE sunt definite mai multe familii de protocole AKE, în funcție de tipul autentificării și de numărul de mesaje. Pentru această lucrare, familia relevantă este MAKE. Un protocol MAKE este un protocol de stabilire a cheilor cu autentificare mutuală, ceea ce înseamnă că ambele părți trebuie să se autentifice și să asocieze cheia de sesiune rezultată cu identitatea corectă a partenerului de comunicație [5], [6]. La nivel generic, un protocol MAKE folosește mesaje de inițializare și mesaje de autentificare. Mesajele de inițializare transportă datele necesare pentru stabilirea materialului criptografic comun. În protocolele bazate pe Diffie-Hellman, aceste mesaje includ nonce-urile participanților și valorile publice DH sau ECDH (Elliptic Curve Diffie-Hellman). Mesajele de

autentificare transportă dovezile criptografice prin care participanții își demonstrează identitatea și autentifică datele relevante schimbate anterior.

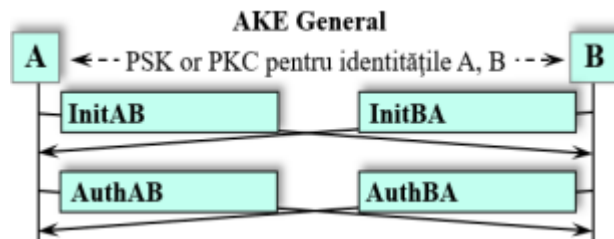


Figura 3.3 Structura generică a protocoloelor AKE [5]

Primele două mesaje formează etapa de inițializare. În această etapă se transmit nonce-urile, valorile publice Diffie-Hellman și, în variantele inspirate de IKEv2, parametrii criptografici necesari sesiunii. După aceste mesaje, ambele părți pot calcula secretul Diffie-Hellman și pot deriva cheile necesare pentru protejarea datelor de autentificare.

Ultimele două mesaje formează etapa de autentificare. Mesajul AuthAB conține autentificatorul inițiatorului, iar mesajul AuthBA conține autentificatorul responderului. În funcție de protocol, autentificatorul poate fi calculat fie prin MAC folosind o cheie pre-partajată, fie prin semnătură digitală folosind cheia privată și certificatul participantului. În variantele inspirate de IKEv2, identitățile și autentificatorii sunt protejați prin criptare autentificată, după ce cheile de sesiune au fost derivate [1], [3], [5].

SJAKE permite și variante MAKE cu 3 mesaje, în care mesajul de inițializare al responderului este combinat cu autentificatorul său. Această variantă reduce numărul de mesaje și schimbă ordinea autentificării: responderul este autentificat înaintea inițiatorului. În lucrarea de față, această formă nu este implementată, aceasta face parte din clasificarea generală a protocoloelor AKE disponibile în platforma [5].

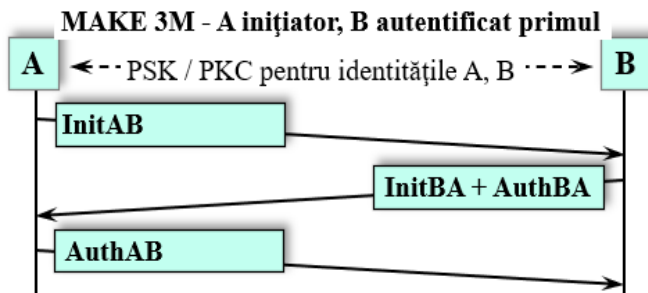


Figura 3.4 Structura MAKE cu 3 mesaje [5]

Pe lângă protocoloalele MAKE, SJAKE include și variante UAKE (Unilateral Authenticated Key Exchange), în care doar una dintre părți este autentificată explicit. Aceste variante nu reprezintă obiectul implementării din această lucrare, deoarece protocoloalele analizate urmăresc autentificarea mutuală. Totuși, ele sunt relevante pentru clasificarea generală a platformei: în timp ce MAKE oferă autentificare pentru ambele părți, UAKE oferă autentificare doar pentru una dintre ele [5].

În implementările realizate, apar și mesaje combinate, în special în protocolul MAKE-ESP-MAC-PSK-DHE 2M. Acest protocol are o structură mai scurtă decât variantele MAKE-IKE cu patru mesaje, deoarece este folosit într-un context în care există deja material criptografic sau o relație de securitate anterioară, similară ideii de sesiune IKE stabilită înaintea protejării traficului ESP. Din acest motiv, mesajele pot combina datele de inițializare cu datele autentificate, reducând numărul de fluxuri necesare pentru obținerea cheilor folosite la transferul securizat de date.

3.6 Automatul de stare pentru protocoalele MAKE

Protocoalele MAKE sunt controlate printr-un automat finit de stare, deoarece fiecare participant trebuie să proceseze mesajele într-o ordine strictă. Automatul nu descrie doar ce mesaje se schimbă între participanți, ci și în ce stare se află fiecare parte după transmiterea sau primirea unui mesaj. Această reprezentare validează ideea că un mesaj valid din punct de vedere criptografic poate fi considerat invalid dacă este primit într-o stare greșită a protocolului.

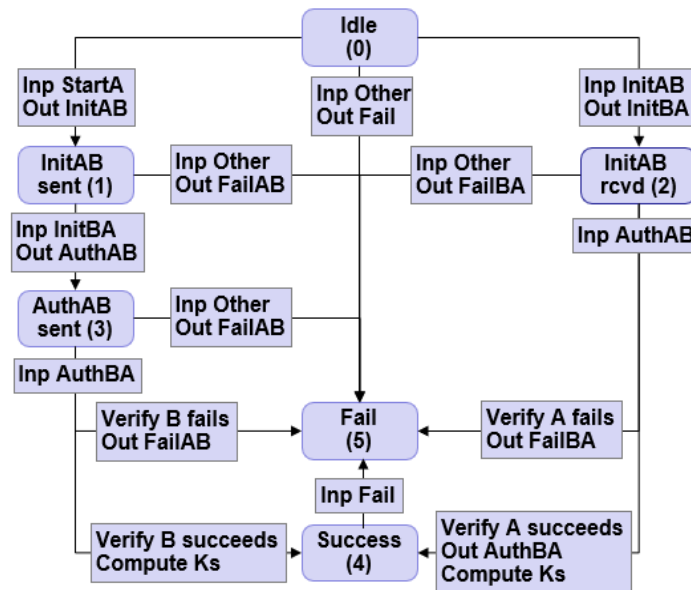


Figura 3.5 FSM MAKE 4M [5]

Pentru protocoalele MAKE cu 4 mesaje, automatul generic conține șase stări principale. În implementarea SJAKE, acestea corespund stărilor logice prin care trece o sesiune de schimb de chei: stare inițială, stări intermediare pentru mesajele trimise sau primite, stare de succes și stare de eșec [5].

Starea Idle este starea inițială a protocolului. În această stare, participantul nu a început încă schimbul de chei și nu a primit niciun mesaj relevant pentru sesiunea curentă. Dacă participantul are rol de inițiator, evenimentul de pornire a protocolului determină construirea și transmiterea mesajului InitAB. După această acțiune, inițiatorul trece în starea InitAB sent. Dacă participantul primește în starea Idle un mesaj InitAB, atunci el preia rolul de responder, procesează datele primite și transmite răspunsul Init-BA, trecând în starea InitAB rcvd.

Starea InitAB sent este starea în care se află inițiatorul după ce a transmis primul mesaj al protocolului. În această stare, inițiatorul a generat deja valorile sale locale, de exemplu nonce-ul și cheia publică Diffie-Hellman, și așteaptă mesajul InitBA de la responder. Dacă primește mesajul așteptat, inițiatorul poate calcula secretul Diffie-Hellman, poate deriva cheile de sesiune și poate pregăti autentificatorul propriu. După trimiterea mesajului AuthAB, inițiatorul trece în starea AuthAB sent. Dacă în această stare primește un alt tip de mesaj decât cel așteptat, protocolul este tratat ca invalid și se ajunge în starea de eșec.

Starea InitAB rcvd este starea responderului după ce acesta a primit mesajul inițial InitAB și a transmis mesajul InitBA. În această stare, responderul a generat propriile valori criptografice și așteaptă autentificatorul inițiatorului, adică mesajul AuthAB. Când acest mesaj este primit, responderul verifică autentificatorul lui A. Dacă verificarea eșuează, responderul transmite un mesaj de eșec și sesiunea se oprește. Dacă verificarea reușește, responderul acceptă autentificarea inițiatorului, construiește propriul autentificator AuthBA și îl transmite către inițiator. După această transmitere, responderul poate considera protocolul finalizat cu succes, deoarece în varianta MAKE 4M inițiatorul este autentificat primul.

Starea AuthAB sent este starea inițiatorului după ce acesta a transmis mesajul său de autentificare către responder. În această stare, inițiatorul așteaptă ultimul mesaj al protocolului, AuthBA. Acest mesaj reprezintă dovada criptografică a identității responderului. Dacă AuthBA este valid, inițiatorul acceptă sesiunea și trece în starea Success. Dacă verificarea autentificatorului responderului eșuează sau dacă este primit un mesaj neașteptat, sesiunea trece în starea Fail.

Starea Success marchează finalizarea corectă a protocolului. În această stare, participantul a verificat autentificatorul necesar și consideră că cheia de sesiune este stabilită cu partenerul corect. Pentru inițiator, succesul apare după verificarea mesajului AuthBA. Pentru responder, succesul apare după verificarea mesajului AuthAB și transmiterea propriului AuthBA. Din acest moment, cheia de sesiune poate fi folosită pentru protejarea datelor prin criptare autentificată.

Starea Fail marchează oprirea protocolului din cauza unei erori sau a unei verificări nereușite. În această stare se ajunge dacă participantul primește un mesaj neașteptat, dacă verificarea MAC-ului sau a semnăturii eșuează, dacă apare o eroare de procesare criptografică sau dacă este primit explicit un mesaj de tip Fail. Starea de eșec este esențială pentru securitate, deoarece participantul nu trebuie să continue protocolul după o verificare invalidă și nu trebuie să accepte o cheie de sesiune într-un context incert.

În diagrama FSM, apar și tranziții marcate prin Other input message. Acestea reprezintă orice mesaj care nu este mesajul așteptat în starea curentă. De exemplu, dacă inițiatorul se află în starea InitAB sent, el așteaptă InitBA; orice alt mesaj primit în această stare este tratat ca intrare invalidă. În mod similar, dacă responderul se află în starea InitAB rcvd, el așteaptă AuthAB; primirea unui alt mesaj determină trimiterea unui mesaj Fail și oprirea sesiunii. Această regulă împiedică adversarul să forțeze execuții neprevăzute ale protocolului prin injectarea sau reordonarea mesajelor.

În implementarea concretă, aceste stări sunt reflectate în cod prin valori precum START, INIT_SENT, INIT_RCVD, AUTH_SENT, ESTABLISHED și FAILED. Starea START corespunde stării inițiale Idle. Starea INIT_SENT corespunde inițiatorului care a trimis InitAB și așteaptă InitBA. Starea INIT_RCVD corespunde responderului care a primit InitAB, a trimis InitBA și așteaptă AuthAB.

Starea AUTH_SENT corespunde inițiatorului care a trimis AuthAB și așteaptă AuthBA. Stările ESTABLISHED și FAILED corespund finalizării cu succes, respectiv finalizării cu eșec a protocolului [5].

Pentru protocoalele implementate în această lucrare, automatul MAKE 4M este relevant în special pentru MAKE-IKE-SIG-PKC-DHE 4M și MAKE-IKE-MAC-PSK-DHE 4M. Ambele folosesc aceeași structură de stări, dar diferă prin operațiile criptografice executate în tranziții. În varianta SIG-PKC, verificările din tranzițiile de autentificare presupun semnături digitale și certificate. În varianta MAC-PSK, aceleași tranziții folosesc coduri MAC calculate pe baza unei chei pre-partajate. Astfel, FSM-ul rămâne același, iar diferențele apar la nivelul conținutului mesajelor și al mecanismelor criptografice apelate.

Pentru protocolul MAKE-ESP-MAC-PSK-DHE 2M, automatul este mai scurt, deoarece schimbul are doar două mesaje. Acest protocol nu folosește întreaga structură MAKE-IKE cu patru mesaje, ci modelează o etapă mai compactă de stabilire a materialului criptografic pentru transfer securizat de date. Din acest motiv, stările intermediare sunt mai puține, iar mesajele pot include simultan date de inițializare și date protejate criptografic. Funcționarea concretă a acestui protocol va fi detaliată în Capitolul 4, unde sunt prezentate protocoalele implementate efectiv.

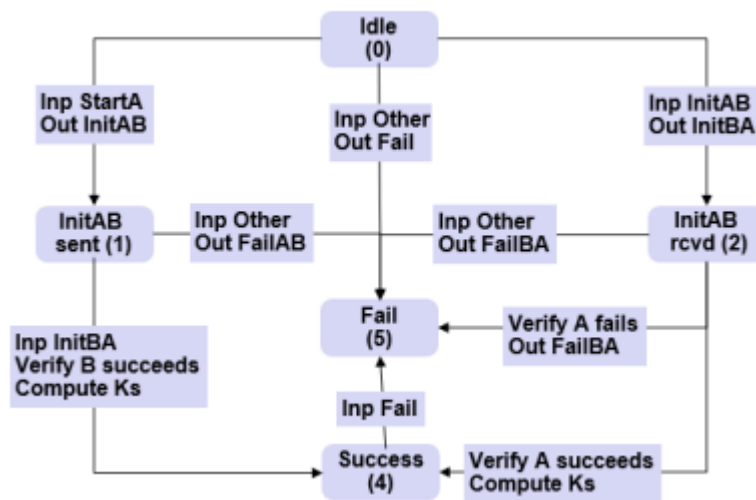


Figura 3.6 FSM MAKE 2M

3.7 Modelul de testare și trasabilitatea execuției

În platforma SJAKE implementarea unui protocol nu este testată doar prin apelarea unor metode din cod. Platforma oferă un model de testare organizat, în care utilizatorul poate selecta scenariul executat, protocolul analizat, configurația criptografică și nivelul de detaliu al informațiilor afișate în timpul execuției [5], [6].

Meniul de testare are rolul de a simplifica alegerea experimentului. În loc ca utilizatorul să modifice direct codul pentru fiecare rulare, platforma grupează testele în categorii și permite selectarea unui scenariu concret. Pot fi rulate teste funcționale pentru protocoale corecte, teste pentru variante

vulnerabile și teste de atac, în care apare și un adversar. Această organizare este utilă mai ales atunci când există mai multe protocoale asemănătoare, dar cu proprietăți diferite, cum este cazul variantelor MAC-PSK, SIG-PKC și ESP-MAC-PSK.

În cadrul meniului, primul nivel de selecție este reprezentat de categoria de teste. Aceasta grupează scenarii apropiate, de exemplu teste pentru protocoale MAKE, teste pentru protocoale vulnerabile sau teste de atac asupra unor protocoale MAKE. Prin această împărțire, utilizatorul nu trebuie să caute într-o listă foarte mare de opțiuni, ci pornește de la tipul de experiment pe care vrea să îl execute. Acest lucru face platforma mai ușor de folosit și mai potrivită pentru analiză experimentală.

După alegerea categoriei, se selectează testul propriu-zis. Un test definește scenariul de execuție: ce protocol se rulează, câți participanți sunt implicați și dacă execuția conține sau nu un adversar. De exemplu, un test funcțional cu doi participanți onești verifică dacă Ana și Bob pot finaliza protocolul și pot stabili chei de sesiune comune. În schimb, un test de atac introduce adversarul în fluxul de comunicație și urmărește dacă acesta poate exploata o variantă vulnerabilă a protocolului.

Exemplu de log pentru meniul de testare:

```
Select test set. Test set options:
```

```
[0] TESTSET_MAKE
[1] TESTSET_UAKE
[2] TESTSET_ATK_MAKE
[3] TESTSET_VUL_MAKE
[4] TESTSET_MESH
```

```
Enter test set index [0]: 0
```

```
Select test. Tests in set TESTSET_MAKE
```

```
[0] TEST_2P_MAKE_SIG_DHE_4M
[1] TEST_2P_MAKE_IKE_SIG_PKC_DH_4M
[2] TEST_2P_MAKE_IKE_MAC_PSK_DH_4M
[3] TEST_2P_MAKE_ESP_MAC_PSK_DH_2M
[4] TEST_2P_MAKE_MAC_M3
[5] TEST_2P_MAKE_MAC_M4
```

```
Enter test index [0]: 1
```

```
Crypto options for test TEST_2P_MAKE_IKE_SIG_PKC_DH_4M
```

```
[0] ECDH_ECDSA_256_CTR_128
[1] ECDH_ECDSA_384_CTR_192
[2] ECDH_ECDSA_521_CTR_256
[3] XDH_EdDSA_256_CTR_128
[4] DH_RSA_2048_CTR_128
[5] DH_RSA_3072_CTR_128
[6] DH_8192_RSA_7680_CTR_192
[7] DH_DSA_2048_CTR_128
[8] DH_DSA_3072_CTR_128
```

```
Enter crypto configuration index [0]: 2
```

Test goals:

- [0] Trace protocol
- [1] Trace protocol more
- [2] Measure time

Enter test goal [0]: 1

TestAke2P: TEST_2P_MAKE_IKE_SIG_PKC_DH_4M; prot MAKE_IKE_SIG_PKC_DH_4M

Common crypto: autSec: HMACSHA512[256]; encSec: AES/CTR/NoPadding[256];

dh: ECDH[521]; autPub: SHA512withECDSA[521];pkcSig: SHA512withECDSA[0];

Meniul de testare oferă și posibilitatea de a alege configurația criptografică. Pentru fiecare protocol sunt disponibile doar opțiunile relevante pentru construcția respectivă. De exemplu, pentru un protocol bazat pe SIG-PKC se pot selecta combinații care includ algoritmi pentru Diffie-Hellman, semnături digitale, certificate și criptare autenticată. Pentru un protocol bazat pe MAC-PSK, configurația urmărește în principal algoritmi simetrici, mecanismul MAC, schimbul Diffie-Hellman și protecția datelor [5]. Această selecție ne oferă și oportunitatea de a analiza performanța protocolului implementat. Același protocol poate fi rulat cu mai multe configurații criptografice, de exemplu cu ECDH P-256 sau cu DH 3072. Prin repetarea testelor pe configurații diferite se poate observa influența algoritmilor asupra timpului de execuție. În acest mod, meniul de testare nu este folosit doar pentru verificarea funcțională, ci și pentru comparații experimentale între variante criptografice.

După selectarea testului, platforma pornește execuția și afișează trace-ul protocolului. Acesta reprezintă o urmă cronologică a execuției și include informațiile despre participanții implicați, identificatorul sesiunii, stările protocolului, mesajele trimise și primite, precum și rezultatul operațiilor criptografice relevante. Prin trace se poate observa, de exemplu, momentul în care este transmis un mesaj Init sau Auth, când este verificat un MAC sau o semnătură digitală, când este acceptat un mesaj protejat prin criptare autenticată și când protocolul ajunge în starea ESTABLISHED [5], [6].

Pentru protocoalele corecte, trace-ul confirmă execuția normală a protocolului: schimbul valorilor Diffie-Hellman, verificarea autenticatorilor, derivarea cheilor de sesiune și acceptarea mesajelor de date transmise ulterior. Testarea nu verifică doar încheierea schimbului de mesaje, ci și faptul că sesiunea rezultată poate fi folosită efectiv pentru comunicație securizată. În cazul variantelor vulnerabile și al atacurilor, trace-ul devine un instrument de analiză. Acesta arată ordinea exactă a mesajelor și modul în care adversarul intervine în execuție. Pentru atacurile de tip Interleaving, această vizualizare este importantă deoarece atacul se bazează pe interceptarea și retransmiterea mesajelor între sesiuni diferite. Fără urmărirea pas cu pas a execuției, vulnerabilitatea ar fi mai greu de explicat doar prin rezultatul final.

Capitolul 4. Implementarea protocoalelor AKE folosite de IKEv2

4.1 Considerații generale privind implementarea

Protocoalele implementate în această lucrare reprezintă variante simplificate ale mecanismelor criptografice folosite de IKEv2 și IPsec-ESP. Scopul implementării nu este reproducerea completă a standardelor, ci izolarea și testarea nucleului criptografic necesar pentru securizarea comunicațiilor: generarea valorilor aleatoare, schimbul Diffie-Hellman efemer, derivarea cheilor de sesiune, autentificarea mutuală și protejarea datelor prin criptare autentificată. Această abordare este potrivită pentru platforma SJAKE, care permite implementarea experimentală a protocoalelor AKE și analiza comportamentului acestora în scenarii controlate [5], [6].

În cadrul proiectului au fost implementate trei construcții principale:

1. MAKE-IKE-SIG-PKC-DHE 4M
2. MAKE-IKE-MAC-PSK-DHE 4M
3. MAKE-ESP-MAC-PSK-DHE 2M

Primele două protocoale folosesc aceeași structură generală cu patru mesaje, bazată pe automatul MAKE prezentat în Capitolul 3. Diferența dintre ele apare în mecanismul de autentificare: primul folosește semnături digitale și certificate, iar al doilea folosește coduri MAC și chei pre-partajate. Al treilea protocol are două mesaje și este orientat către obținerea materialului criptografic utilizat pentru transfer securizat de date, într-un model inspirat de ESP.

Implementarea fiecărui protocol folosește infrastructura comună SJAKE: mesajele sunt transmise prin AppComm, stările sunt reprezentate prin AkeState, configurația criptografică este preluată din CryptoConfig, iar cheile și certificatele locale sunt accesate prin OwnCrypto și componentele de management al cheilor. Clasele complete implementate vor fi listate în anexe; în acest capitol sunt prezentate doar principiile de implementare și fragmentele relevante pentru înțelegerea funcționării protocoalelor.

Pentru a evidenția integrarea celor trei protocoale în aceeași infrastructură, va fi introdus un fragment scurt cu anteturile claselor Java. Acest fragment arată că toate protocoalele extind clasa de bază SecProBase, iar diferențele apar în metodele care construiesc mesajele, generează autentificatorii și derivează cheile.

```
public class MakeSigIKE extends SecProBase{
    ...
    public MakeSigIKE(AppComm appComm, String locId, byte[] sessionId,
        CryptoConfig commonCrypto, OwnCrypto ownCrypto) throws Exception
    {...}
    ...
}
```

4.2 Implementarea protocolului MAKE-IKE-SIG-PKC-DHE 4M

Protocolul MAKE-IKE-SIG-PKC-DHE 4M implementează varianta cu autentificare bazată pe semnături digitale și certificate X.509. Funcțional, protocolul folosește patru mesaje: două mesaje de inițializare și două mesaje de autentificare. Mesajele de inițializare transportă nonce-urile, valorile Diffie-Hellman și configurația criptografică locală, iar mesajele de autentificare transportă certificatul, semnătura digitală și date inițiale protejate prin criptare autenticată.

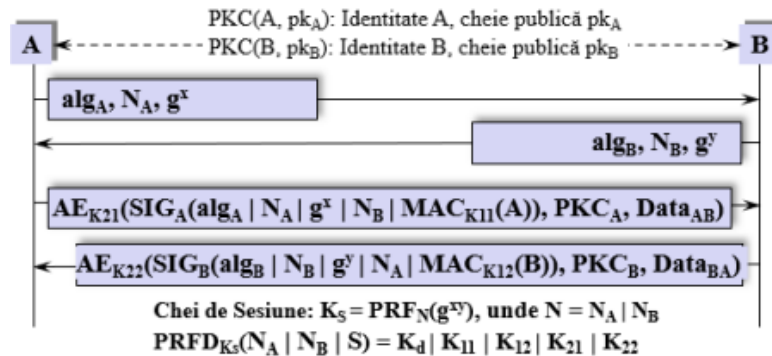


Figura 4.1 Implementarea Protocolului MAKE-IKE-SIG-PKC-DHE 4M

Presupunem că participantul A, Ana, inițiază protocolul cu participantul B, Bob. În prima etapă, Ana generează nonce-ul local N_A și o pereche efemeră Diffie-Hellman. Dacă folosim notația clasică DH, Ana alege exponentul privat X și calculează cheia publică g^X .

În implementare, exponentul privat nu este manipulat direct în codul protocolului, ci este ascuns în obiectul `KeyPair` generat prin `KeyAgrDH.genDHKeyPair()`. Cheia publică g^X este obținută prin `locDHKeyPair.getPublic().getEncoded()` și este introdusă în mesajul `INIT_A`.

La pornirea protocolului, inițiatorul setează identitatea partenerului, marchează rolul de inițiator, pregătește datele inițiale care vor fi transmise ulterior protejat, generează nonce-ul local și perechea efemeră Diffie-Hellman, apoi trece în starea `INIT_SENT` și transmite primul mesaj de inițializare. În implementare, această etapă este realizată prin metoda `doKeyExch()`.

```
/**
 * Initiator starts handshakes.
 */
public void doKeyExch(String locId, String remId) throws Exception
{
    // sw.start(); // start stopwatch for initiator
    // InitMsg handshake
    if (state == AkeState.START) {
        this.remId = remId;
        initiator = true;
        locData = TestConst.msg10.getBytes();
    }
}
```

```

        //generare nonce si DH
        generateNonceAndDHKeys();
        state = AkeState.INIT_SENT;
        appComm.sendInitMsg(buildInitMsg());
    }
}

```

Generarea valorilor proaspete este realizată prin metoda `generateNonceAndDHKeys()`. Aceasta generează un nonce de 256 de biți și perechea efemeră Diffie-Hellman. Prin această metoda obținem valorile, care vor determina secretul de sesiune final.

```

/**
 * Generates local protocol fresh values:
 * - random local nonce
 * - local ephemeral DH key pair
 */

private void generateNonceAndDHKeys() throws Exception
{
    SecureRandom rand=new SecureRandom();
    locNonce= new byte[nonceLen/8];
    rand.nextBytes(locNonce);
    locDHKeyPair=KeyAgrDH.genDHKeyPair(commonCrypto.getDhName(),
commonCrypto.getDhKeyLen());
    KeyAgrDH.printDHPublicKey("[KeyEx: "+locId+"] DH public key:/n",
locDHKeyPair.getPublic());
}

```

Mesajul `INIT_A` este construit în metoda `buildInitMsg()`. Acesta conține tipul mesajului, identificatorii participanților, identificatorul sesiunii, nonce-ul local `NA`, cheia publică Diffie-Hellman `X` și configurația criptografică locală `algA`.

```

/**
 * Builds InitMsg containing the sender's alg, nonce and DH public key.
 * Variant with 4 messages: InitMsgA, InitMsgB, AuthMsgA, AuthMsgB.
 */

protected InitMsg buildInitMsg() throws Exception
{
    Utils.trace("[KeyEx: "+locId+"] build InitMsg to " + remId + ", new state " +
state);

    // InitMsg contains local alg, local nonce and local ephemeral DH public key
    MsgType type = isInitiator()? MsgType.INIT_A : MsgType.INIT_B;
    return new InitMsg(type, locId, remId, sessionId,
        locNonce,
        locDHKeyPair.getPublic().getEncoded(),
        null,

```

```
locAlg.getBytes()); } }
```

Când Bob primește INIT_A, acesta extrage N_A , cheia publică g^X și algoritmul propus. Apoi își generează propriul nonce N_B și propria pereche efemeră Diffie-Hellman. În notație matematică, Bob alege exponentul privat Y și calculează g^Y .

În implementare, această etapă corespunde ramurii case START din metoda `recvInitMsg()`. Responderul memorează valorile primite, verifică dacă algoritmul primit corespunde algoritmului local, generează propriile valori și transmite INIT_B.

```
public byte[] recvInitMsg(InitMsg msg) throws Exception
{
    Utils.trace("[KeyEx: "+locId+"] recv InitMsg in state " + state);
    switch (state) {
        case START: // Responder receives InitReqMsg
            // Store remote alg, identity, nonce and DH public key
            ... }
}
```

După schimbul celor două mesaje de inițializare, ambele părți pot calcula același secret Diffie-Hellman. Ana folosește cheia publică g^Y primită de la Bob și exponentul său privat X : $S_A = (g^Y)^X = g^{XY}$, iar Bob folosește cheia publică g^X primită de la Ana și exponentul său privat Y : $S_B = (g^X)^Y = g^{XY}$, rezultă că: $S_A = S_B = S = g^{XY}$. În implementare, calculul secretului comun este realizat prin apelul: `KeyAgrDH.genDHSecret(dhName, locDHKeyPair, remDHPubBytes)`.

În starea INIT_SENT, inițiatorul primește mesajul INIT_B, memorează N_B , cheia publică g^Y și algoritmul responderului, verifică algoritmul, calculează secretul DH, derivează cheile de sesiune și apoi transmite AUTH_A.

```
public byte[] recvInitMsg(InitMsg msg) throws Exception
{
    ...
    case INIT_SENT: // Initiator receives InitRspMsg
        // Store remote alg, nonce and DH public key
    ... }
}
```

Cheile de sesiune sunt generate în metoda `genSessionKeys()`. Această metodă pornește de la secretul DH obținut de cei doi participanți și de la nonce-urile N_A și N_B , care oferă prospețime execuției curente. Secretul DH nu este folosit direct ca cheie finală, ci este prelucrat prin metoda `extract-then-expand`, folosită pentru derivarea controlată a materialului criptografic. În prima etapă a metodei, `extract`, secretul DH este combinat cu un seed, pentru a obține o cheie intermediară. În a doua etapă, `expand`, această cheie intermediară este extinsă într-un număr suficient de octeți pentru toate cheile necesare protocolului. Astfel, dintr-un singur secret comun se obține material criptografic separat pentru mai multe scopuri: autentificare internă, criptare autentificată și protecția mesajelor pe direcții diferite [1]. În acest protocol, seed-ul folosit la derivare este: $N = N_A \parallel N_B$. Pentru ca ambele părți să ajungă la aceeași ordine a nonce-urilor, inițiatorul construiește seed-ul ca `locNonce | remNonce`, iar responderul îl construiește ca `remNonce | locNonce`. În ambele cazuri, rezultatul logic este același: $N_A \parallel N_B$. După construirea seed-ului, implementarea aplică cele două etape: $kdk = \text{extract}(S, N)$ și $\text{randBytes} =$

`expand(kdk, N, randBytesLen)`, unde S reprezintă secretul Diffie-Hellman, iar `randBytes` reprezintă materialul de cheie obținut după derivare. În cazul acestui protocol, materialul derivat este împărțit în chei pentru autentificatorii interni și chei pentru criptarea autentificată. Cheia K_{11} este folosită pentru $MAC_{K_{11}}(A)$, iar cheia K_{12} este folosită pentru $MAC_{K_{12}}(B)$. Cheile K_{21} și K_{22} sunt folosite pentru protecția mesajelor `AUTH_A` și `AUTH_B`, prin criptare autentificată. În cod, K_{11} și K_{12} apar sub forma `locKeyAuth` și `remKeyAuth`, în funcție de rolul participantului, iar K_{21} și K_{22} sunt setate intern prin apelul `setSessionKeys(dataRandBytes)`. Această separare este importantă deoarece aceeași valoare g^{xy} nu trebuie folosită direct în mai multe mecanisme criptografice. Prin derivare, protocolul obține chei distincte pentru scopuri distincte, ceea ce face implementarea mai clară și mai sigură.

```
protected void genSessionKeys() throws Exception {
    ...
    byte[] seed = (initiator)?
        Utils.concatenate(locNonce, remNonce): // initiator
        Utils.concatenate(remNonce, locNonce); // responder
    // The PRF for key derivation is the selected MAC scheme
    SecretKey kdk = genKeyBytes.extract(secretBytesDH, seed);
    byte[] randBytes = genKeyBytes.expand(kdk, seed, randBytesLen);
    ...}
```

După derivarea cheilor, Ana construiește mesajul de autentificare `AUTH_A`. Autentificatorul este generat în metoda `generateAuth()`. Mai întâi se calculează $MAC_{K_{11}}(A)$, adică un MAC al identității locale, folosind cheia de autentificare derivată din g^{xy} . Apoi se construiește transcriptul: $algA \parallel NA \parallel g^x \parallel NB \parallel MAC_{K_{11}}(A)$. Acest transcript este semnat cu cheia privată a Anei, rezultând $SIG_A(\dots)$.

```
protected byte[] generateAuth() throws Exception {...}
```

Mesajul `AUTH_A` conține semnătura SIG_A , certificatul $PKCA$ și datele inițiale $Data_{AB}$. Aceste câmpuri sunt grupate într-un obiect `AuthData`, apoi sunt protejate prin criptare autentificată folosind cheia de trimitere asociată direcției Ana $\rightarrow$ Bob, notată în schemă cu K_{21} . În cod, rezultatul criptării este transmis prin câmpul `edata` al mesajului `AuthMsg`.

```
protected AuthMsg buildAuthMsg() throws Exception {...}
```

Când Bob primește `AUTH_A`, acesta decriptează `edata`, extrage certificatul $PKCA$, semnătura SIG_A și datele $Data_{AB}$. Pentru verificare, Bob reconstruiește transcriptul așteptat: $algA \parallel NA \parallel g^x \parallel NB \parallel MAC_{K_{11}}(A)$. Dacă certificatul este valid și semnătura se verifică, Bob acceptă autentificarea Anei și construiește propriul mesaj: $AE_{K_{22}}(SIG_B(algB \parallel NB \parallel g^y \parallel NA \parallel MAC_{K_{12}}(B)), PKCB, Data_{BA})$. Fragmentul următor arată cum mesajul de autentificare este procesat și cum succesul verificării determină trecerea în starea `ESTABLISHED`.

```
public byte[] recvAuthMsg(AuthMsg msg) throws Exception {...}
```

Verificarea semnăturii este realizată în metoda `verifyAuth()`. Înainte de verificarea semnăturii, certificatul primit este validat. Apoi cheia publică extrasă din certificat este folosită pentru verificarea semnăturii digitale.

```
protected boolean verifyAuth(byte[] auth) throws Exception {...}
```

Verificarea certificatului este realizată separat, prin metoda `verifyCert()`. Implementarea verifică certificatul față de emitentul de încredere, verifică perioada de validitate și compară numele comun din certificat cu identitatea așteptată `remId`. Acest pas este necesar, deoarece o semnătură validă criptografic este relevantă doar dacă cheia publică folosită la verificare aparține participantului așteptat.

```
private void verifyCert(X509Certificate cert) throws Exception {...}
```

Din punct de vedere al proprietăților de securitate, această implementare urmărește autentificarea mutuală prin certificate și semnături, stabilirea unei chei de sesiune proaspete prin DHE, protejarea datelor de autentificare prin criptare autenticată și separarea cheilor folosite în scopuri diferite. Proprietatea de forward secrecy este susținută de folosirea valorilor Diffie-Hellman efemere, cu condiția ca acestea să nu fie reutilizate după finalizarea sesiunii [1], [2], [6]. Codul complet se regăsește în Anexa A1.

4.3 Implementarea protocolului MAKE-IKE-MAC-PSK-DHE 4M

Protocolul MAKE-IKE-MAC-PSK-DHE 4M păstrează aceeași structură cu patru mesaje, dar înlocuiește autentificarea cu semnătură și certificat prin autentificare cu MAC și cheie pre-partajată. Din acest motiv, partea de inițializare și calculul cheilor sunt similare cu protocolul anterior, iar diferența principală apare în modul de construire și verificare a autentificatorului. Protocolul folosește acest flux:

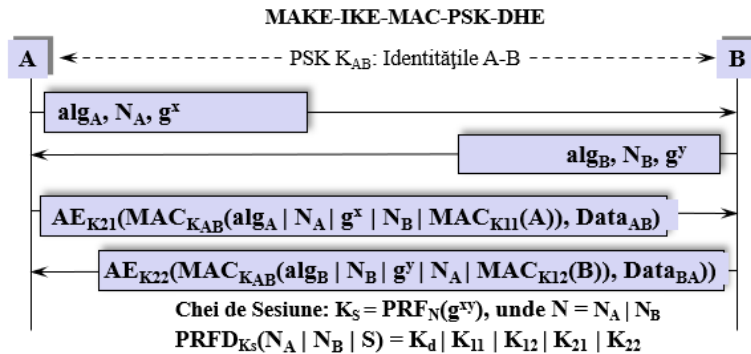


Figura 4.2 Implementarea Protocolului MAKE-IKE-MAC-PSK-DHE 4M

Mesajele de inițializare sunt construite în același mod ca în protocolul SIG-PKC: conțin algoritmul local, nonce-ul local și cheia publică DH locală. După schimbul celor două mesaje de inițializare, participanții calculează același secret: $S_A = (g^Y)^X = g^{YX}$, $S_B = (g^X)^Y = g^{XY}$, apoi derivă cheile în

aceeași formă ca în protocolul SIG-PKC: $K_S = PRF_N(g^{xy})$, unde $N = N_A \parallel N_B$ și $PRFD_{K_S} = (N_A \parallel N_B \parallel S) = K_d \parallel K_{11} \parallel K_{12} \parallel K_{22}$.

În implementare, metoda `genSessionKeys()` este aproape identică cu cea din protocolul SIG-PKC. Secretul DH este calculat cu `KeyAgrpDH.genDHSecret()`, seed-ul este construit din $N_A \parallel N_B$, iar materialul derivat este împărțit în chei pentru MAC(id) și chei pentru criptare autenticată. Cheile K_{11} și K_{12} sunt folosite pentru $MAC_{K_{11}}(A)$ și $MAC_{K_{12}}(B)$, iar K_{21} și K_{22} sunt folosite pentru protecția mesajelor de autentificare.

```
protected void genSessionKeys() throws Exception {...}
```

Elementul specific acestei variante este cheia pre-partajată K_{AB} . În implementare, aceasta este obținută prin metoda `getAuthPsk()`, care încarcă keystore-ul simetric al participantului local și extrage cheia asociată partenerului, folosind aliasul `remId + "_ake"`.

```
private SecretKey getAuthPsk() throws Exception {
    String ksFile = "keys/skeys/" + locId + ".pkcs12";
    KeyManagerKS ks = new KeyManagerKS(ksFile, SetupPSK.KS_PWD);
    ks.loadKeyStore();
    return ks.getSecretKey(remId + "_ake"); }
```

Ana își trimite autenticatorul: $MAC_{K_{AB}}(alg_A \parallel N_A \parallel g^x \parallel N_B \parallel MAC_{K_{11}}(A))$. În implementare, metoda `generateAuth()` calculează mai întâi $MAC_{K_{11}}(A)$, folosind cheia `locKeyAuth`, derivată din secretul DH. Apoi construiește transcriptul: $alg_A \parallel N_A \parallel g^x \parallel N_B \parallel MAC_{K_{11}}(A)$ și calculează MAC-ul principal folosind cheia pre-partajată K_{AB} .

În cazul lui Bob, formula este simetrică: $MAC_{K_{AB}}(alg_B \parallel N_B \parallel g^y \parallel N_A \parallel MAC_{K_{12}}(B))$. Această construcție are două niveluri: MAC(id) leagă identitatea participantului de cheia derivată din DH, iar MAC-ul principal cu K_{AB} demonstrează cunoașterea secretului pre-partajat.

```
protected byte[] generateAuth() throws Exception {...}
```

MAC-ul principal și $Data_{AB}$ sunt introduse în obiectul `AuthData`. Pentru că această variantă nu folosește certificate, primul câmp al obiectului `AuthData` este null. Apoi obiectul este protejat prin criptare autenticată și transmis în câmpul `edata`.

```
protected AuthMsg buildAuthMsg() throws Exception {...}
```

La verificare, Bob reconstruiește transcriptul așteptat: $alg_A \parallel N_A \parallel g^x \parallel N_B \parallel MAC_{K_{11}}(A)$, apoi recalculează MAC-ul cu aceeași cheie pre-partajată K_{AB} . Dacă valoarea calculată corespunde autenticatorului primit, autentificarea este acceptată, altfel sesiunea este abandonată.

```
protected boolean verifyAuth(byte[] auth) throws Exception {...}
```

La recepția mesajului `AUTH_A` sau `AUTH_B`, metoda `recvAuthMsg()` decriptează câmpul `edata`, extrage obiectul `AuthData`, verifică autenticatorul și, în caz de succes, trece protocolul în starea `ESTABLISHED`. Dacă autenticatorul nu se verifică, este trimis `FailMsg`.

```
public byte[] recvAuthMsg(AuthMsg msg) throws Exception {...}
```

Din punct de vedere al proprietăților, protocolul oferă autentificare mutuală bazată pe cunoașterea cheii K_{AB} , stabilirea unei chei de sesiune proaspete prin DHE, protejarea datelor de autentificare prin $AE_{K_{21}}$ și $AE_{K_{22}}$ și separarea cheilor derivate pe scopuri diferite. Comparativ cu varianta SIG-PKC, implementarea este mai simplă și mai rapidă, dar presupune existența anterioară a unei chei secrete comune între participanți [1], [2], [6]. Codul complet se regăsește în Anexa A2.

4.4 Implementarea protocolului MAKE-ESP-MAC-PSK-DHE 2M

Protocolul MAKE-ESP-MAC-PSK-DHE 2M are o structură diferită față de protocoalele MAKE-IKE. Acesta folosește doar două mesaje InitMsg și nu are mesaje separate de autentificare. În acest caz, valorile de inițializare sunt introduse direct în câmpul protejat eData.

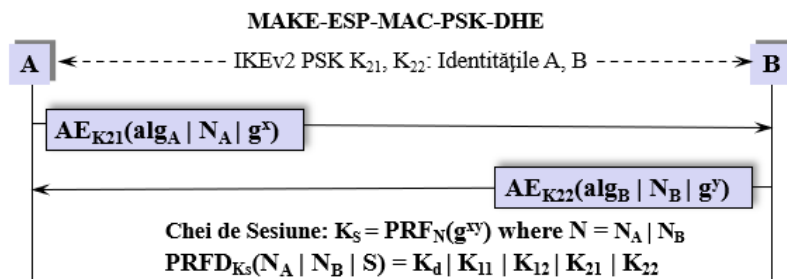


Figura 4.3 Implementarea Protocolului MAKE-ESP-MAC-PSK-DHE 2M

Protocolul este folosit într-un context în care participanții sunt considerați deja autentificați printr-o asociere de securitate anterioară. Din acest motiv, nu mai există mesaje AUTH_A și AUTH_B. Scopul protocolului este să protejeze valorile de inițializare și să deriveze un nou material criptografic pentru transferul securizat de date.

Înainte de transmiterea primului mesaj, protocolul inițializează mecanismul de criptare autenticată folosit pentru protejarea câmpului eData. În schema protocolului, această protecție este notată prin $AE_{K_{21}}$ și $AE_{K_{22}}$. În implementare, ea este realizată prin obiectul `aeWithPsk`, inițializat în metoda `initAeWithPsk()`. Această metodă extrage PSK-ul și derivează materialul necesar pentru protejarea payload-ului de inițializare.

Seed-ul folosit pentru derivarea cheilor AE este construit din identitățile participanților într-o ordine deterministă: $\text{seedPSK} = \min(A, B) | \max(A, B)$. Această ordine fixă este necesară deoarece pentru unul dintre participanți $\text{locId} = A$, iar pentru celălalt $\text{locId} = B$. Fără ordonarea deterministă, cei doi ar putea deriva chei diferite pentru protecția mesajelor de inițializare. Fragmentul următor este important, deoarece arată cum PSK-ul este transformat în material folosit pentru criptarea autenticată a mesajelor INIT_A și INIT_B:

```
private void initAeWithPsk() throws Exception {
    SecretKey psk = getEspPsk();
```

```

...
// Symmetric seed so both peers derive the same AE key set
String id1 = (locId.compareTo(remId) <= 0) ? locId : remId;
String id2 = (locId.compareTo(remId) <= 0) ? remId : locId;
...
GenSessionKeyBytes genKeyBytes = new GenSessionKeyBytes(macName);
SecretKey kdk = genKeyBytes.extract(psk.getEncoded(), seed);
int numKeyBytes = isGCM ? keyLen : 2 * keyLen;
byte[] derivedBytes = genKeyBytes.expand(kdk, seed, numKeyBytes);
...
aeWithPsk = new AutEnc(encName, macName);
aeWithPsk.setKeyEnc(new SecretKeySpec(derivedBytes, 0, keyLen, encAlgName));
...}

```

După inițializarea protecției AE cu PSK, Ana generează nonce-ul N_A și perechea efemeră Diffie-Hellman, calculând g^X . Spre deosebire de protocoalele MAKE-IKE, aceste valori nu sunt transmise în clar în câmpurile obișnuite ale mesajului. În schimb, se construiește payload-ul: $alg_A|N_A|g^X$. Payload-ul este protejat prin criptare autenticată: $eDataA = AE_{K_{21}}(alg_A|N_A|g^X)$ și transmis către Bob. În implementare, acest pas este realizat prin metoda `buildInitMsg()`. Payload-ul este creat prin `encodeInitData(locAlg, locNonce, dhPubBytes)`, apoi este protejat cu `aeWithPsk.encrypt(payload)`, iar rezultatul este pus în câmpul `eData`.

```

protected InitMsg buildInitMsg() throws Exception {...}

```

Formatul payload-ului este definit explicit prin metoda `encodeInitData()`. Fiecare câmp este precedat de lungimea sa pe 2 octeți: `[2B algLen][alg][2B nonceLen][nonce][2B dhLen][dhKey]`. Această codificare permite receptorului să extragă fără ambiguitate algoritmul, nonce-ul și cheia publică Diffie-Hellman. În plus, lungimile sunt verificate la decodare, ceea ce ajută la respingerea mesajelor invalide.

```

private byte[] encodeInitData(String alg, byte[] nonce, byte[] dhKey) {
    byte[] algBytes = (alg == null) ? null : alg.getBytes();
    int algLen = (algBytes == null) ? 0 : algBytes.length;
    int nonceLen = (nonce == null) ? 0 : nonce.length;
    int dhLen = (dhKey == null) ? 0 : dhKey.length;
    int encodedLen = 6 + algLen + nonceLen + dhLen;
    byte[] encodedBytes = new byte[encodedLen];
    int pos = 0;
    ...}

```

Bob primește `INIT_A`, decriptează `eDataA`, verifică autenticitatea payload-ului și extrage: $alg_A|N_A|g^X$. În implementare, această operație este realizată prin metoda `verifyAndExtract()`. Dacă decriptarea eșuează, sau algoritmul nu corespunde configurației locale, sau valorile extrase sunt invalide, protocolul transmite `FailMsg`.

```

private boolean verifyAndExtract(InitMsg msg){...}

```

După validarea mesajului, Bob generează nonce-ul N_B și valoarea publică Diffie-Hellman: g^Y . Bob va calcula secretul comun: $S_B = (g^X)^Y = g^{XY}$, apoi construiește payload-ul propriu: $alg_B|N_B|g^Y$. Payload-ul este protejat prin criptare autenticată: $eDataB = AE_{K_{22}}(alg_B|N_B|g^Y)$ și transmis către Ana. Când Ana primește mesajul, decriptează payload-ul și extrage alg_B , N_B , g^Y . Apoi calculează: $S_A = (g^Y)^X = g^{YX}$. Fragmentul următor din implementare arată cum protocolul cu două mesaje este procesat atât de responder, cât și de inițiator, fără folosirea mesajelor AuthMsg.

```
public byte[] recvInitMsg(InitMsg msg) throws Exception {...}
```

După schimbul celor două mesaje, cheile finale sunt derivate din secretul DH și nonce-uri: $K_S = PRFN(g^{XY})$, unde $N = N_A | N_B$, $PRFK_S(N_A | N_B | S) = K_d | K_{11} | K_{12} | K_{21} | K_{22}$.

Metoda `genSessionKeys()` folosește aceeași idee de bază: calculează secretul DH, construiește seed-ul $N_A | N_B$, aplică `extract()` și `expand()`, apoi setează cheile de sesiune. Diferența față de protocoalele MAKE-IKE este că aici nu se mai derivă chei pentru $MAC(A)$ și $MAC(B)$, deoarece nu există mesaje separate de autentificare. Materialul derivat este folosit direct pentru protejarea transferului securizat de date. Fragmentul următor evidențiază diferența dintre protocolul ESP și protocoalele MAKE-IKE la nivel de derivare a cheilor.

```
protected void genSessionKeys() throws Exception {
    ...
    byte[] secretBytesDH = KeyAgrDH.genDHSecret(dhName, locDHKeyPair, remDHPubBytes);
    byte[] seed = initiator
        ? Utils.concatenate(locNonce, remNonce)
        : Utils.concatenate(remNonce, locNonce);
    // The PRF for key derivation
    GenSessionKeyBytes genKeyBytes = new GenSessionKeyBytes(macName);
    SecretKey kdk = genKeyBytes.extract(secretBytesDH, seed);
    byte[] randBytes = genKeyBytes.expand(kdk, seed, randBytesLen);
    setSessionKeys(randBytes); }
```

În acest protocol, metodele `recvInitAuthMsg()` și `recvAuthMsg()` tratează aceste mesaje ca intrări neașteptate și trimit `FailMsg`. Acest detaliu arată clar că protocolul folosește exclusiv cele două mesaje `InitMsg`.

```
public byte[] recvInitAuthMsg(InitAuthMsg msg) throws Exception
{
    sendFailMsg(remId, "[KeyEx: " + locId + "] Unexpected InitAuthMsg in state " +
state);
    return null;}
public byte[] recvAuthMsg(AuthMsg msg) throws Exception
{
    sendFailMsg(remId, "[KeyEx: " + locId + "] Unexpected AuthMsg in state " + state);
    return null;
}
```

Din punct de vedere al proprietăților, protocolul protejează payload-ul de inițializare prin criptare autenticată și derivă chei proaspete prin DHE pentru transferul securizat ulterior. Totuși, el nu înlocuiește protocoalele MAKE-IKE cu patru mesaje, deoarece presupune existența unui context de securitate anterior. Rolul său în lucrare este să modeleze partea de obținere a materialului criptografic pentru protejarea datelor în stil ESP [4]. Codul complet se regăsește în Anexa A3.

4.5 Testarea funcțională a protocoalelor implementate

După implementarea protocoalelor, acestea au fost testate în platforma SJAKE prin scenarii de comunicație între doi participanți onești, Ana și Bob. Scopul testării a fost verificarea comportamentului complet al fiecărui protocol: transmiterea mesajelor în ordinea corectă, verificarea autenticatorilor, generarea cheilor de sesiune și folosirea acestora pentru protecția datelor. Trace-urile generate de platformă permit urmărirea fiecărei execuții și confirmă dacă protocolul ajunge în starea finală.

Pentru protocolul MAKE-IKE-MAC-PSK-DHE 4M, Ana și Bob schimbă inițial valorile necesare pentru negocierea algoritmilor, nonce-urile și valorile Diffie-Hellman. Pe baza acestor date, fiecare participant calculează secretul comun și derivă cheile de sesiune. Partea importantă a testului este verificarea autenticatorilor de tip MAC. Bob trebuie să verifice faptul că mesajul de autentificare provine de la Ana, iar aceasta trebuie să confirme identitatea lui Bob. Dacă aceste verificări reușesc, înseamnă că ambele părți folosesc aceeași cheie pre-partajată și au ajuns la același material criptografic de sesiune. Trecerea în starea ESTABLISHED confirmă finalizarea schimbului de chei, iar acceptarea mesajelor de date arată că sesiunea stabilită poate fi folosită efectiv pentru comunicație securizată.

```
TestAke2P: TEST_2P_MAKE_IKE_MAC_PSK_DH_4M; prot MAKE_IKE_MAC_PSK_DH_4M
>> Start key exchange (2 parties)
...
Session[0]: sid FD3BE815CDA6D213 from ana to bob state START
[KeyEx: ana] build InitMsg to bob, new state INIT_SENT
...
[KeyEx: bob] recv InitMsg in state START
[KeyEx: bob] build InitMsg to ana, new state INIT_RCVD
...
[KeyEx: ana] recv InitMsg in state INIT_SENT
[KeyEx: ana] build AuthMsg to bob, new state AUTH_SENT
[Comm] MsgHeader: { AUTH_A from ana to bob sid FD3BE815CDA6D213 }
AUTH_A:
  edata[116]:
...
[KeyEx: bob] recvAuth in state INIT_RCVD
[KeyEx: bob] Successful verification of ana's authenticator (MAC)
  Successful authentication: remote user is ana
[KeyEx: bob] Received early data: Hello, I'm Ana (the initiator)
[KeyEx: bob] build AuthMsg to ana, new state INIT_RCVD
[Comm] MsgHeader: { AUTH_B from bob to ana sid FD3BE815CDA6D213 }
[KeyEx: bob] Responder: Auth handshake done, state ESTABLISHED. Success
```

```

AUTH_B:
  edata[116]:
...
[KeyEx: ana] recvAuth in state AUTH_SENT
[KeyEx: ana] Successful verification of bob's authenticator (MAC)
  Successful authentication: remote user is bob
[KeyEx: ana] Initiator: Auth handshake done, state ESTABLISHED. Success
[KeyEx: ana] Received early data: Hello, I'm Bob (the responder)

>> Current sessions
[Party: ana] 1 sessions
  Session[0]: sid FD3BE815CDA6D213 from ana to bob state ESTABLISHED
[Party: bob] 1 sessions
  Session[0]: sid FD3BE815CDA6D213 from bob to ana state ESTABLISHED
>> Key exchange finished (success) <<
>> System ready for communications << ...

```

În cazul protocolului MAKE-IKE-SIG-PKC-DHE 4M, structura generală a execuției este asemănătoare cu cea a protocolului anterior: participanții schimbă mesaje de inițializare, calculează secretul Diffie-Hellman și derivă cheile de sesiune. Diferența apare în etapa de autentificare, unde dovada identității nu mai este realizată printr-o cheie pre-partajată, ci prin semnătura digitală a fiecărui participant. În trace trebuie urmărit momentul în care Bob verifică semnătura Anei și momentul în care Ana verifică semnătura lui Bob. Reușita acestor verificări arată că certificatele sunt acceptate, semnăturile sunt valide, iar datele autentificate corespund sesiunii curente. După această etapă, protocolul ajunge în starea ESTABLISHED, iar mesajele de date sunt acceptate.

```

TestAke2P: TEST_2P_MAKE_IKE_SIG_PKC_DH_4M; prot MAKE_IKE_SIG_PKC_DH_4M
>> Start key exchange (2 parties)
...
[KeyEx: ana] build InitMsg to bob, new state INIT_SENT
...
[KeyEx: bob] recv InitMsg in state START
[KeyEx: bob] build InitMsg to ana, new state INIT_RCVD
...
[KeyEx: ana] recv InitMsg in state INIT_SENT
[KeyEx: ana] build AuthMsg to bob, new state AUTH_SENT
[Comm] MsgHeader: { AUTH_A from ana to bob sid C3771BCE560A22B3 }
AUTH_A:
  edata[706]:
...
[KeyEx: bob] recvAuth in state INIT_RCVD
[KeyEx: bob] Successful verification of ana's authenticator (SIG)
  Successful authentication: remote user is ana
[KeyEx: bob] Received early data: Hello, I'm Ana (the initiator)
[KeyEx: bob] build AuthMsg to ana, new state INIT_RCVD
[Comm] MsgHeader: { AUTH_B from bob to ana sid C3771BCE560A22B3 }
[KeyEx: bob] Responder: Auth handshake done, state ESTABLISHED. Success

```

```

AUTH_B:
  edata[708]:
...
[KeyEx: ana] recvAuth in state AUTH_SENT
[KeyEx: ana] Successful verification of bob's authenticator (SIG)
  Successful authentication: remote user is bob
[KeyEx: ana] Initiator: Auth handshake done, state ESTABLISHED. Success
[KeyEx: ana] Received early data: Hello, I'm Bob (the responder)
>> Current sessions
[Party: ana] 1 sessions
  Session[0]: sid C3771BCE560A22B3 from ana to bob state ESTABLISHED
[Party: bob] 1 sessions
  Session[0]: sid C3771BCE560A22B3 from bob to ana state ESTABLISHED
>> Key exchange finished (success) <<
>> System ready for communications << ...

```

Al treilea test urmărește protocolul MAKE-ESP-MAC-PSK-DHE 2M. Spre deosebire de cele două protocoale anterioare, acesta are un flux mai scurt, format din două mesaje de inițializare. Nu există mesaje Auth separate. Informațiile necesare stabilirii sesiunii sunt introduse direct în câmpul protejat eData, folosind criptare autenticată. În acest test, elementul esențial este verificarea câmpului eData. Inițiatorul trimite primul mesaj, iar responderul trebuie să îl decripteze și să îl verifice înainte de a genera răspunsul. Apoi, inițiatorul verifică la rândul său mesajul primit de la responder. Dacă ambele verificări reușesc, participanții pot calcula același secret Diffie-Hellman și pot deriva materialul criptografic necesar protejării datelor. Trecerea în starea ESTABLISHED arată că protocolul cu două mesaje s-a finalizat corect. Acceptarea mesajelor de date confirmă faptul că materialul derivat nu este doar calculat, ci poate fi folosit efectiv pentru transfer securizat, în stilul mecanismelor inspirate din IPsec-ESP.

```

TestAke2P: TEST_2P_MAKE_ESP_MAC_PSK_DH_2M; prot MAKE_ESP_MAC_PSK_DH_2M
>> Start key exchange (2 parties)
[KeyEx: ana] build InitMsg to bob, new state INIT_SENT
[Comm] MsgHeader: { INIT_A from ana to bob sid C16E847041C7D146 }
INIT_A:
  eData[250]:
    data[202]:
      ...
      IV[16]:
      ...
      mac[32]:
      ...
[KeyEx: bob] recv InitMsg in state START
[KeyEx: bob] build InitMsg to ana, new state INIT_SENT
[Comm] MsgHeader: { INIT_B from bob to ana sid C16E847041C7D146 }
[KeyEx: bob] Responder: handshake done, state ESTABLISHED. Success
INIT_B:
  eData[250]:
    data[202]:
      ...

```

```

IV[16]:
...
mac[32]:
...
[KeyEx: ana] recv InitMsg in state INIT_SENT
[KeyEx: ana] Initiator: handshake done, state ESTABLISHED. Success
>> Current sessions
[Party: ana] 1 sessions
  Session[0]: sid C16E847041C7D146 from ana to bob state ESTABLISHED
[Party: bob] 1 sessions
  Session[0]: sid C16E847041C7D146 from bob to ana state ESTABLISHED
>> Key exchange finished (success) <<
>> System ready for communications << ...

```

4.6 Analiza comparativă a variantelor MAC-PSK și SIG-PKC

Pentru analiza performanțelor au fost comparate cele două variante principale de stabilire a cheilor implementate în lucrare: MAKE-IKE-MAC-PSK-DHE 4M și MAKE-IKE-SIG-PKC-DHE 4M. Scopul testului este observarea timpului de execuție pentru protocoale în funcție de mecanismul de autentificare și de algoritmul Diffie-Hellman folosit. Testele au fost realizate pentru patru configurații. Pentru protocolul bazat pe semnături digitale și certificate au fost folosite variantele ECDH P-256 + ECDSA și DH 3072 + RSA 3072. Pentru protocolul bazat pe cheie pre-partajată au fost folosite variantele ECDH P-256 și DH 3072. În toate cazurile, protecția datelor a fost realizată cu AES/CTR-128. Pentru fiecare configurație au fost efectuate cinci rulări, iar timpul mediu a fost calculat în microsecunde. (μs).

Protocol	Criptare	Test1(μs)	Test2(μs)	Test3(μs)	Test4(μs)	Test5(μs)	Media(μs)
MAKE-IKE-SIG-PKC-DHE 4M.	ECDH_256_CTR_128	8900.8	9298.4	9212.2	9052.5	9167.3	9126.24
MAKE-IKE-SIG-PKC-DHE 4M.	DH_RSA_3072_CTR_128	14968.0	15896.0	15679.5	15261.4	14998.3	15360.64
MAKE-IKE-MAC-PSK-DHE 4M.	ECDH_256_CTR_128	3873.1	3710.5	3621.2	3672.8	3772.5	3730.02
MAKE-IKE-MAC-PSK-DHE 4M.	DH_3072_CTR_128	5311.0	5324.4	5240.3	5426.1	5410.0	5342.36

Tabelul 4.1 Rezultatele testelor de performanță pentru protocoalele MAKE-IKE

Se observă că MAKE-IKE-MAC-PSK-DHE 4M este mai rapidă decât varianta MAKE-IKE-SIG-PKC-DHE 4M pentru ambele configurații analizate. În cazul configurației bazate pe ECDH P-256, protocolul MAC-PSK are un timp mediu de aproximativ 3.73 ms, în timp ce protocolul SIG-PKC are un

timp mediu de aproximativ 9.13 ms. Această diferență este explicabilă prin natura celor două metode de autentificare. În varianta MAC-PSK, autentificarea se bazează pe operații simetrice, care sunt în general mai rapide. Participantul calculează și verifică un cod MAC folosind o cheie secretă pre-partajată. În schimb, varianta SIG-PKC presupune operații mai costisitoare: generarea semnăturii digitale, verificarea semnăturii și procesarea certificatului participantului. Din acest motiv, protocolul bazat pe semnături și certificate are un timp de execuție mai mare.

O diferență asemănătoare se observă și în configurațiile bazate pe grupuri Diffie-Hellman clasice. Protocolul MAKE-IKE-MAC-PSK-DHE 4M cu DH 3072 are un timp mediu de aproximativ 5.34 ms, în timp ce protocolul MAKE-IKE-SIG-PKC-DHE 4M cu DH 3072 + RSA 3072 are un timp mediu de aproximativ 15.36 ms. În acest caz, costul suplimentar este influențat atât de schimbul Diffie-Hellman pe grup finit, cât și de operațiile RSA folosite în autentificarea cu semnături digitale. Rezultatele evidențiază și diferența dintre configurațiile bazate pe curbe eliptice și cele bazate pe grupuri finite mari. Pentru protocolul SIG-PKC, trecerea de la ECDH P-256 la DH 3072 + RSA 3072 crește timpul mediu de la 9126.24 μ s la 15360.64 μ s. Pentru protocolul MAC-PSK, trecerea de la ECDH P-256 la DH 3072 crește timpul mediu de la 3730.02 μ s la 5342.36 μ s. În ambele cazuri, configurațiile bazate pe curbe eliptice sunt mai eficiente, deoarece oferă un nivel de securitate comparabil folosind chei mai mici și operații criptografice mai rapide.

Rezultatele trebuie interpretate în contextul implementării din SJAKE. Testul nu măsoară o singură primitivă criptografică izolată, cum ar fi un MAC, o semnătură digitală sau un schimb Diffie-Hellman. El măsoară execuția completă a protocolului: generarea valorilor efemere, construirea și procesarea mesajelor, derivarea cheilor, autentificarea participanților, protecția mesajelor prin criptare autenticată și logica internă a platformei. Din acest motiv, rezultatele sunt relevante pentru comportamentul practic al protocoalelor implementate, nu doar pentru comparația teoretică dintre primitive.

Capitolul 5. Atacuri asupra unor variante vulnerabile ale protocoalelor

5.1 Considerații generale privind atacurile ILV implementate

În acest capitol sunt analizate atacurile de tip Interleaving, implementate asupra unor variante vulnerabile ale protocoalelor MAKE-IKE-MAC-PSK-DHE 4M și MAKE-IKE-SIG-PKC-DHE 4M. Spre deosebire de Capitolul 4, unde au fost prezentate protocoalele corecte, aici sunt folosite variante modificate intenționat, pentru a evidenția efectul unui transcript autentificat incomplet.

Atacul nu urmărește spargerea primitivelor criptografice. Adversarul nu recuperează cheia pre-partajată, nu falsifică semnături digitale și nu rezolvă problema Diffie-Hellman. În schimb, el profită de faptul că autentificatorul din protocolul vulnerabil este calculat peste prea puține date. Din această cauză, un autentificator valid, produs de un participant onest, poate fi folosit într-un alt context de sesiune.

În variantele vulnerabile analizate, autentificatorul este calculat doar peste: `remoteID | remoteNonce | remoteDHPublicKey`. Această construcție este insuficientă. Ea leagă autentificatorul de datele partenerului, dar nu îl leagă complet de întreaga execuție a protocolului. În special, nu autentifică valoarea Diffie-Hellman transmisă de participantul care se autentifică. De aici apare vulnerabilitatea principală: adversarul poate înlocui cheia publică Diffie-Hellman a Anei cu propria cheie publică, iar Bob poate ajunge să creadă că stabilește cheia cu Ana, deși cheia este de fapt comună cu adversarul.

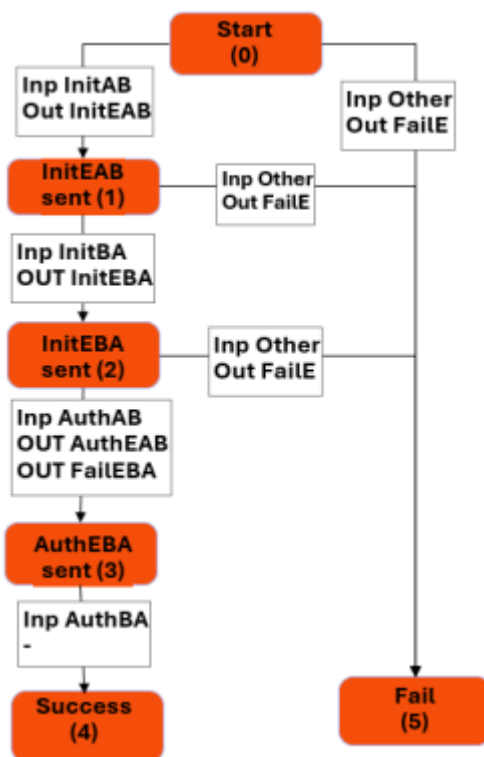


Figura 5.1 FSM-ul adversarului pentru atacul ILV asupra protocoalelor MAKE-4M vulnerabile

Figura 5.1 prezintă logica generală a adversarului folosit în atacurile ILV. Automatul pornește din starea Start, unde adversarul așteaptă primul mesaj al protocolului, InitAB, transmis de Ana către Bob. În loc să transmită mesajul mai departe fără modificări, adversarul construiește un nou mesaj, notat InitEAB.

Acest mesaj păstrează identitatea Anei și nonce-ul Anei, dar înlocuiește cheia publică Diffie-Hellman a Anei cu cheia publică a adversarului. Acesta este pasul care face atacul posibil. Bob primește mesajul ca și cum ar veni de la Ana, dar valoarea Diffie-Hellman folosită în calculul secretului aparține adversarului.

După transmiterea mesajului InitEAB, automatul trece în starea InitEAB sent. În această stare, adversarul așteaptă răspunsul lui Bob, adică mesajul InitBA. Acest mesaj conține nonce-ul lui Bob și cheia publică Diffie-Hellman a acestuia. Adversarul îl retransmite către Ana sub forma InitEBA. Rolul acestui pas este să o determine pe Ana să genereze un autentificator valid pentru datele lui Bob.

După această retransmitere, automatul ajunge în starea InitEBA sent. În continuare, Ana transmite mesajul AuthAB. Acesta conține autentificatorul Anei. În protocolul vulnerabil, autentificatorul este valid pentru datele lui Bob, dar nu confirmă cheia publică Diffie-Hellman folosită de Bob în prima parte a sesiunii. Adversarul poate retransmite acest autentificator către Bob sub forma AuthEAB.

În aceeași etapă, adversarul trimite și un mesaj de eșec către Ana, notat în figură prin FailEBA. Prin acest mesaj, sesiunea Anei este oprită. Practic, adversarul nu mai are nevoie ca Ana să continue protocolul, deoarece autentificatorul ei a fost deja obținut și retransmis către Bob.

Ultimul mesaj relevant este AuthBA, transmis de Bob. Dacă acest mesaj este primit în starea așteptată, automatul ajunge în starea Success. Acest lucru înseamnă că atacul a reușit: Bob a acceptat autentificarea și crede că partenerul său este Ana, dar cheia de sesiune a fost stabilită cu adversarul.

Figura evidențiază și ramurile de eroare. Dacă adversarul primește un alt tip de mesaj decât cel așteptat într-o anumită stare, execuția trece în starea Fail. Această parte este importantă deoarece arată că atacul nu este o simplă retransmitere aleatorie de mesaje, ci o secvență controlată, dependentă de ordinea exactă a mesajelor.

Logica din FSM se corelează direct cu implementarea adversarilor din SJAKE. În clasele AdvMAkeMacIKEILV și AdvMAkeSigIKEILV, fiecare etapă este tratată în funcție de starea curentă a atacului. În starea START, adversarul interceptează primul InitMsg și înlocuiește cheia Diffie-Hellman. În starea INIT_SENT, acesta primește răspunsul lui Bob și îl retransmite către Ana. În starea INIT_RCVD, adversarul interceptează autentificatorul Anei și îl trimite către Bob. În final, după mesajul de autentificare al lui Bob, atacul este marcat ca reușit.

Cele două atacuri implementate sunt:

1. AdvMAkeMacIKEILV – atac ILV asupra variantei vulnerabile MAkeMacIKEILV
2. AdvMAkeSigIKEILV – atac ILV asupra variantei vulnerabile MAkeSigIKEILV

În ambele cazuri, structura atacului este aceeași. Diferența apare doar în tipul autentificatorului retransmis către Bob. În varianta MAC-PSK, adversarul retransmite un MAC valid produs de Ana. În varianta SIG-PKC, adversarul retransmite semnătura digitală și certificatul Anei. În ambele situații, autentificatorul este real, dar este folosit într-un context greșit.

5.2 Atacul ILV asupra variantei vulnerabile MAKE-IKE-MAC-PSK-DHE 4M

Atacul asupra variantei MAKE-IKE-MAC-PSK-DHE 4M pornește de la o modificare aparent mică a protocolului corect. În varianta corectă, autentificatorul Anei are forma: $\text{MAC}_{\text{KAB}}(\text{algA} \mid \text{NA} \mid g^X \mid \text{NB} \mid \text{MAC}_{\text{KII}}(\text{A}))$. Această formulă leagă autentificatorul de algoritmul ales, de nonce-uri, de cheia publică Diffie-Hellman a Anei și de identitatea Anei legată de cheia derivată din secretul DH. În varianta vulnerabilă, autentificatorul este redus la: $\text{MAC}_{\text{KAB}}(\text{remId} \mid \text{remNonce} \mid \text{remDHPubKey})$. Pentru Ana, care crede că vorbește cu Bob, această valoare devine: $\text{MAC}_{\text{KAB}}(\text{B} \mid \text{NB} \mid g^Y)$. Problema este că autentificatorul nu include g^X , adică valoarea Diffie-Hellman publică a Anei. Din acest motiv, adversarul poate înlocui în primul mesaj cheia publică a Anei cu propria cheie publică, fără ca autentificatorul generat ulterior de Ana să devină invalid.

În implementarea variantei vulnerabile, acest lucru este vizibil direct în metoda `generateAuth()`, unde datele autentificate sunt doar `remId`, `remNonce` și `remDHPubBytes`. MAC-ul nu acoperă transcriptul complet, ci doar identitatea, nonce-ul și cheia publică DH ale partenerului.

```
protected byte[] generateAuth() throws Exception
{
    AutSec authSec = new AutSec(commonCrypto.getAutSecName());
    authSec.setKey(getAuthPsk());
    byte[] data = Utils.concatenate(
        remId.getBytes(),
        remNonce,
        remDHPubBytes
    );
    return authSec.genMac(data);
}
```

Verificarea autentificatorului se face simetric. Bob reconstruiește valoarea: `locId` | `locNonce` | `locDHPubKey`, adică, din perspectiva lui Bob: $\text{B} \mid \text{NB} \mid g^Y$. Aceasta este exact valoarea pe care Ana a autentificat-o. Bob verifică deci un MAC valid, dar nu observă că în primul mesaj cheia publică atribuită Anei a fost înlocuită cu cheia publică a adversarului.

```
protected boolean verifyAuth(byte[] auth) throws Exception
{
    AutSec authSec = new AutSec(commonCrypto.getAutSecName());
    authSec.setKey(getAuthPsk());
    byte[] data = Utils.concatenate(
        locId.getBytes(),
        locNonce,
        locDHKeyPair.getPublic().getEncoded()
    );
    return authSec.verMac(data, auth);
}
```

}

În cazul atacului propriu-zis, adversarul așteaptă primul mesaj de la Ana, apoi construiește un mesaj falsificat către Bob. Mesajul păstrează identitatea Anei și nonce-ul Anei, dar înlocuiește cheia publică g^x cu cheia publică a adversarului, notată aici g^z . Codul adversarului descrie explicit această strategie: interceptează InitMsg, substituie cheia publică DH, transmite mesaje între Ana și Bob și reutilizează autentificatorul Anei pentru a se autentifica la Bob.

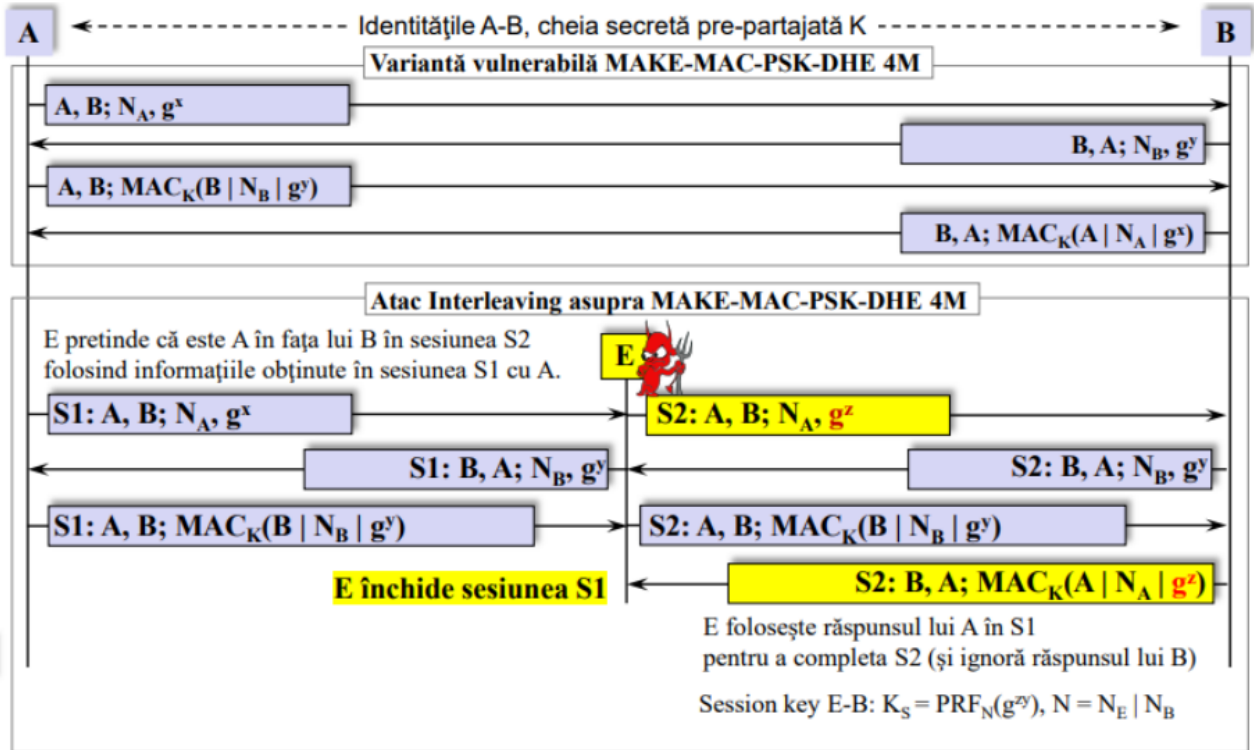


Figura 5.2 Atac ILV asupra MAKE-MAC-PSK-DHE 4M

În acest moment, Bob calculează secretul Diffie-Hellman folosind cheia publică primită în primul mesaj. El crede că acea cheie aparține Anei, dar ea aparține adversarului: $S_B = (g^z)^x = g^{zx}$. Adversarul poate calcula aceeași valoare folosind cheia publică a lui Bob și exponentul său privat: $S_{Adv} = (g^y)^z = g^{yz}$. Bob ajunge să aibă o cheie comună cu adversarul. Autentificatorul Anei este totuși acceptat, deoarece el a fost calculat doar pe datele lui Bob, nu și pe cheia DH reală a Anei. În implementare, înlocuirea cheii publice are loc în ramura case START din `recvInitMsg()`: adversarul salvează `anaId`, `bobId`, `nonceA`, generează propria pereche DH și construiește `initToBob` folosind `advDHKeyPair.getPublic().getEncoded()`.

```
public byte[] recvInitMsg(InitMsg msg) throws Exception
{ ...
    switch (state) {
        case START:
            // intercept InitAB (A -> B)
```

```

anaId = msg.getSrc();
bobId = msg.getDst();
// Store Bob as the real peer of the adversary session
remId = bobId;
// Adversary plays initiator role towards Bob
initiator = true;
// Reuse Ana's nonce and session identifier
nonceA = msg.getNonce();
sidAtk = msg.getSid();
// Generate adversary DH key pair
advDHKeyPair = KeyAgrDH.genDHKeyPair(
    commonCrypto.getDhName(),
    commonCrypto.getDhKeyLen()
);
// Send forged InitMsg to B with Ana's identity and nonce,
// but adversary's own DH public key
InitMsg initToBob = new InitMsg(
    MessageType.INIT_A,
    anaId,
    bobId,
    sidAtk,
    nonceA,
    advDHKeyPair.getPublic().getEncoded(),
    null,
    null
);
...}

```

După ce Bob trimite INIT_B, adversarul salvează nonceB și bobDHPubBytes, calculează cheile cu Bob și retransmite mesajul către Ana. Cheile adversarului sunt calculate din perechea sa DH și cheia publică a lui Bob. Metoda genSessionKeys() a adversarului folosește explicit advDHKeyPair și bobDHPubBytes, apoi seed-ul nonceA | nonceB, pentru a obține același material de sesiune ca Bob.

```

protected void genSessionKeys() throws Exception
{...
    // DH shared secret: (advprivate, B public)
    byte[] secretBytesDH = KeyAgrDH.genDHSecret(
        commonCrypto.getDhName(),
        advDHKeyPair,
        bobDHPubBytes
    );
    //Na | Nb
    byte[] seed = Utils.concatenate(nonceA, nonceB);
    GenSessionKeyBytes genKeyBytes = new
GenSessionKeyBytes(commonCrypto.getAutSecName());
    SecretKey kdk = genKeyBytes.extract(secretBytesDH, seed);
    byte[] randBytes = genKeyBytes.expand(kdk, seed, secBytesLen);
    setSessionKeys(randBytes);
}

```

```

        Utils.trace("[AdvKeyEx: " + locId + "] Generated session keys with Bob.");
    }

```

Apoi adversarul primește AUTH_A de la Ana și îl retransmite către Bob fără să îl modifice. Pentru că AUTH_A autentifică doar $B \mid NB \mid g^Y$, Bob îl consideră valid. La final, adversarul marchează atacul ca reușit și trimite către Ana un mesaj FAIL_B, pentru a opri sesiunea ei.

```

public byte[] recvAuthMsg(AuthMsg msg) throws Exception
{...
    switch (state) {
    case INIT_RCVD:
        ...
        AuthMsg authToBob = new AuthMsg(
            MsgType.AUTH_A,
            anaId,
            bobId,
            sidAtk,
            msg.getAuth(),
            null
        );
        state = AkeState.AUTH_SENT;
        ...
    case AUTH_SENT:
        ...
        success = true;
        state = AkeState.ESTABLISHED;
        Utils.trace("[AdvKeyEx: " + locId + "] Attack successful, state "
            + state + ". Bob believes peer is " + anaId);
        //aborts session S1 explicitly using FAIL_B.
        sendFailMsgToAna("Session aborted");
        notifyTerminate();
        break;
    ...}...}

```

Protocolul corect din Capitolul 4 nu este vulnerabil la acest atac deoarece autentificatorul Anei include g^X și $MAC_{K11}(A)$ ($MAC_{KAB}(\text{algA} \mid NA \mid g^X \mid NB \mid MAC_{K11}(A))$). Dacă adversarul înlocuiește g^X cu g^Z , Bob va verifica un transcript diferit de cel autentificat de Ana. În plus, $MAC_{K11}(A)$ este calculat cu o cheie derivată din secretul DH corect, ceea ce leagă identitatea Anei de cheia de sesiune, iar autentificatorul nu mai poate fi mutat într-o sesiune în care Bob a calculat cheia cu adversarul. Codul complet se regăsește în Anexa A4.

5.3 Atacul ILV asupra variantei vulnerabile MAKE-IKE-SIG-PKC-DHE 4M

Atacul asupra variantei MAKE-IKE-SIG-PKC-DHE 4M este foarte asemănător cu atacul asupra variantei MAC-PSK. Diferența este că autentificatorul nu mai este un MAC calculat cu PSK, ci o semnătură digitală însoțită de certificatul participantului. În protocolul corect, autentificatorul Anei are forma: $SIGA(alg_A \mid NA \mid g^X \mid NB \mid MAC_{K_H}(A))$. În varianta vulnerabilă, semnătura este calculată doar peste: $remoteId \mid remoteNonce \mid remoteDHPublicKey$. Pentru Ana, această valoare devine: $SIGA(B \mid NB \mid g^Y)$. Aceasta este o semnătură validă, iar certificatul Anei este valid. Totuși, semnătura nu dovedește că Ana a autentificat cheia publică DH pe care Bob a primit-o în primul mesaj. Dacă adversarul înlocuiește g^X cu g^Z , semnătura Anei rămâne validă, deoarece ea nu include g^X .

În implementarea variantei vulnerabile semnătura nu acoperă transcriptul complet, ci doar identitatea, nonce-ul și cheia DH publică ale partenerului; metoda `generateAuth()` concatenează exact `remId`, `remNonce` și `remDHPubBytes`, apoi generează semnătura digitală.

```
protected byte[] generateAuth() throws Exception
{
    AutPub autPub = new AutPub(commonCrypto.getAutPubName());
    autPub.setPrivKey(
        ownCrypto.getKeyManager().getPrivateKey(
            locId + commonCrypto.getSigPrivKeyAlias()
        )
    );
    byte[] data = Utils.concatenate(
        remId.getBytes(),
        remNonce,
        remDHPubBytes
    );
    return autPub.genSig(data);
}
```

La verificare, Bob reconstruiește datele semnate ca: $localId \mid localNonce \mid localDHPublicKey$, adică $B \mid NB \mid g^Y$. Această valoare coincide cu ce a semnat Ana. Prin urmare, semnătura se verifică, iar certificatul Anei se verifică, dar autentificarea rămâne incomplet legată de cheia Diffie-Hellman folosită de Bob.

```
protected boolean verifyAuth(byte[] auth) throws Exception
{
    if (remCert == null) {
        return false;
    }
    verifyCert(remCert);
    byte[] data = Utils.concatenate(
        locId.getBytes(),
        locNonce,
        locDHKeyPair.getPublic().getEncoded()
    );
}
```

```

    );
    AutPub autPub = new AutPub(commonCrypto.getAutPubName());
    autPub.setPubKey(remCert.getPublicKey());
    return autPub.verSig(data, auth);
}

```

În prima etapă a atacului propriu-zis, adversarul interceptează INIT_A de la Ana, salvează identitatea și nonce-ul Anei, generează propria pereche DH și transmite către Bob un INIT_A falsificat. Mesajul păstrează anaId, bobId, sidAtk și nonceA, dar folosește cheia publică a adversarului. Codul adversarului este descris chiar în comentarii ca un atac care interceptează InitMsg, substituie cheia publică DH, redirecționează mesajele și reutilizează semnătura și certificatul Anei pentru autentificare la Bob.

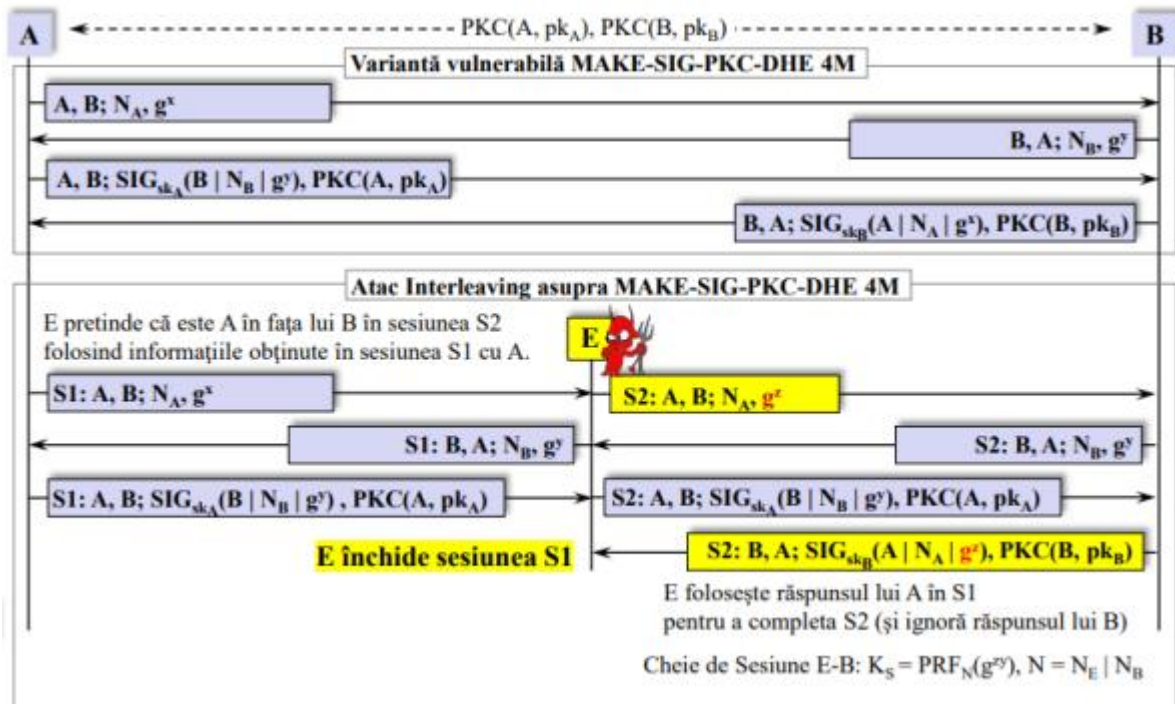


Figura 5.3 Atac ILV asupra MAKE-SIG-PKC-DHE 4M

Bob calculează cheia cu adversarul: $S_B = (g^Z)^Y = g^{ZY}$. Adversarul calculează aceeași cheie: $S_{Adv} = (g^Y)^Z = g^{YZ}$. Bob verifică semnătura Anei și certificatul PKCA. Verificarea reușește, deoarece semnătura este reală și certificatul este valid. Problema este că Bob nu verifică faptul că valoarea DH folosită pentru calculul secretului aparține într-adevăr Anei. În implementare, primul pas al atacului este ramura case START din `recvInitMsg()`, unde adversarul construiește mesajul `initToBob` cu identitatea și nonce-ul Anei, dar cu propria cheie publică DH.

```

public byte[] recvInitMsg(InitMsg msg) throws Exception
{
    ...
    switch (state) {

```

```

        case START:
        ...
        advDHKeyPair = KeyAgrDH.genDHKeyPair(
            commonCrypto.getDhName(),
            commonCrypto.getDhKeyLen()
        );...
        InitMsg initToBob = new InitMsg(
            MessageType.INIT_A,
            anaId,
            bobId,
            sidAtk,
            nonceA,
            advDHKeyPair.getPublic().getEncoded(),
            null,
            null
        );
        ...}
    ...}

```

În pasul următor, adversarul retransmite INIT_B de la Bob către Ana, pentru ca Ana să producă o semnătură validă pe datele lui Bob. Apoi, când Ana trimite AUTH_A, adversarul îl retransmite către Bob fără modificări, păstrând atât semnătura, cât și certificatul Anei.

```

public byte[] recvAuthMsg(AuthMsg msg) throws Exception
{
    ...
    switch (state) {
    case INIT_RCVD:
    ...
    AuthMsg authToBob = new AuthMsg(
        MessageType.AUTH_A,
        anaId,
        bobId,
        sidAtk,
        msg.getAuth(),
        msg.getCert(),
        null
    );
    ...}
    ...}

```

După ce Bob trimite AUTH_B, adversarul își derivează cheile de sesiune cu Bob. Metoda genSessionKeys() folosește perechea DH a adversarului și cheia publică DH a lui Bob, deci obține același secret ca Bob. Apoi adversarul marchează atacul ca reușit și trimite FAIL_B către Ana, fără să își schimbe starea de succes în metoda recvAuthMsg().

```

protected void genSessionKeys() throws Exception {...}

```

```

public byte[] recvAuthMsg(AuthMsg msg) throws Exception
...
    case AUTH_SENT:
        ...
        success = true;
        state = AkeState.ESTABLISHED;
        Utils.trace("[AdvKeyEx: " + locId + "] Attack successful, state "
                    + state + ". Bob believes peer is " + anaId);
        sendFailMsgToAna("Session aborted");
        notifyTerminate();}
...}

```

Protocolul corect din Capitolul 4 nu este vulnerabil la acest atac, deoarece Ana nu semnează doar datele lui Bob. În varianta corectă, ea semnează: $SIG_A(\text{alg}_A \mid N_A \mid g^X \mid N_B \mid \text{MAC}_{K_{11}}(A))$. Această valoare include cheia publică DH a Anei și $\text{MAC}_{K_{11}}(A)$, calculat cu o cheie derivată din secretul DH. Dacă adversarul înlocuiește g^X cu g^Z , semnătura Anei nu mai corespunde transcriptului verificat de Bob. În plus, $\text{MAC}_{K_{11}}(A)$ leagă identitatea Anei de cheia de sesiune corectă, deci semnătura nu mai poate fi reutilizată într-o sesiune în care Bob a stabilit cheia cu adversarul. Codul complet se regăsește în Anexa A5.

5.4 Testarea atacurilor în SJAKE

Testarea atacurilor se face prin scenarii SJAKE în care participanții onești rulează variantele vulnerabile, iar comunicarea este controlată de adversar. Spre deosebire de testele funcționale din Capitolul 4, aici nu se urmărește doar dacă protocolul ajunge în ESTABLISHED, ci mai ales cine împarte cheia cu cine și ce identitate crede participantul că a autentificat.

Primul test urmărește atacul de tip Interleaving asupra variantei vulnerabile MAC-PSK. În trace trebuie urmărită mai întâi intervenția adversarului asupra mesajului de inițializare. Ana pornește protocolul și trimite mesajul InitAB, însă adversarul îl interceptează și construiește un mesaj modificat către Bob. Acest mesaj păstrează datele necesare pentru ca Bob să creadă că sesiunea este cu Ana, dar introduce valoarea Diffie-Hellman a adversarului, iar Bob calculează ulterior secretul de sesiune cu adversarul, nu cu Ana. Următoarea parte importantă din trace este retransmiterea mesajelor. Adversarul transmite către Ana răspunsul lui Bob și apoi folosește autentificatorul produs de Ana pentru a continua sesiunea cu Bob. Atacul reușește, deoarece autentificatorul vulnerabil nu leagă suficient de bine identitatea, cheia Diffie-Hellman și transcriptul complet al sesiunii. În final, Bob ajunge să accepte sesiunea și va crede că partenerul onest este Ana, când de fapt partenerul său este adversarul.

```

...
Session[0]: sid 55A758CE085D66EF from ana to bob state START
[KeyEx: ana] build InitMsg to bob, new state INIT_SENT
...
Session[0]: sid 55A758CE085D66EF from adv to ana state START
[AdvKeyEx: adv] recv InitMsg in state START from ana to bob

```

```

[AdvKeyEx: adv] send forged InitAB to bob, new state INIT_SENT
...
Session[0]: sid 55A758CE085D66EF from bob to ana state START
[KeyEx: bob] recv InitMsg in state START
[KeyEx: bob] build InitMsg to ana, new state INIT_RCVD
...
[AdvKeyEx: adv] recv InitMsg in state INIT_SENT from bob to ana
[AdvKeyEx: adv] Generated session keys with Bob.
[AdvKeyEx: adv] relay InitBA to ana, new state INIT_RCVD
...
[KeyEx: ana] recv InitMsg in state INIT_SENT
[KeyEx: ana] build AuthMsg to bob, new state AUTH_SENT
...
[AdvKeyEx: adv] recv AuthMsg in state INIT_RCVD from ana to bob
[AdvKeyEx: adv] relay AuthAB to bob, new state AUTH_SENT
...
[KeyEx: bob] Successful verification of ana's authenticator (MAC-PSK-DHE-VULN)
    Successful authentication: remote user is ana
[KeyEx: bob] build AuthMsg to ana, new state INIT_RCVD
...
[AdvKeyEx: adv] recv AuthMsg in state AUTH_SENT from bob to ana
[AdvKeyEx: adv] Attack successful, state ESTABLISHED. Bob believes peer is ana
[AdvKeyEx: adv] sendFail to ana: Session aborted
[Comm] MsgHeader: { FAIL_B from bob to ana sid 55A758CE085D66EF }
FAIL_B: { Session aborted }
[KeyEx: ana] recvFail, new state FAILED (Fail). From bob: Session aborted

>> Current sessions
[Party: ana] 1 sessions
    Session[0]: sid 55A758CE085D66EF from ana to bob state FAILED
[Party: bob] 1 sessions
    Session[0]: sid 55A758CE085D66EF from bob to ana state ESTABLISHED
[Party: adv] 1 sessions
    Session[0]: sid 55A758CE085D66EF from adv to bob state ESTABLISHED
...

```

Pentru atacul SIG-PKC, structura atacului este asemănătoare cu cea din cazul MAC-PSK, însă autentificarea se bazează pe semnături digitale și certificate. Chiar dacă semnătura Anei este validă, protocolul rămâne vulnerabil deoarece datele semnate nu includ toate elementele necesare pentru legarea corectă a sesiunii. În trace trebuie observat faptul că adversarul nu falsifică semnătura Anei și nu compromite certificatul acesteia. El folosește semnătura validă produsă de Ana într-un alt context de sesiune. Mai exact, adversarul interceptează mesajul inițial, înlocuiește valoarea Diffie-Hellman cu propria valoare și apoi redirecționează mesajele astfel încât Bob să accepte execuția ca fiind realizată cu Ana. Momentul decisiv al testului este acela în care Bob verifică semnătura și acceptă autentificarea. Din punct de vedere criptografic, semnătura este corectă. Problema este că ea nu autentifică transcriptul

complet al sesiunii. De aceea, Bob ajunge în starea de succes cu o asociere greșită între identitatea Anei și cheia de sesiune.

```
...
[KeyEx: ana] recv InitMsg in state INIT_SENT
[KeyEx: ana] build AuthMsg to bob, new state AUTH_SENT
[Comm] MsgHeader: { AUTH_A from ana to bob sid 1F203162D7ABE4FF }
AUTH_A:
  auth[72]:
    ...
  cert[552]:
    ...
[AdvKeyEx: adv] recv AuthMsg in state INIT_RCVD from ana to bob
[AdvKeyEx: adv] relay AuthAB to bob, new state AUTH_SENT
[Comm] MsgHeader: { AUTH_A from ana to bob sid 1F203162D7ABE4FF }
AUTH_A:
  auth[72]:
    ...
  cert[552]:
    ...

[KeyEx: bob] recvAuth in state INIT_RCVD
[KeyEx: bob] Successful verification of ana's authenticator (SIG-PKC-DHE-ILV)
  Successful authentication: remote user is ana
...
[AdvKeyEx: adv] recv AuthMsg in state AUTH_SENT from bob to ana
[AdvKeyEx: adv] Generated session keys with Bob.
[AdvKeyEx: adv] Attack successful, state ESTABLISHED. Bob believes peer is ana
[AdvKeyEx: adv] sendFail to ana: Session aborted
[Comm] MsgHeader: { FAIL_B from bob to ana sid 1F203162D7ABE4FF }
FAIL_B: { Session aborted }
[KeyEx: ana] recvFail, new state FAILED (Fail). From bob: Session aborted

>> Current sessions
[Party: ana] 1 sessions
  Session[0]: sid 1F203162D7ABE4FF from ana to bob state FAILED
[Party: bob] 1 sessions
  Session[0]: sid 1F203162D7ABE4FF from bob to ana state ESTABLISHED
[Party: adv] 1 sessions
  Session[0]: sid 1F203162D7ABE4FF from adv to bob state ESTABLISHED
```

Atacurile implementate arată că securitatea unui protocol AKE nu rezultă doar din folosirea unor primitive sigure. În ambele variante vulnerabile, primitivele sunt în continuare corecte: MAC-ul este calculat cu o cheie reală, semnătura este generată cu cheia privată corectă, certificatul este valid, iar

Diffie-Hellman produce un secret comun. Problema este că aceste primitive sunt aplicate pe date insuficiente.

În variantele vulnerabile, autentificatorul acoperă doar: $\text{remId} \mid \text{remNonce} \mid \text{remDHPubKey}$. În protocoalele corecte, autentificatorul include transcriptul relevant al sesiunii: $\text{MAC}_{K_{AB}}(\text{alg}_A \mid N_A \mid g^X \mid N_B \mid \text{MAC}_{K_{II}}(A))$, respectiv $\text{SIG}_A(\text{alg}_A \mid N_A \mid g^X \mid N_B \mid \text{MAC}_{K_{II}}(A))$. Această diferență este decisivă. În forma corectă, autentificatorul nu poate fi mutat într-o sesiune în care adversarul a înlocuit cheia DH a Anei, deoarece valoarea g^X și $\text{MAC}_{K_{II}}(A)$ nu se mai potrivesc cu transcriptul verificat de Bob. Atacurile ILV confirmă importanța autentificării complete a transcriptului. Identitățile, nonce-urile, valorile Diffie-Hellman, direcția mesajului și cheile derivate trebuie legate între ele. Dacă această legătură lipsește, adversarul poate produce o asociere greșită între identitate și cheia de sesiune, chiar fără să spargă niciuna dintre primitivele criptografice folosite [1], [6].

Concluzii

Lucrarea a avut ca scop analiza și implementarea unor protocoale criptografice inspirate din IKEv2 și IPsec. Nu s-a urmărit reproducerea completă a standardelor, cu toate detaliile lor operaționale, ci extragerea și implementarea mecanismelor importante de securitate: schimbul Diffie-Hellman efemer, autentificarea participanților, derivarea cheilor de sesiune și protejarea datelor prin criptare autentificată.

În cadrul lucrării au fost implementate trei protocoale principale: MAKE-IKE-SIG-PKC-DHE 4M, MAKE-IKE-MAC-PSK-DHE 4M și MAKE-ESP-MAC-PSK-DHE 2M. Primele două modelează etapa de stabilire a cheilor în stil IKEv2, iar al treilea modelează obținerea materialului criptografic necesar pentru protejarea comunicației în stil ESP. Implementarea acestora în SJAKE a permis urmărirea concretă a pașilor unui protocol AKE: generarea valorilor proaspete, schimbul de mesaje, autentificarea, derivarea cheilor și folosirea acestora pentru transmiterea datelor.

Contribuția personală a lucrării constă în identificarea nucleului criptografic al protocoalelor IKEv2 și IPsec-ESP și în transpunerea acestuia în protocoale simplificate, dar relevante din punct de vedere al securității. Am implementat variante funcționale pentru autentificare prin semnături digitale și certificate, autentificare prin MAC cu cheie pre-partajată și protejarea datelor prin mecanisme de tip ESP. În plus, am analizat variante vulnerabile ale protocoalelor și am implementat atacuri de tip Interleaving, pentru a arăta practic ce se poate întâmpla atunci când autentificarea nu acoperă toate datele importante ale sesiunii.

Un aspect mai interesant al temei este negocierea autentificată a algoritmilor criptografici. În implementarea realizată, modelul este simplificat: fiecare participant transmite o singură configurație criptografică, nu o listă de propuneri ca în IKEv2. Din punct de vedere al securității, mecanismul rămâne relevant, deoarece algoritmul ales este inclus în datele autentificate, iar un adversar nu poate modifica în mod liber configurația criptografică fără ca verificarea autenticatorului să eșueze.

Includerea datelor inițiale în mesajele Auth reprezintă de asemenea un punct forte al acestei lucrări. Datele sunt transmise protejate, prin criptare autentificată. Într-un protocol IKEv2 complet, ele ar corespunde unor informații necesare pentru stabilirea asocierilor de securitate IKE SA și ESP SA. În lucrare au fost eliminate anumite detalii funcționale, pentru ca implementarea să rămână clară și ușor de urmărit. Cerința de securitate a fost păstrată: aceste informații sunt transmise protejate.

A fost urmărită și cerința de ascundere a identității participanților, importantă în proiectarea IKEv2. În protocolul MAKE-IKE-SIG-PKC-DHE 4M, identitatea criptografică este ascunsă prin transmiterea certificatului în partea criptată a mesajului Auth. Astfel, certificatul nu este expus în primele mesaje ale protocolului. În varianta MAKE-IKE-MAC-PSK-DHE 4M, această proprietate nu a putut fi obținută complet, deoarece SJAKE nu distinge adresele de rețea de numele participanților. Identitatea este încă necesară pentru rutarea mesajelor în platformă. Aceasta este o limitare practică a mediului de implementare, nu a principiului criptografic analizat.

Testarea funcțională a fost etapa prin care s-a verificat dacă protocoalele implementate funcționează corect de la început până la final. Nu a fost suficient ca mesajele să fie transmise în ordinea corectă. Important a fost ca fiecare participant să ajungă la aceeași sesiune, să verifice autenticatorul celuilalt și să poată folosi cheia obținută pentru comunicație protejată. Rezultatele au confirmat acest

lucru. În toate protocoalele testate, participanții au ajuns în starea ESTABLISHED, ceea ce arată că schimbul de chei s-a încheiat cu succes. În cazul protocoalelor MAKE-IKE-MAC-PSK-DHE 4M și MAKE-IKE-SIG-PKC-DHE 4M, autentificarea mutuală a fost verificată explicit în log-uri, prin confirmarea autentificatorilor pentru Ana și Bob. Astfel, fiecare parte a putut confirma identitatea partenerului înainte de folosirea cheilor de sesiune. Testarea nu s-a oprit la finalizarea formală a protocolului. După stabilirea sesiunii, au fost transmise și mesaje de date între participanți. Aceste mesaje au fost acceptate de destinatar, ceea ce confirmă că ambele părți au derivat chei compatibile și că mecanismele de protecție a datelor funcționează corect.

Analiza de performanță a arătat că alegerea algoritmilor criptografici are un impact vizibil asupra timpului de execuție al protocoalelor. Diferențele nu apar doar din mecanismul de autentificare, ci și din tipul schimbului Diffie-Hellman folosit. În testele realizate, configurațiile bazate pe curbe eliptice au avut rezultate mai bune decât variantele bazate pe grupuri finite mari și RSA. Acest lucru era de așteptat, deoarece algoritmii pe curbe eliptice oferă un nivel de securitate comparabil folosind chei mai mici și operații mai eficiente. Cea mai rapidă configurație a fost MAKE-IKE-SIG-PKC-DHE 4M cu ECDH P-256. Rezultatele trebuie însă interpretate în contextul platformei SJAKE. Testul nu măsoară o singură operație criptografică izolată, ci întreaga execuție a protocolului. Timpul include generarea valorilor Diffie-Hellman, construirea și procesarea mesajelor, derivarea cheilor, autentificarea participanților și protejarea datelor. Din acest motiv, analiza este relevantă pentru implementarea completă a protocolului, nu doar pentru comparația teoretică dintre algoritmi.

Implementarea de atacuri Interleaving a completat analiza protocoalelor. Variantele vulnerabile au arătat că folosirea unor primitive sigure nu este suficientă dacă acestea sunt aplicate pe date greșite sau incomplete. În aceste tipuri de atac, adversarul nu sparge Diffie-Hellman, nu află cheia pre-partajată și nu falsifică semnături. El profită de faptul că autentificatorul nu acoperă întregul transcript al sesiunii. Din acest motiv, poate înlocui cheia publică Diffie-Hellman a inițiatorului și poate reutiliza un autentificator valid într-un context greșit. Protocoalele corecte rezistă acestor atacuri deoarece autentificatorii includ elementele importante ale sesiunii: algoritmul negociat, nonce-urile, valoarea Diffie-Hellman locală și MAC(id) derivat din cheia de sesiune. Această legătură face ca mesajele să nu poată fi mutate dintr-o sesiune în alta fără ca atacul să fie detectat. Aceasta este una dintre concluziile principale ale lucrării: securitatea unui protocol nu depinde doar de algoritmi folosiți, ci și de modul în care aceștia sunt combinați.

Lucrarea poate fi continuată în mai multe direcții. O primă direcție ar fi implementarea unei negocieri mai apropiate de IKEv2, cu mai multe oferte criptografice și alegerea uneia dintre ele de către responder. O altă direcție ar fi modificarea platformei astfel încât identitatea criptografică să fie separată clar de adresa de comunicație. Acest lucru ar permite ascunderea completă a identității și în varianta MAC-PSK. Analiza poate fi extinsă și prin testarea mai multor configurații criptografice, pe un număr mai mare de rulări. Astfel, comparația de performanță ar deveni mai stabilă. De asemenea, ar putea fi implementate și alte tipuri de atacuri, nu doar Interleaving, pentru a testa mai multe proprietăți de securitate. O direcție utilă ar fi și folosirea unor metode de verificare formală, care să completeze testarea practică realizată în SJAKE.

În final, lucrarea a arătat că proiectarea unui protocol criptografic nu înseamnă doar alegerea unor algoritmi considerați siguri. La fel de important este modul în care acești algoritmi sunt combinați în

interiorul protocolului. Schimbul Diffie-Hellman permite obținerea unui secret comun, autentificatorii confirmă identitatea participanților, iar criptarea autenticată protejează datele transmise. Fiecare mecanism are un rol clar, dar securitatea apare doar atunci când toate aceste elemente sunt legate corect între ele. Platforma SJAKE a făcut această idee mai ușor de realizat. Protocoalele corecte au demonstrat cum se stabilește o sesiune sigură, cum se verifică participanții și cum pot fi folosite cheile rezultate pentru comunicație protejată. În schimb, variantele vulnerabile și atacurile de tip Interleaving au arătat ce se poate întâmpla atunci când autentificatorul nu acoperă toate datele importante ale sesiunii.

Bibliografie

- [1] O. Catrina. Security Protocols. Lectures and labs, 2025.
- [2] Alfred J. Menezes, Paul C. van Oorschot and Scott A. Vanstone. Handbook of Applied Cryptography. CRC Press, 1996, 2001.
- [3] IETF. RFC 7296, Internet Key Exchange Protocol Version 2 (IKEv2), 2014.
- [4] IETF. RFC 4303, IP Encapsulating Security Payload (ESP), 2005.
- [5] O. Catrina. JAKE and SJAKE: Java Frameworks for Prototype Implementation and Analysis of Authenticated Key Exchange Protocols, Internal Research Report, 2024.
- [6] O. Catrina, R. Lupu. Experimental Analysis of Key Exchange Protocols Using the JAKE Framework. 14th International Conference on Communications (COMM 2022), Bucharest, Romania, pg. 1-6, IEEE, 2022.
- [7] O. Catrina, S. Stanciu. Comparative Performance Evaluation of Key Exchange Protocols. 14th International Conference on Communications (COMM 2022), June 16-18, 2022, Bucharest, Romania, pg. 1-6, IEEE, 2022.
- [8] Oracle. Java Cryptography Architecture (JCA) Reference Guide.
<https://docs.oracle.com/en/java/javase/17/security/java-cryptography-architecture-jca-reference-guide.html>
accesat la 12 martie 2026.
- [9] L. Chen, NIST Special Publication 800-56C. Recommendation for Key Derivation through Extraction-then-Expansion. NIST, 2011.
- [10] H. Krawczyk. SIGMA: the Sign-and-MAC Approach to Authenticated Diffie-Hellman and its use in IKE Protocols. Lecture Notes in Computer Science 2729(CRYPTO 2003), Springer, 2003.

Anexe

Anexa A1. Cod complet MAKE-IKE-SIG-PKC-DHE 4M

```
package secpro.pkc;

import java.security.KeyPair;
import java.security.SecureRandom;
import java.security.cert.X509Certificate;

import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import java.security.PublicKey;
import crypto.AutPub;

import comm.AppComm;
import crypto.AutSec;
import crypto.GenSessionKeyBytes;
import crypto.KeyAgrDH;
import crypto.ProtectedData;
import crypto.X509PKC;
import misc.Utills;
import secpro.AkeState;
import secpro.CryptoConfig;
import secpro.OwnCrypto;
import secpro.SecProBase;
import secpro.msg.AuthData;
import secpro.msg.AuthMsg;
import secpro.msg.InitAuthMsg;
import secpro.msg.InitMsg;
import secpro.msg.MsgType;
import tests.TestConst;

/**
 * MA-KE-SIG-PKC-DHE: variant with 4 messages.
 * Protocol with mutual authentication using
digital signatures and
 * public-key certificates; key exchange using
ephemeral Diffie-Hellman.
 *
 */

public class MakeSigIKE extends SecProBase
{
    // Diffie-Hellman
    private KeyPair locDHKeyPair; // local DH
key pair
    private byte[] remDHPubBytes; // remote DH
public key
    // Certificates
    private X509Certificate locCert;
    private X509Certificate remCert;
    // AKE protocol
    private int nonceLen = 256; // nonce bit-
length
    private byte[] locNonce;
    private byte[] remNonce;
    // Session key for MAC(locId)

    private SecretKey locKeyAuth; // MAC key
for own authenticator
    private SecretKey remKeyAuth; // MAC key
for remote authenticator
    // Algorithm negotiation
    private String locAlg;
    private String remAlg;
    //Data(msg)
    private byte[] locData;

    public MakeSigIKE(AppComm appComm, String
locId, byte[] sessionId,
        CryptoConfig commonCrypto,
OwnCrypto ownCrypto) throws Exception
    {
        super(appComm, locId, sessionId,
commonCrypto, ownCrypto);
        locCert =
X509PKC.getCert(ownCrypto.getCertFile());
        locAlg = commonCrypto.toString();
        remAlg = null;
        locData = null;
    }

    /**
     * Initiator starts handshakes.
     */
    public void doKeyExch(String locId, String
remId) throws Exception
    {
        // sw.start(); // start stopwatch
for initiator
        // InitMsg handshake
        if (state == AkeState.START) {
            this.remId = remId;
            initiator = true;
            locData =
TestConst.msg10.getBytes();
//generare nonce si DH
generateNonceAndDHKeys();
            state =
AkeState.INIT_SENT;

            appComm.sendInitMsg(buildInitMsg());
        }
    }

    /**
     * Builds InitMsg containing the
sender's alg, nonce and DH public key.
     * Variant with 4 messages: InitMsgA,
InitMsgB, AuthMsgA, AuthMsgB.
     */
}
```

```

        protected InitMsg buildInitMsg() throws
Exception
    {
        Utils.trace("[KeyEx: "+locId+"] build
InitMsg to " + remId + ", new state " + state);
        // InitMsg contains local alg,
local nonce and local ephemeral DH public key
        MsgType type = isInitiator()?
MsgType.INIT_A : MsgType.INIT_B;
        return new InitMsg(type, locId,
remId, sessionId,
            locNonce,

locDHKeyPair.getPublic().getEncoded(),
            null,
            locAlg.getBytes());
    }

/**
 * Builds AuthMsg containing the
sender's authenticator and certificate.
 * Variant with 4 messages: InitMsgA,
InitMsgB, AuthMsgA, AuthMsgB.
 */
    protected AuthMsg buildAuthMsg() throws
Exception
    {
        Utils.trace("[KeyEx: "+locId+"] build
AuthMsg to " + remId + ", new state " + state);
        // Compute own authenticator
        byte[] sig = generateAuth();
        // DataBA (responder)
        if (!isInitiator() && locData == null)
        {
            locData =
TestConst.msg11.getBytes();
        }

        AuthData authData = new AuthData(
            locCert.getEncoded(),
            sig,
            locData
        );
        // AE encryption
        ProtectedData encAuthData =
autEncSend.encrypt(authData.encode());
        MsgType type = isInitiator() ?
MsgType.AUTH_A : MsgType.AUTH_B;
        return new AuthMsg(type, locId, remId,
sessionId,
            null, null, encAuthData);
    }

/**
 * Generates the local authenticator.
 *
 * The authenticator is a digital
signature over:
 *   locAlg | locNonce | locDHPubKey |
remNonce | MAC(locId)
 */

```

```

        * The internal MAC(locId) is computed
with a session key derived from
        * the DH shared secret and provides key
confirmation.
        */
        protected byte[] generateAuth() throws
Exception
    {
        // Compute MAC(locId) with the local
authentication session key
        AutSec authSec = new
AutSec(commonCrypto.getAutSecName());
        authSec.setKey(locKeyAuth);
        byte[] mac =
authSec.genMac(locId.getBytes());

        // SIG authenticator
        AutPub autPub = new
AutPub(commonCrypto.getAutPubName());
        autPub.setPrivKey(
ownCrypto.getKeyManager().getPrivateKey(
            locId +
commonCrypto.getSigPrivKeyAlias()
        )
        );
        // Build the signed transcript:
        // alg | nonce | DH | nonce | MAC
        byte[] data =
Utils.concatenate(locAlg.getBytes(), locNonce);
        data = Utils.concatenate(data,
locDHKeyPair.getPublic().getEncoded());
        data = Utils.concatenate(data,
remNonce, mac);

        return autPub.genSig(data);
    }

/**
 * Verifies the received authenticator.
 *
 * Reconstructs the signed transcript:
 *   remAlg | remNonce | remDHPubKey |
locNonce | MAC(remId)
 * and verifies the remote digital
signature using the public key
 * extracted from the received
certificate.
 */
    protected boolean verifyAuth(byte[] auth)
throws Exception
    {
        if (remCert == null) {
            return false;
        }

        verifyCert(remCert);
        // Recompute MAC(remId) with the remote
authentication session key
        byte[] remIdMac = verifyIdMac();
        byte[] data =
Utils.concatenate(remAlg.getBytes(), remNonce);

```

```

        data = Utils.concatenate(data,
remDHPubBytes);
        data = Utils.concatenate(data,
locNonce, remIdMac);
        // Verify the signature using the public
key from the remote certificate
        AuthPub authPub = new
AuthPub(commonCrypto.getAuthPubName());

        authPub.setPubKey(remCert.getPublicKey()
);
        return authPub.verSig(data, auth);
    }

    /**
     * Processes received InitMsg.
     * Variant in 4 messages: InitMsgA,
InitMsgB, AuthMsgA, AuthMsgB.
     * Different order of operations:
optimistic/faster, pessimistic/slower.
     */
    public byte[] recvInitMsg(InitMsg msg)
throws Exception
    {
        Utils.trace("[KeyEx: "+locId+"]
recv InitMsg in state " + state);
        switch (state) {
            case START: // Responder receives
InitReqMsg

                // Store remote alg,
identity, nonce and DH public key
                remId = msg.getSrc();
                remNonce =
msg.getNonce();
                remDHPubBytes =
msg.getDHKey();
                remAlg = (msg.getAlg() ==
null) ?

                    null :
                    new String(msg.getAlg());
                if (remAlg == null ||
!remAlg.equals(locAlg)) {
                    sendFailMsg(remId, "Algorithm
mismatch");
                    return null;
                }
                // DataBA
                locData =
TestConst.msg11.getBytes();
                generateNonceAndDHKeys();
                // Update state and send
InitRspMsg

                    state =
AkeState.INIT_RCVD; // InitMsg received and sent

                    appComm.sendInitMsg(buildInitMsg());
                    genSessionKeys(); //
Variant: Before verifyAuth, faster
                    break;
                    case INIT_SENT: // Initiator
receives InitRspMsg

```

```

                // Store remote alg,
nonce and DH public key
                remNonce =
msg.getNonce();
                remDHPubBytes =
msg.getDHKey();
                remAlg = (msg.getAlg() ==
null) ?

                    null :
                    new String(msg.getAlg());

                if (remAlg == null ||
!remAlg.equals(locAlg)) {
                    sendFailMsg(remId, "Algorithm
mismatch");
                    return null;
                }
                genSessionKeys(); //
Variant: Before verifyAuth, faster
                // Update state and send
AuthReqMsg

                    state =
AkeState.AUTH_SENT; // InitMsg sent and
received, AuthMsg sent

                    appComm.sendAuthMsg(buildAuthMsg());
                    break;
                    case ESTABLISHED: // terminated +
success

                        break; // ignore (abort?)
                    case FAILED: // terminated + fail
                        break; // ignore
                    default: // abort: send FailMsg
and terminate + fail

                        sendFailMsg(remId,
"[KeyEx: "+locId+"] Illegal state " + state); //
-> state=FAILED
                    }
                    return null;
                }

            /**
             * Processes received AuthMsg.
             * Variant in 4 messages: InitMsgA,
InitMsgB, AuthMsgA, AuthMsgB.
             * Different order of operations:
optimistic/faster, pessimistic/slower.
             */
            public byte[] recvAuthMsg(AuthMsg msg)
throws Exception
            {
                Utils.trace("[KeyEx: "+locId+"]
recvAuth in state " + state);
                // Reject if message source does
not match the expected peer
                if (!remId.equals(msg.getSrc()))
                {
                    sendFailMsg(msg.getSrc(),
"[KeyEx: "+locId+"] Illegal state");
                    return null;
                }
            }

```

```

        byte[] ptxt = null; //AuthMsg
with early data
        switch(state) {
            case INIT_RCVD: // responder
                try {
                    //
sw.printDeltaTime("Responder recvAuthMsg");
                    AuthData authData
= new
AuthData(autEncRecv.decrypt(msg.getEData()));
                    remCert =
X509PKC.getCert(authData.getCert());
                    // Verify the
received digital signature
                    success =
verifyAuth(authData.getAuth());
                    if (success) {

                        Utils.trace("[KeyEx: "+locId+"]
Successful verification of " + remId + "'s
authenticator (SIG)\n"
                                + " Successful
authentication: remote user is " + remId);
//AuthMsg
with early data
                                ptxt =
authData.getData();

                                Utils.trace("[KeyEx: "+locId+"] Received
early data: " + new String(ptxt));
//
Variant: Send precomputed AuthRspMsg, faster

                                appComm.sendAuthMsg(buildAuthMsg());
                                state =
AkeState.ESTABLISHED; // AuthMsg received and
sent, success
//
sw.printElapsedTime("Responder duration");

                                Utils.trace("[KeyEx: "+locId+"]
Responder: Auth handshake done, state "
                                        + state + ". Success");

                                notifyTerminate();
                                }
                                else { // Reject.
Terminate.

                                Utils.trace("[KeyEx: "+locId+"] Failed
authentication of " + remId);

                                sendFailMsg(remId, "Responder: Key
exchange failed"); // -> state=FAILED
                                }
                                catch (Exception e) {

                                    System.out.println("[KeyEx: "+locId +
"]: Responder (recv AuthMsg). Key exchange
failed (exception)");

```

```

                                sendFailMsg(remId,
"Responder: Key exchange failed"); // ->
state=FAILED

                                e.printStackTrace();
                                }
                                break;
                                case AUTH_SENT: // initiator
                                    try {
                                        //
sw.printDeltaTime("Initiator recvAuthMsg");
                                        // Verify the
remote certificate from AuthMsg
                                        AuthData authData
= new
AuthData(autEncRecv.decrypt(msg.getEData()));
                                        remCert =
X509PKC.getCert(authData.getCert());
                                        // Verify the
digital signature
                                        success =
verifyAuth(authData.getAuth());
                                        if (success) {

                                            Utils.trace("[KeyEx: "+locId+"]
Successful verification of " + remId + "'s
authenticator (SIG)\n"
                                                    + " Successful authentication: remote
user is " + remId);
                                                    state =
AkeState.ESTABLISHED; // AuthMsg received and
sent, success
//
sw.printElapsedTime("Initiator total duration");

                                            Utils.trace("[KeyEx: "+locId+"]
Initiator: Auth handshake done, state "
                                                    + state + ". Success");
//AuthMsg
with early data
                                                    ptxt =
authData.getData();

                                            Utils.trace("[KeyEx: "+locId+"] Received
early data: " + new String(ptxt));

                                            notifyTerminate();
                                            }
                                            else { // Reject.
Terminate.

                                            Utils.trace("[KeyEx: "+locId+"] Failed
authentication of " + remId);

                                            sendFailMsg(remId, "Initiator: Key
exchange failed"); // -> state=FAILED
                                            }
                                            catch (Exception e) {

```

```

        System.out.println("[KeyEx: "+locId +
        "]: Initiator (recv AuthMsg). Key exchange
        failed (exception)");
        sendFailMsg(remId,
        "Initiator: Key exchange failed"); // ->
        state=FAILED

        e.printStackTrace();
        }
        break;
        case ESTABLISHED: // terminated +
success
        break; // ignore (abort?)
        case FAILED: // terminated + fail
        break; // ignore
        default: // abort: send FailMsg
and terminate + fail
        sendFailMsg(remId,
        "[KeyEx: "+locId+"] Illegal state " + state); //
-> state=FAILED
        }
        return null;
    }

    /* Not used in 4-message variant */
    public byte[]
recvInitAuthMsg(InitAuthMsg msg) throws
Exception
    {
        sendFailMsg(remId, "[KeyEx:
        "+locId+"] Illegal message " + state); // ->
        state=FAILED
        return null;
    }

    /**
     * Generates local protocol fresh
values:
     * - random local nonce
     * - local ephemeral DH key pair
     */

    private void generateNonceAndDHKeys()
throws Exception
    {
        SecureRandom rand=new
SecureRandom();
        locNonce= new byte[nonceLen/8];
        rand.nextBytes(locNonce);

        locDHKeyPair=KeyAgrDH.genDHKeyPair(commo
nCrypto.getDhName(),
commonCrypto.getDhKeyLen());

        KeyAgrDH.printDHPublicKey("[KeyEx:
        "+locId+"] DH public key:/n",
        locDHKeyPair.getPublic());
    }

    /**

```

```

     * Generates session keys for
authenticated-encryption (AE) in
     * protocols that compute session keys
using DH, nonces, and PRF.
     * These protocols use the extract and
expand method: extract a
     * random secret from the DH secret and
expand it when more random
     * bytes are necessary (longer keys, more
keys).
     * The selected AE scheme determines the
number of keys per data stream:
     * one key for dedicated AE scheme (GCM)
or two keys for Encrypt-then-MAC
     * (EtM) generic construction with
selected MAC and ENC-SEC schemes.
     *
     * Different keys for each direction.
     * Additional keys for MAC(locID)
authenticator.
     */
    protected void genSessionKeys() throws
Exception
    {
        // One key per direction if AE
with selected dedicated scheme (GCM)
        // Two keys per direction if AE
with EtM and selected ENC/MAC schemes
        // Two additional keys for
local/remote authenticator (MAC(id))
        int keyLen =
commonCrypto.getSecKeyLen() / 8;
        boolean isGCM =
(commonCrypto.getEncSecName().split("/") [1].equa
ls("GCM"));
        int randBytesLen = ((isGCM)? 4 :
6) * keyLen; // 2 + 2; 4 + 2
        String macName =
commonCrypto.getAutSecName();
        // Compute DH shared secret
        String dhName =
commonCrypto.getDhName();
        byte[] secretBytesDH =
KeyAgrDH.genDHSecret(dhName, locDHKeyPair,
remDHPubBytes);
        GenSessionKeyBytes genKeyBytes =
new GenSessionKeyBytes(macName);
        byte[] seed = (initiator)?

Utils.concatenate(locNonce, remNonce): //
initiator

Utils.concatenate(remNonce, locNonce); //
responder

        // The PRF for key derivation is
the selected MAC scheme
        SecretKey kdk =
genKeyBytes.extract(secretBytesDH, seed);
        byte[] randBytes =
genKeyBytes.expand(kdk, seed, randBytesLen);
        // Keys for local/remote
authenticator and for data send/receive

```

```

        byte[] authRandBytes = new
byte[2 * keyLen];
        byte[] dataRandBytes = new
byte[randBytesLen - 2 * keyLen];
        Utils.split(randBytes,
authRandBytes, dataRandBytes);
        byte[][] authKeyBytes = new
byte[2][keyLen];
        Utils.split(authRandBytes,
authKeyBytes);
        locKeyAuth = new
SecretKeySpec(authKeyBytes[initiator? 0 : 1],
macName);
        remKeyAuth = new
SecretKeySpec(authKeyBytes[initiator? 1 : 0],
macName);
        setSessionKeys(dataRandBytes);
    }

    /**
     * Recomputes MAC(remId) with the
remote authentication session key.
     */
    private byte[] verifyIdMac()
throws Exception
    {
        AutSec authSec = new
AutSec(commonCrypto.getAutSecName());

        authSec.setKey(remKeyAuth);
        return
authSec.genMac(remId.getBytes());
    }

    /**

```

```

        * Verifies a certificate: Gets
from the key store the trusted
        * certificate of the issuer and
verifies the CA signature of
        * the certificate. Verifies
validity interval and subject name.
        * Throws an exception if the
verification fails.
        */
        private void
verifyCert(X509Certificate cert) throws
Exception
        {
            X509Certificate
issuerCert =
ownCrypto.getKeyManager().getTrustedIssuerCert(c
ert);
            PublicKey issuerPubKey =
issuerCert.getPublicKey();
            // Verify certificate
signature with trusted issuer public key

            cert.verify(issuerPubKey);
            cert.checkValidity();

            String pkcSubName =
cert.getSubjectX500Principal().getName();
            String pkcComName =
pkcSubName.split(",")[0].split("=")[1];
            if
(!remId.equals(pkcComName)) {
                throw new
Exception("[KeyEx: "+locId+"] Invalid
Certificate: Wrong Subject name");
            }
        }
    }

```

Anexa A2. Cod complet MAKE-IKE-MAC-PSK-DHE 4M

```

package secpro.psk;

import java.security.KeyPair;
import java.security.SecureRandom;

import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;

import comm.AppComm;
import crypto.AutSec;
import crypto.GenSessionKeyBytes;
import crypto.KeyAgrDH;
import crypto.KeyManagerKS;
import crypto.ProtectedData;
import misc.Utils;
import secpro.AkeState;
import secpro.CryptoConfig;
import secpro.OwnCrypto;
import secpro.SecProBase;

```

```

import secpro.SetupPSK;
import secpro.msg.AuthData;
import secpro.msg.AuthMsg;
import secpro.msg.InitAuthMsg;
import secpro.msg.InitMsg;
import secpro.msg.MsgType;

import tests.TestConst;

/**
 * MA-KE-MAC-IKE: variant with 4 messages.
 * Protocol with mutual authentication using MAC
and PSK;
 * key exchange using ephemeral Diffie-Hellman.
 *
 *
 */

```

```

public class MakeMacIKE extends SecProBase
{
    // Diffie-Hellman
    private KeyPair locDHKeyPair;
    private byte[] remDHPubBytes;
    // AKE protocol
    private int nonceLen = 256;
    private byte[] locNonce;
    private byte[] remNonce;
    // Session keys for MAC(locId) / MAC(remId)
    private SecretKey locKeyAuth;
    private SecretKey remKeyAuth;
    // Algorithm negotiation (simulated)
    private String locAlg;
    private String remAlg;
    //Data(msg)
    private byte[] locData;

    public MakeMacIKE(AppComm appComm, String
locId, byte[] sessionId,
        CryptoConfig commonCrypto, OwnCrypto
ownCrypto) throws Exception
    {
        super(appComm, locId, sessionId,
commonCrypto, ownCrypto);
        locAlg = commonCrypto.toString();
        remAlg = null;
        locData = null;
    }

    /**
     * Initiator starts handshakes.
     */
    public void doKeyExch(String locId, String
remId) throws Exception
    {
        if (state == AkeState.START) {
            this.remId = remId;
            initiator = true;
            locData =
TestConst.msg10.getBytes();
            generateNonceAndDHKeys();
            state = AkeState.INIT_SENT;
            appComm.sendInitMsg(buildInitMsg());
        }

        /**
         * Builds InitMsg containing the sender's
         nonce and DH public key.
         * Variant with 4 messages: InitMsgA,
         InitMsgB, AuthMsgA, AuthMsgB.
         */
        protected InitMsg buildInitMsg() throws
Exception
        {
            Utils.trace("[KeyEx: " + locId + "] build
InitMsg to " + remId + ", new state " + state);
            // InitMsg contains alg, local nonce,
            local ephemeral DH public key,

```

```

MsgType type = isInitiator() ?
MsgType.INIT_A : MsgType.INIT_B;
        return new InitMsg(
            type,
            locId,
            remId,
            sessionId,
            locNonce,

            locDHKeyPair.getPublic().getEncoded(),
            null,
            locAlg.getBytes());
    }

    /**
     * Builds AuthMsg containing the sender's
     authenticator.
     * Variant with 4 messages: InitMsgA,
     InitMsgB, AuthMsgA, AuthMsgB.
     *
     * The authenticator and data are packed in
     AuthData and then
     * protected with authenticated encryption.
     */
    protected AuthMsg buildAuthMsg() throws
Exception
    {
        Utils.trace("[KeyEx: " + locId + "] build
AuthMsg to " + remId + ", new state " + state);
        // Compute own authenticator
        byte[] mac = generateAuth();
        // DataBA (responder)
        if (!isInitiator() && locData == null) {
            locData =
TestConst.msg11.getBytes();
        }
        AuthData authData = new AuthData(null,
mac, locData);
        ProtectedData encAuthData =
autEncSend.encrypt(authData.encode());
        MsgType type = isInitiator() ?
MsgType.AUTH_A : MsgType.AUTH_B;
        return new AuthMsg(type, locId, remId,
sessionId, null, null, encAuthData);
    }

    /**
     * Gets the PSK used for the main
     authenticator.
     *
     * The PSK is stored in the symmetric key
     store associated with the
     * local party, under alias remId + "_ake".
     */
    private SecretKey getAuthPsk() throws
Exception
    {
        String ksFile = "keys/skeys/" + locId +
".pkcs12";
        KeyManagerKS ks = new
KeyManagerKS(ksFile, SetupPSK.KS_PWD);
        ks.loadKeyStore();
    }

```

```

        return ks.getSecretKey(remId + "_ake");
    }

    /**
     * Generates the local authenticator.
     *
     * The authenticator is a MAC with the pre-
     shared key over:
     *   alg | locNonce | locDHPubKey | remNonce
     | MAC(locId)
     *
     * The internal MAC(locId) is computed with
     a session key derived from
     * the DH shared secret and provides
     additional key confirmation.
     */
    protected byte[] generateAuth() throws
    Exception
    {
        // Compute MAC(locId) with the local
        authentication session key
        AutSec authSecId = new
        AutSec(commonCrypto.getAutSecName());
        authSecId.setKey(locKeyAuth);
        byte[] macId =
        authSecId.genMac(locId.getBytes());
        // Main MAC authenticator with PSK
        AutSec authSec = new
        AutSec(commonCrypto.getAutSecName());
        SecretKey kauKey = getAuthPsk();
        authSec.setKey(kauKey);
        // Build the authenticated transcript:
        // alg | nonce | DH | nonce | MAC
        byte[] data =
        Utils.concatenate(locAlg.getBytes(), locNonce);
        data = Utils.concatenate(data,
        locDHKeyPair.getPublic().getEncoded());
        data = Utils.concatenate(data, remNonce,
        macId);
        return authSec.genMac(data);
    }

    /**
     * Verifies the received authenticator.
     *
     * Reconstructs the authenticated transcript:
     *   remAlg | remNonce | remDHPubKey |
     locNonce | MAC(remId)
     * and verifies the received MAC using the
     shared PSK.
     */
    protected boolean verifyAuth(byte[] auth)
    throws Exception
    {
        // Recompute MAC(remId) with the remote
        authentication session key
        AutSec authSecId = new
        AutSec(commonCrypto.getAutSecName());
        authSecId.setKey(remKeyAuth);
        byte[] remIdMac =
        authSecId.genMac(remId.getBytes());
        // Main MAC authenticator with PSK

```

```

        AutSec authSec = new
        AutSec(commonCrypto.getAutSecName());
        SecretKey kauKey = getAuthPsk();
        authSec.setKey(kauKey);
        byte[] data =
        Utils.concatenate(remAlg.getBytes(), remNonce);
        data = Utils.concatenate(data,
        remDHPubBytes);
        data = Utils.concatenate(data, locNonce,
        remIdMac);

        return authSec.verMac(data, auth);
    }

    /**
     * Processes received InitMsg.
     * Variant in 4 messages: InitMsgA, InitMsgB,
     AuthMsgA, AuthMsgB.
     * Different order of operations:
     optimistic/faster, pessimistic/slower.
     */
    public byte[] recvInitMsg(InitMsg msg) throws
    Exception
    {
        Utils.trace("[KeyEx: " + locId + "] recv
        InitMsg in state " + state);
        switch (state) {
            case START:
                // Responder receives InitReqMsg
                // Store remote identity, nonce, DH
                public key and algorithm
                remId = msg.getSrc();
                remNonce = msg.getNonce();
                remDHPubBytes = msg.getDhKey();
                remAlg = (msg.getAlg() == null)
                ? null
                : new String(msg.getAlg());

                if (remAlg == null ||
                !remAlg.equals(locAlg)) {
                    sendFailMsg(remId, "Algorithm
                    mismatch");
                    return null;
                }
                // DataBA
                locData =
                TestConst.msg11.getBytes();
                generateNonceAndDHKeys();
                // Update state and send InitRspMsg
                state = AkeState.INIT_RCVD;
                appComm.sendInitMsg(buildInitMsg());
                // Variant: Before verifyAuth, faster
                genSessionKeys();
                break;
            case INIT_SENT:
                // Initiator receives InitRspMsg
                // Reject if response source does not
                match expected peer
                if (!msg.getSrc().equals(remId)) {
                    sendFailMsg(msg.getSrc(),
                    "[KeyEx: " + locId + "] Illegal peer");
                    return null;
                }

```

```

    }
    // Store remote nonce, DH public key
    and algorithm
    remNonce = msg.getNonce();
    remDHPubBytes = msg.getDHKey();
    remAlg = (msg.getAlg() == null)
        ? null
        : new String(msg.getAlg());

    if (remAlg == null ||
!remAlg.equals(locAlg)) {
        sendFailMsg(remId, "Algorithm
mismatch");
        return null;
    }
    // Variant: Before verifyAuth, faster
    genSessionKeys();
    // Update state and send AuthReqMsg
    state = AkeState.AUTH_SENT;//
InitMsg sent and received, AuthMsg sent
    appComm.sendAuthMsg(buildAuthMsg());
    break;
    case ESTABLISHED: // terminated + success
        break; //ignore (abort?)
    case FAILED: // terminated + fail
        break; // ignore
    default:
        // abort: send FailMsg and terminate
+ fail
        sendFailMsg(remId, "[KeyEx: " + locId
+ "]" Illegal state " + state); // -> state=FAILED
    }
    return null;
}

/**
 * Processes received AuthMsg.
 * Variant in 4 messages: InitMsgA, InitMsgB,
AuthMsgA, AuthMsgB.
 * Different order of operations:
optimistic/faster, pessimistic/slower.
 */
public byte[] recvAuthMsg(AuthMsg msg) throws
Exception
{
    Utils.trace("[KeyEx: " + locId + "]"
recvAuth in state " + state);
    // Reject if message source does not
match the expected peer
    if (!remId.equals(msg.getSrc())) {
        sendFailMsg(msg.getSrc(), "[KeyEx: "
+ locId + "]" Illegal state");
        return null;
    }
    byte[] ptxt = null; // AuthMsg with early
data
    // sw.start();
    switch (state) {
    case INIT_RCVD: // responder
        try {
            // sw.printDeltaTime("Responder
recvAuthMsg");

```

```

        AuthData authData = new
AuthData(autEncRecv.decrypt(msg.getEData()));
        // Verify the received MAC
        authenticator
            success =
verifyAuth(authData.getAuth());
            if (success) {
                Utils.trace("[KeyEx: " +
locId + "]" Successful verification of " + remId
+ "'s authenticator
(MAC)\n Successful authentication: remote user
is " + remId);
                // AuthMsg with early data
                ptxt =
authData.getData();

                Utils.trace("[KeyEx: "+locId+"] Received
early data: " + new String(ptxt));
                // Variant: Send precomputed
AuthRspMsg, faster
                appComm.sendAuthMsg(buildAuthMsg());
                state =
AkeState.ESTABLISHED;
                //
                sw.printElapsedTime("Responder duration");
                Utils.trace("[KeyEx: " +
locId + "]" Responder: Auth handshake done, state
" + state + ". Success");
                notifyTerminate();
            }
            else {
                // Reject. Terminate.
                Utils.trace("[KeyEx: " +
locId + "]" Failed authentication of " + remId);
                sendFailMsg(remId,
"Responder: Key exchange failed");// ->
state=FAILED
            }
        }
        catch (Exception e) {
            System.out.println("[KeyEx: " +
locId + "]: Responder (recv AuthMsg). Key
exchange failed (exception)");
            sendFailMsg(remId, "Responder:
Key exchange failed");// -> state=FAILED
            e.printStackTrace();
        }
        break;

        case AUTH_SENT: // initiator
            try {
                // sw.printDeltaTime("Initiator
recvAuthMsg");
                AuthData authData = new
AuthData(autEncRecv.decrypt(msg.getEData()));
                // Verify the received MAC
                authenticator
                    success =
verifyAuth(authData.getAuth());
                    if (success) {

```

```

        Utils.trace("[KeyEx: " +
locId + "] Successful verification of " + remId
        + "'s authenticator
(MAC)\n Successful authentication: remote user
is " + remId);

        state =
AkeState.ESTABLISHED;// AuthMsg received and
sent, success

        //
sw.printElapsedTime("Initiator total duration");
        Utils.trace("[KeyEx: " +
locId + "] Initiator: Auth handshake done, state
" + state + ". Success");
        // AuthMsg with early data
        ptxt =
authData.getData();

        Utils.trace("[KeyEx: "+locId+"] Received
early data: " + new String(ptxt));
        notifyTerminate();
    }
    else {
        // Reject. Terminate.
        Utils.trace("[KeyEx: " +
locId + "] Failed authentication of " + remId);
        sendFailMsg(remId,
"Initiator: Key exchange failed");// ->
state=FAILED
    }
    catch (Exception e) {
        System.out.println("[KeyEx: " +
locId + "]: Initiator (recv AuthMsg). Key
exchange failed (exception)");
        sendFailMsg(remId, "Initiator:
Key exchange failed");// -> state=FAILED
        e.printStackTrace();
    }
    break;
    case ESTABLISHED:// terminated + success
        break;// ignore (abort?)
    case FAILED:// terminated + fail
        break;// ignore
    default:
        // abort: send FailMsg and terminate
+ fail
        sendFailMsg(remId, "[KeyEx: " + locId
+ "] Illegal state " + state);
    }

    return null;
}

/* Not used in 4-message variant */
public byte[] recvInitAuthMsg(InitAuthMsg
msg) throws Exception
{
    sendFailMsg(remId, "[KeyEx: " + locId +
"] Illegal message " + state);
    return null;
}

```

```

/**
 * Generates local protocol fresh values:
 * - random local nonce
 * - local ephemeral DH key pair
 */
private void generateNonceAndDHKeys() throws
Exception
{
    SecureRandom rand = new SecureRandom();
    locNonce = new byte[nonceLen / 8];
    rand.nextBytes(locNonce);
    locDHKeyPair =
KeyAgrDH.genDHKeyPair(commonCrypto.getDhName(),
commonCrypto.getDhKeyLen());
    KeyAgrDH.printDHPublicKey("[KeyEx: " +
locId + " ] DH public key:\n",
locDHKeyPair.getPublic());
}

/**
 * Generates session keys for authenticated-
encryption (AE) in
 * protocols that compute session keys using
DH, nonces, and PRF.
 * These protocols use the extract and expand
method: extract a
 * random secret from the DH secret and expand
it when more random
 * bytes are necessary (longer keys, more
keys).
 * The selected AE scheme determines the
number of keys per data stream:
 * one key for dedicated AE scheme (GCM) or
two keys for Encrypt-then-MAC
 * (EtM) generic construction with selected
MAC and ENC-SEC schemes.
 *
 * Different keys for each direction.
 * Additional keys for MAC(locID)
authenticator.
 */
protected void genSessionKeys() throws
Exception
{
    // One key per direction if AE with
selected dedicated scheme (GCM)
    // Two keys per direction if AE with EtM
and selected ENC/MAC schemes
    // Two additional keys for local/remote
authenticator (MAC(id))
    int keyLen = commonCrypto.getSecKeyLen()
/ 8;

    boolean isGCM =
commonCrypto.getEncSecName().split("/")[1].equal
s("GCM");
    int randBytesLen = (isGCM ? 4 : 6) *
keyLen;

    String macName =
commonCrypto.getAutSecName();
    // Compute DH shared secret
    String dhName =
commonCrypto.getDhName();

```

```

        byte[] secretBytesDH =
KeyAgrDH.genDHSecret(dhName, locDHKeyPair,
remDHPubBytes);

        byte[] seed = initiator
? Utils.concatenate(locNonce,
remNonce)
: Utils.concatenate(remNonce,
locNonce);
// The PRF for key derivation is the
selected MAC scheme
        GenSessionKeyBytes genKeyBytes = new
GenSessionKeyBytes(macName);

        SecretKey kdk =
genKeyBytes.extract(secretBytesDH, seed);
        byte[] randBytes =
genKeyBytes.expand(kdk, seed, randBytesLen);
// Keys for local/remote authenticator
and for data send/receive
        byte[] authRandBytes = new byte[2 *
keyLen];

```

```

        byte[] dataRandBytes = new
byte[randBytesLen - 2 * keyLen];
        Utils.split(randBytes, authRandBytes,
dataRandBytes);
        byte[][] authKeyBytes = new
byte[2][keyLen];
        Utils.split(authRandBytes,
authKeyBytes);

        locKeyAuth = new
SecretKeySpec(authKeyBytes[initiator ? 0 : 1],
macName);
        remKeyAuth = new
SecretKeySpec(authKeyBytes[initiator ? 1 : 0],
macName);
        setSessionKeys(dataRandBytes);
    }
}

```

Anexa A3. Cod complet MAKE-ESP-MAC-PSK-DHE

```

package secpro.psk;

import java.security.KeyPair;
import java.security.SecureRandom;

import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;

import comm.AppComm;
import crypto.AutoEnc;
import crypto.GenSessionKeyBytes;
import crypto.KeyAgrDH;
import crypto.KeyManagerKS;
import crypto.ProtectedData;
import misc.Utils;
import secpro.AkeState;
import secpro.CryptoConfig;
import secpro.OwnCrypto;
import secpro.SecProBase;
import secpro.SetupPSK;
import secpro.msg.AuthMsg;
import secpro.msg.InitAuthMsg;
import secpro.msg.InitMsg;
import secpro.msg.MsgType;

/**
 * MAKE-ESP-MAC-PSK-DHE: 2-message protocol.
 *
 * Protocol used after an IKE SA was already
established.
 * Peers are already authenticated and use an
existing PSK only to
 * protect the new keying material with
authenticated encryption.
 *

```

```

 * The AE-protected payload is carried in
InitMsg.eData.
 */
public class MakeMacESP extends SecProBase
{
    // Diffie-Hellman
    private KeyPair locDHKeyPair;

    private byte[] remDHPubBytes;
    // AKE protocol
    private final int nonceLen = 256;
    private byte[] locNonce;
    private byte[] remNonce;
    // Algorithm negotiation (simulated)
    private String locAlg;
    private String remAlg;
    // AE protected with a key derived from the
PSK
    private AutoEnc aeWithPsk;

    public MakeMacESP(AppComm appComm, String
locId, byte[] sessionId,
CryptoConfig commonCrypto,
OwnCrypto ownCrypto) throws Exception
    {
        super(appComm, locId, sessionId,
commonCrypto, ownCrypto);
        locAlg = commonCrypto.toString();
        remAlg = null;
    }

    /**
 * Initiator starts the 2-message exchange.

```

```

        */
        public void doKeyExch(String locId, String
remId) throws Exception
        {
            if (state == AkeState.START) {
                this.remId = remId;
                initiator = true;
                initAeWithPsk();
                generateNonceAndDHKeys();
                state = AkeState.INIT_SENT;

appComm.sendInitMsg(buildInitMsg());
            }
        }

        /**
         * Initializes AE with key material derived
         from the PSK.
         * a single PSK-derived AE key set is used
         in both directions.
         */
        private void initAeWithPsk() throws
Exception
        {
            SecretKey psk = getEspPsk();
            String macName =
commonCrypto.getAutSecName();
            String encName =
commonCrypto.getEncSecName();
            int keyLen =
commonCrypto.getSecKeyLen() / 8;
            boolean isGCM =
encName.split("/") [1].equals("GCM");
            // Symmetric seed so both peers derive
the same AE key set
            String id1 = (locId.compareTo(remId) <=
0) ? locId : remId;
            String id2 = (locId.compareTo(remId) <=
0) ? remId : locId;
            byte[] seed = Utils.concatenate(
                id1.getBytes(),
                id2.getBytes());
            GenSessionKeyBytes genKeyBytes = new
GenSessionKeyBytes(macName);
            SecretKey kdk =
genKeyBytes.extract(psk.getEncoded(), seed);
            int numKeyBytes = isGCM ? keyLen : 2 *
keyLen;
            byte[] derivedBytes =
genKeyBytes.expand(kdk, seed, numKeyBytes);
            String encAlgName =
encName.split("/") [0];
            aeWithPsk = new AutEnc(encName,
macName);
            aeWithPsk.setKeyEnc(new
SecretKeySpec(derivedBytes, 0, keyLen,
encAlgName));
            if (!isGCM) {
                aeWithPsk.setKeyMac(new
SecretKeySpec(derivedBytes, keyLen, keyLen,
macName));
            }
        }

```

```

        }

        /**
         * Gets the PSK used for AE protection of
         INIT messages.
         * Stored under alias remId + "_ake" in the
         local key store.
         */
        private SecretKey getEspPsk() throws
Exception
        {
            String ksFile = "keys/skeys/" + locId +
".pkcs12";
            KeyManagerKS ks = new
KeyManagerKS(ksFile, SetupPSK.KS_PWD);
            ks.loadKeyStore();
            return ks.getSecretKey(remId + "_ake");
        }

        /**
         * Builds INIT_A / INIT_B carrying:
         * eData = AE(alg | nonce | dhKey)
         */
        protected InitMsg buildInitMsg() throws
Exception
        {
            Utils.trace("[KeyEx: " + locId + "]
build InitMsg to " + remId + ", new state " +
state);
            byte[] dhPubBytes =
locDHKeyPair.getPublic().getEncoded();
            byte[] payload = encodeInitData(locAlg,
locNonce, dhPubBytes);
            ProtectedData encData =
aeWithPsk.encrypt(payload);
            MsgType type = isInitiator() ?
MsgType.INIT_A : MsgType.INIT_B;
            return new InitMsg(
                type,
                locId,
                remId,
                sessionId,
                null,
                null,
                null,
                encData
            );
        }

        /**
         * Processes received InitMsg.
         */
        public byte[] recvInitMsg(InitMsg msg)
throws Exception
        {
            Utils.trace("[KeyEx: " + locId + "]
recv InitMsg in state " + state);
            switch (state) {
                case START:
                    // Responder receives INIT_A
                    remId = msg.getSrc();
            }
        }

```

```

        initiator = false;
        initAeWithPsk();
        if (!verifyAndExtract(msg)) {
            sendFailMsg(remId, "[KeyEx: " +
locId + "] AE verification failed on InitMsg");
            return null;
        }
        generateNonceAndDHKeys();
        genSessionKeys();
        // Reuse generic INIT_SENT state
before sending INIT_B
        state = AkeState.INIT_SENT;

appComm.sendInitMsg(buildInitMsg());
        state = AkeState.ESTABLISHED;
        success = true;
        Utils.trace("[KeyEx: " + locId + "]
Responder: handshake done, state "
            + state + ". Success");
        notifyTerminate();
        break;
        case INIT_SENT:
            // Initiator receives INIT_B
            if (!msg.getSrc().equals(remId)) {
                sendFailMsg(msg.getSrc(),
"[KeyEx: " + locId + "] Illegal peer in state
INIT_SENT");
                return null;
            }
            if (!verifyAndExtract(msg)) {
                sendFailMsg(remId, "[KeyEx: " +
locId + "] AE verification failed on InitMsg");
                return null;
            }
            genSessionKeys();
            state = AkeState.ESTABLISHED;
            success = true;
            Utils.trace("[KeyEx: " + locId + "]
Initiator: handshake done, state "
                + state + ". Success");
            notifyTerminate();
            break;
        case ESTABLISHED: // terminated +
success
            break;//ignore (abort?)
        case FAILED: // terminated + fail
            break; // ignore
        default:
            // abort: send FailMsg and
terminate + fail
            sendFailMsg(remId, "[KeyEx: " +
locId + "] Illegal state " + state); // ->
state=FAILED
        }
        return null;
    }

/**
 * Decrypts and verifies the AE-protected
init payload and extracts:
 * remAlg, remNonce, remDHPubBytes
 * Checks remAlg == locAlg.

```

```

        */
        private boolean verifyAndExtract(InitMsg
msg)
        {
            ProtectedData eData = msg.getEData();
            if (eData == null) {
                return false;
            }
            try {
                byte[] plaintext =
aeWithPsk.decrypt(eData);
                if (plaintext == null ||
plaintext.length < 6) {
                    return false;
                }
                decodeInitData(plaintext);

                if (remAlg == null ||
!remAlg.equals(locAlg)) {
                    Utils.trace("[KeyEx: " + locId
+ "] Algorithm mismatch: local=" + locAlg
+ ", remote=" +
remAlg);
                    return false;
                }
                if (remNonce == null ||
remNonce.length != nonceLen / 8) {
                    return false;
                }
                if (remDHPubBytes == null ||
remDHPubBytes.length == 0) {
                    return false;
                }
                return true;
            }
            catch (Exception e) {
                Utils.trace("[KeyEx: " + locId + "]
AE verification failed: " + e.getMessage());
                return false;
            }
        }

        public byte[] recvInitAuthMsg(InitAuthMsg
msg) throws Exception
        {
            sendFailMsg(remId, "[KeyEx: " + locId +
"] Unexpected InitAuthMsg in state " + state);
            return null;
        }

        public byte[] recvAuthMsg(AuthMsg msg)
throws Exception
        {
            sendFailMsg(remId, "[KeyEx: " + locId +
"] Unexpected AuthMsg in state " + state);
            return null;
        }

        /**
 * Generates local nonce and ephemeral DH
key pair.
 */

```

```

    private void generateNonceAndDHKeys()
    throws Exception
    {
        SecureRandom rand = new SecureRandom();
        locNonce = new byte[nonceLen / 8];
        rand.nextBytes(locNonce);
        locDHKeyPair =
        KeyAgrDH.genDHKeyPair(commonCrypto.getDhName(),
        commonCrypto.getDhKeyLen());
        KeyAgrDH.printDHPublicKey("[KeyEx: " +
        locId + " ] DH public key:\n",
        locDHKeyPair.getPublic());
    }

    /**
     * Generates session keys for the new ESP
     SA using DH and nonces.
     *
     * K_S = PRF_N(g^xy), where N = N_A | N_B
     * two keys per direction for EtM.
     */
    protected void genSessionKeys() throws
    Exception
    {
        int keyLen =
        commonCrypto.getSecKeyLen() / 8;
        boolean isGCM =
        commonCrypto.getEncSecName().split("/") [1].equal
        s("GCM");
        int randBytesLen = (isGCM ? 2 : 4) *
        keyLen;
        String macName =
        commonCrypto.getAutSecName();
        // Compute DH shared secret
        String dhName =
        commonCrypto.getDhName();
        byte[] secretBytesDH =
        KeyAgrDH.genDHSecret(dhName, locDHKeyPair,
        remDHPubBytes);
        byte[] seed = initiator
        ? Utils.concatenate(locNonce,
        remNonce)
        : Utils.concatenate(remNonce,
        locNonce);
        // The PRF for key derivation
        GenSessionKeyBytes genKeyBytes = new
        GenSessionKeyBytes(macName);
        SecretKey kdk =
        genKeyBytes.extract(secretBytesDH, seed);
        byte[] randBytes =
        genKeyBytes.expand(kdk, seed, randBytesLen);
        setSessionKeys(randBytes);
    }

    /**
     * Encodes: alg | nonce | dhKey
     * as: [2B algLen][alg][2B
        nonceLen][nonce][2B dhLen][dhKey]
     */
    private byte[] encodeInitData(String alg,
        byte[] nonce, byte[] dhKey)
    {

```

```

        byte[] algBytes = (alg == null) ? null
        : alg.getBytes();

        int algLen = (algBytes == null) ? 0 :
        algBytes.length;
        int nonceLen = (nonce == null) ? 0 :
        nonce.length;
        int dhLen = (dhKey == null) ? 0 :
        dhKey.length;
        int encodedLen = 6 + algLen + nonceLen
        + dhLen;
        byte[] encodedBytes = new
        byte[encodedLen];
        int pos = 0;

        System.arraycopy(Utils.int16ToBytes(algLen), 0,
        encodedBytes, pos, 2);
        pos += 2;
        if (algBytes != null) {
            System.arraycopy(algBytes, 0,
            encodedBytes, pos, algLen);
            pos += algLen;
        }

        System.arraycopy(Utils.int16ToBytes(nonceLen),
        0, encodedBytes, pos, 2);
        pos += 2;
        if (nonce != null) {
            System.arraycopy(nonce, 0,
            encodedBytes, pos, nonceLen);
            pos += nonceLen;
        }

        System.arraycopy(Utils.int16ToBytes(dhLen), 0,
        encodedBytes, pos, 2);
        pos += 2;
        if (dhKey != null) {
            System.arraycopy(dhKey, 0,
            encodedBytes, pos, dhLen);
        }
        return encodedBytes;
    }

    /**
     * Decodes: [2B algLen][alg][2B
        nonceLen][nonce][2B dhLen][dhKey]
     * and stores the received values in:
        remAlg, remNonce, remDHPubBytes
     */
    private void decodeInitData(byte[]
        encodedBytes) throws Exception
    {
        int pos = 0;
        if (encodedBytes == null ||
        encodedBytes.length < 6) {
            throw new
            IllegalArgumentException("Invalid init data");
        }
        int len = Utils.bytesToInt16(new byte[]
        { encodedBytes[pos], encodedBytes[pos + 1] });
        pos += 2;

```

```

        if (len <= 0 || pos + len >
encodedBytes.length) {
            throw new
IllegalArgumentException("Invalid alg length");
        }
        byte[] algBytes = new byte[len];
        System.arraycopy(encodedBytes, pos,
algBytes, 0, len);
        remAlg = new String(algBytes);
        pos += len;
        len = Utils.bytesToInt16(new byte[] {
encodedBytes[pos], encodedBytes[pos + 1] });
        pos += 2;
        if (len <= 0 || pos + len >
encodedBytes.length) {
            throw new
IllegalArgumentException("Invalid nonce
length");
        }
        remNonce = new byte[len];
        System.arraycopy(encodedBytes, pos,
remNonce, 0, len);
    }
}

```

```

        pos += len;
        len = Utils.bytesToInt16(new byte[] {
encodedBytes[pos], encodedBytes[pos + 1] });
        pos += 2;
        if (len <= 0 || pos + len >
encodedBytes.length) {
            throw new
IllegalArgumentException("Invalid DH key
length");
        }
        remDHPubBytes = new byte[len];
        System.arraycopy(encodedBytes, pos,
remDHPubBytes, 0, len);
        pos += len;
        if (pos != encodedBytes.length) {
            throw new
IllegalArgumentException("Trailing bytes in init
data");
        }
    }
}

```

Anexa A4. Cod complet atac ILV asupra SIG-PKC

```

package secpro.pkc;

import java.security.KeyPair;

import javax.crypto.SecretKey;

import comm.AppComm;
import crypto.GenSessionKeyBytes;
import crypto.KeyAgrDH;
import misc.Utils;
import secpro.AkeState;
import secpro.CryptoConfig;
import secpro.OwnCrypto;
import secpro.SecProBase;
import secpro.msg.AuthMsg;
import secpro.msg.FailMsg;
import secpro.msg.InitAuthMsg;
import secpro.msg.InitMsg;
import secpro.msg.MsgType;

/**
 *
 * Adversary implementation for interleaving
attack (ILV) on
 * MAKE-SIG-PKC-DHE 4-message vulnerable
protocol.
 *
 * The adversary impersonates A to B by:
 * - intercepting InitMsg from A
 * - substituting own DH public key
 * - forwarding messages between A and B

```

```

 * - reusing A's signature and certificate to
authenticate to B
 */

public class AdvMakeSigIKEILV extends
SecProBase
{
    // Honest parties identities
    private String anaId;
    private String bobId;
    // Attack session identifier (same as Ana's
session)
    private byte[] sidAtk;
    // Nonces used in the interleaving attack

    private byte[] nonceA;
    private byte[] nonceB;
    // Bob's DH public key intercepted from
InitBA
    private byte[] bobDHPubBytes;
    // Adversary's own ephemeral DH key pair
    private KeyPair advDHKeyPair;

    public AdvMakeSigIKEILV(AppComm appComm,
String locId, byte[] sessionId,
CryptoConfig commonCrypto,
OwnCrypto ownCrypto) throws Exception
    {
        super(appComm, locId, sessionId,
commonCrypto, ownCrypto);
    }
}

```

```

    }

    /**
     * Initializes adversary state.
     * No active message is sent; waits for
    intercepted traffic.
     */
    public void doKeyExch(String locId, String
remId) throws Exception
    {
        this.locId = locId;
        this.remId = remId;
        this.state = AkeState.START;
        this.success = false;

        this.anaId = null;
        this.bobId = null;
        this.sidAtk = null;
        this.nonceA = null;
        this.nonceB = null;
        this.bobDHPubBytes = null;
        this.advDHKeyPair = null;

        Utils.trace("[AdvKeyEx: " + this.locId
+ "]" waiting for InitAB...");
    }

    /**
     * Processes intercepted InitMsg.
     *
     * Two roles:
     * - First InitMsg: from A -> intercepted
    and modified -> sent to B
     * - Second InitMsg: from B -> forwarded
    to A
     */
    public byte[] recvInitMsg(InitMsg msg)
    throws Exception
    {
        Utils.trace("[AdvKeyEx: " + locId + "]"
recv InitMsg in state " + state
+ " from " + msg.getSrc() + "
to " + msg.getDst());

        switch (state) {
        case START:
            // intercept InitAB (A -> B)
            anaId = msg.getSrc();
            bobId = msg.getDst();
            // Store Bob as the real peer of
the adversary session
            remId = bobId;
            // Adversary plays initiator role
towards Bob
            initiator = true;
            // Reuse Ana's nonce and session
identifier
            nonceA = msg.getNonce();
            sidAtk = msg.getSid();
            // Generate adversary DH key pair
            advDHKeyPair =
KeyAgrDH.genDHKeyPair(

```

```

commonCrypto.getDhName(),
commonCrypto.getDhKeyLen()
);
// Send forged InitMsg to B with
Ana's identity and nonce,
// but adversary's own DH public key
InitMsg initToBob = new InitMsg(
    MessageType.INIT_A,
    anaId,
    bobId,
    sidAtk,
    nonceA,

advDHKeyPair.getPublic().getEncoded(),
    null,
    null
);
state = AkeState.INIT_SENT;
Utils.trace("[AdvKeyEx: " + locId +
"] send forged InitAB to " + bobId
+ ", new state " + state);
appComm.sendInitMsg(initToBob);
break;

case INIT_SENT:
    // intercept InitBA (B -> A)
    if (!msg.getSrc().equals(bobId) ||
!msg.getDst().equals(anaId)) {
        sendFailMsg(msg.getSrc(),
            "[AdvKeyEx: " + locId +
"] Illegal peer in INIT_SENT");
        return null;
    }
    // Store Bob's nonce and DH public
key
    nonceB = msg.getNonce();
    bobDHPubBytes = msg.getDhKey();
    // Forward Bob's InitMsg to Ana
without modification
    InitMsg initToAna = new InitMsg(
        MessageType.INIT_B,
        bobId,
        anaId,
        sidAtk,
        nonceB,
        bobDHPubBytes,
        null,
        null
    );
    state = AkeState.INIT_RCVD;
    Utils.trace("[AdvKeyEx: " + locId +
"] relay InitBA to " + anaId
+ ", new state " + state);
    appComm.sendInitMsg(initToAna);
    break;
case ESTABLISHED:
    break;
case FAILED:
    break;
default:
    sendFailMsg(msg.getSrc(),

```

```

        "[AdvKeyEx: " + locId + "]
Illegal state " + state);
    }
    return null;
}

/**
 * Processes intercepted AuthMsg.
 *
 * Steps:
 * - intercept AuthAB -> forward to B
 * - intercept AuthBA -> derive attack
success and abort S1
 */
public byte[] recvAuthMsg(AuthMsg msg)
throws Exception
{
    Utils.trace("[AdvKeyEx: " + locId + "]
recv AuthMsg in state " + state
+ " from " + msg.getSrc() + "
to " + msg.getDst());

    switch (state) {
    case INIT_RCVD:
        // intercept AuthAB (A -> B)
        if (!msg.getSrc().equals(anaId) ||
!msg.getDst().equals(bobId)) {
            sendFailMsg(msg.getSrc(),
                "[AdvKeyEx: " + locId +
"] Illegal peer in INIT_RCVD");
            return null;
        }
        // Forward Ana's authenticator and
certificate to Bob unchanged
        AuthMsg authToBob = new AuthMsg(
            MsgType.AUTH_A,
            anaId,
            bobId,
            sidAtk,
            msg.getAuth(),
            msg.getCert(),
            null
        );
        state = AkeState.AUTH_SENT;
        Utils.trace("[AdvKeyEx: " + locId +
"] relay AuthAB to " + bobId
+ ", new state " + state);
        appComm.sendAuthMsg(authToBob);
        break;
    case AUTH_SENT:
        // intercept AuthBA (B -> A)
        if (!msg.getSrc().equals(bobId) ||
!msg.getDst().equals(anaId)) {
            sendFailMsg(msg.getSrc(),
                "[AdvKeyEx: " + locId +
"] Illegal peer in AUTH_SENT");
            return null;
        }

        try {
            // Derive session keys using
adversary DH private key and Bob's public key

```

```

            genSessionKeys();
            // Attack successful: Bob
believes peer is Ana,
            // but the established key is
shared with the adversary
            success = true;
            state = AkeState.ESTABLISHED;
            Utils.trace("[AdvKeyEx: " +
locId + "] Attack successful, state "
+ state + ". Bob
believes peer is " + anaId);
            // Abort session S1 explicitly
using FAIL_B.
            sendFailMsgToAna("Session
aborted");
            notifyTerminate();
        }
        catch (Exception e) {
            success = false;
            state = AkeState.FAILED;
            sendFailMsg(bobId,
                "[AdvKeyEx: " + locId +
"] Failed to derive forged session keys");
            e.printStackTrace();
            notifyTerminate();
        }
        break;
    case ESTABLISHED:
        break;
    case FAILED:
        break;
    default:
        sendFailMsg(msg.getSrc(),
            "[AdvKeyEx: " + locId + "]
Illegal state " + state);
        }
        return null;
    }

    /**
     * Not used in this protocol variant.
     */
    public byte[] recvInitAuthMsg(InitAuthMsg
msg) throws Exception
    {
        sendFailMsg(msg.getSrc(),
            "[AdvKeyEx: " + locId + "]
Illegal message");
        return null;
    }

    /**
     * Handles received FailMsg.
     *
     * If attack already succeeded, ignore late
failure from Ana.
     * Otherwise terminate the adversary
session.
     */
    public void recvFailMsg(FailMsg msg) throws
Exception
    {

```

```

        Utils.trace("[AdvKeyEx: " + locId + "]
recvFail, state " + state
        + ". From " + msg.getSrc() + ":
" + msg.getReason());
        if (state != AkeState.ESTABLISHED) {
            state = AkeState.FAILED;
            success = false;
            notifyTerminate();
        }
    }

/**
 * Sends FAIL_B directly to Ana without
changing Adv's ESTABLISHED state.
 *
 * Adv plays Bob's role in session S1 with
Ana, therefore the correct
 * message type is FAIL_B.
 */
private void sendFailMsgToAna(String
reason) throws Exception
{
    Utils.trace("[AdvKeyEx: " + locId + "]
sendFail to " + anaId + ": " + reason);
    FailMsg failMsg = new FailMsg(
        MsgType.FAIL_B,
        bobId,
        anaId,
        sidAtk,
        reason
    );
    appComm.sendFailMsg(failMsg);
}

/**
 * Generates session keys using DH shared
secret between

```

```

 * adversary and B.
 *
 * The adversary obtains the same session
key material as Bob.
 */
protected void genSessionKeys() throws
Exception
{
    boolean isGCM =
commonCrypto.getEncSecName().split("/")[1].equal
s("GCM");
    int secBytesLen = (isGCM ? 2 : 4) *
commonCrypto.getSecKeyLen() / 8;
    // DH shared secret: (adv private, B
public)
    byte[] secretBytesDH =
KeyAgrpDH.genDHSecret(
        commonCrypto.getDhName(),
        advDHKeyPair,
        bobDHPubBytes
    );
    // Na | Nb
    byte[] seed = Utils.concatenate(nonceA,
nonceB);
    GenSessionKeyBytes genKeyBytes = new
GenSessionKeyBytes(commonCrypto.getAutSecName())
;
    SecretKey kdk =
genKeyBytes.extract(secretBytesDH, seed);
    byte[] randBytes =
genKeyBytes.expand(kdk, seed, secBytesLen);
    setSessionKeys(randBytes);
    Utils.trace("[AdvKeyEx: " + locId + "]
Generated session keys with Bob.");
}
}

```

Anexa A5. Cod complet atac ILV asupra MAC-PSK

```

package secpro.psk;

import java.security.KeyPair;

import javax.crypto.SecretKey;

import comm.AppComm;
import crypto.GenSessionKeyBytes;
import crypto.KeyAgrpDH;
import misc.Utils;
import secpro.AkeState;
import secpro.CryptoConfig;
import secpro.OwnCrypto;
import secpro.SecProBase;
import secpro.msg.AuthMsg;
import secpro.msg.FailMsg;
import secpro.msg.InitAuthMsg;
import secpro.msg.InitMsg;
import secpro.msg.MsgType;

```

```

/**
 *
 * Adversary implementation for interleaving
attack (ILV) on
 * MAKE-MAC-PSK-DHE 4-message vulnerable
protocol.
 *
 * The adversary impersonates A to B by:
 * - intercepting InitMsg from A
 * - substituting own DH public key
 * - forwarding messages between A and B
 * - reusing A's authenticator to authenticate
to B
 *
 */

public class AdvMakeMacIKEILV extends
SecProBase

```

```

{
    // Honest parties identities
    private String anaId;
    private String bobId;
    // Attack session identifier (same as Ana's
session)
    private byte[] sidAtk;
    // Nonces used in the interleaving attack
    private byte[] nonceA;
    private byte[] nonceB;
    // Bob's DH public key intercepted from
InitBA
    private byte[] bobDHPubBytes;
    // Adversary's own ephemeral DH key pair
    private KeyPair advDHKeyPair;

    public AdvMakeMacIKEILV(AppComm appComm,
String locId, byte[] sessionId,
CryptoConfig commonCrypto,
OwnCrypto ownCrypto) throws Exception
    {
        super(appComm, locId, sessionId,
commonCrypto, ownCrypto);
    }

    /**
     * Initializes adversary state.
     * No active message is sent; waits for
intercepted traffic.
     */
    public void doKeyExch(String locId, String
remId) throws Exception
    {
        this.locId = locId;
        this.remId = remId;
        this.state = AkeState.START;
        this.success = false;
        this.anaId = null;
        this.bobId = null;
        this.sidAtk = null;
        this.nonceA = null;
        this.nonceB = null;
        this.bobDHPubBytes = null;
        this.advDHKeyPair = null;

        Utils.trace("[AdvKeyEx: " + this.locId
+ "]" + " waiting for InitAB...");
    }

    /**
     * Processes intercepted InitMsg.
     *
     * Two roles:
     * - First InitMsg: from A -> intercepted
and modified -> sent to B
     * - Second InitMsg: from B -> forwarded
to A
     */
    public byte[] recvInitMsg(InitMsg msg)
throws Exception
    {

```

```

        Utils.trace("[AdvKeyEx: " + locId + "]"
recv InitMsg in state " + state
+ " from " + msg.getSrc() + "
to " + msg.getDst());

        switch (state) {
            case START:
                // intercept InitAB (A -> B)
                anaId = msg.getSrc();
                bobId = msg.getDst();
                // Store Bob as the real peer of
the adversary session
                remId = bobId;
                // Adversary plays initiator role
towards Bob

                initiator = true;
                // Reuse Ana's nonce and session
identifier

                nonceA = msg.getNonce();
                sidAtk = msg.getSid();
                // Generate adversary DH key pair
                advDHKeyPair =
KeyAgrDH.genDHKeyPair(
                    commonCrypto.getDhName(),
                    commonCrypto.getDhKeyLen()
                );
                // Send forged InitMsg to B with
Ana's identity and nonce,
                // but adversary's own DH public key
                InitMsg initToBob = new InitMsg(
                    MessageType.INIT_A,
                    anaId,
                    bobId,
                    sidAtk,
                    nonceA,

                    advDHKeyPair.getPublic().getEncoded(),
                    null,
                    null
                );
                state = AkeState.INIT_SENT;
                Utils.trace("[AdvKeyEx: " + locId +
"] send forged InitAB to " + bobId
+ ", new state " + state);
                appComm.sendInitMsg(initToBob);
                break;

            case INIT_SENT:
                // intercept InitBA (B -> A)
                if (!msg.getSrc().equals(bobId) ||
!msg.getDst().equals(anaId)) {
                    sendFailMsg(msg.getSrc(),
"[AdvKeyEx: " + locId +
"] Illegal peer in INIT_SENT");
                    return null;
                }
                // Store Bob's nonce and DH public
key

                nonceB = msg.getNonce();
                bobDHPubBytes = msg.getDhKey();
                // Derive session keys using
adversary DH private key and Bob's public key
                genSessionKeys();

```

```

        // Forward Bob's InitMsg to Ana
without modification
        InitMsg initToAna = new InitMsg(
            MsgType.INIT_B,
            bobId,
            anaId,
            sidAtk,
            nonceB,
            bobDHPubBytes,
            null,
            null
        );
        state = AkeState.INIT_RCVD;
        Utils.trace("[AdvKeyEx: " + locId +
            "] relay InitBA to " + anaId
                + ", new state " + state);
        appComm.sendInitMsg(initToAna);
        break;
    case ESTABLISHED:
        break;
    case FAILED:
        break;
    default:
        sendFailMsg(msg.getSrc(),
            "[AdvKeyEx: " + locId + "]
Illegal state " + state);
    }
    return null;
}

/**
 * Processes intercepted AuthMsg.
 *
 * Steps:
 * - intercept AuthAB -> forward to B
 * - intercept AuthBA -> derive attack
success and forward to A
 */
public byte[] recvAuthMsg(AuthMsg msg)
throws Exception
{
    Utils.trace("[AdvKeyEx: " + locId + "]
recv AuthMsg in state " + state
        + " from " + msg.getSrc() + "
to " + msg.getDst());
    switch (state) {
        case INIT_RCVD:
            // intercept AuthAB (A -> B)
            if (!msg.getSrc().equals(anaId) ||
!msg.getDst().equals(bobId)) {
                sendFailMsg(msg.getSrc(),
                    "[AdvKeyEx: " + locId +
            "] Illegal peer in INIT_RCVD");
                return null;
            }
            // Forward Ana's authenticator to
Bob unchanged
            AuthMsg authToBob = new AuthMsg(
                MsgType.AUTH_A,
                anaId,
                bobId,
                sidAtk,

```

```

                msg.getAuth(),
                null
            );
            state = AkeState.AUTH_SENT;
            Utils.trace("[AdvKeyEx: " + locId +
            "] relay AuthAB to " + bobId
                + ", new state " + state);
            appComm.sendAuthMsg(authToBob);
            break;
        case AUTH_SENT:
            // intercept AuthBA (B -> A)
            if (!msg.getSrc().equals(bobId) ||
!msg.getDst().equals(anaId)) {
                sendFailMsg(msg.getSrc(),
                    "[AdvKeyEx: " + locId +
            "] Illegal peer in AUTH_SENT");
                return null;
            }
            // Attack successful: Bob believes
peer is Ana,
            // but the established key is shared
with the adversary
            success = true;
            state = AkeState.ESTABLISHED;
            Utils.trace("[AdvKeyEx: " + locId +
            "] Attack successful, state "
                + state + ". Bob believes
peer is " + anaId);

            //aborts session S1 explicitly
using FAIL_B.
            sendFailMsgToAna("Session
aborted");
            notifyTerminate();
            break;
        case ESTABLISHED:
            break;
        case FAILED:
            break;
        default:
            sendFailMsg(msg.getSrc(),
                "[AdvKeyEx: " + locId + "]
Illegal state " + state);
    }
    return null;
}

/**
 * Not used in this protocol variant.
 */
public byte[] recvInitAuthMsg(InitAuthMsg
msg) throws Exception
{
    sendFailMsg(msg.getSrc(),
        "[AdvKeyEx: " + locId + "]
Illegal message");
    return null;
}

/**
 * Handles received FailMsg.

```

```

    *
    * If attack already succeeded, ignore late
    failure from Ana.
    * Otherwise terminate the adversary
    session.
    */
    public void recvFailMsg(FailMsg msg) throws
    Exception
    {
        Utils.trace("[AdvKeyEx: " + locId + "]
    recvFail, state " + state
        + ". From " + msg.getSrc() + ":
    " + msg.getReason());
        if (state != AkeState.ESTABLISHED) {
            state = AkeState.FAILED;
            success = false;
            notifyTerminate();
        }
    }

    /**
    * Sends FAIL_B directly to Ana without
    changing Adv's ESTABLISHED state.
    *
    * Adv plays Bob's role in session S1 with
    Ana, therefore the correct
    * message type is FAIL_B.
    */
    private void sendFailMsgToAna(String
    reason) throws Exception
    {
        Utils.trace("[AdvKeyEx: " + locId + "]
    sendFail to " + anaId + ": " + reason);
        FailMsg failMsg = new FailMsg(
            MsgType.FAIL_B,
            bobId,
            anaId,
            sidAtk,
            reason
        );
        appComm.sendFailMsg(failMsg);
    }

```

```

    /**
    * Generates session keys using DH shared
    secret between
    * adversary and B.
    *
    * The adversary obtains the same session
    key material as Bob.
    */
    protected void genSessionKeys() throws
    Exception
    {
        boolean isGCM =
    commonCrypto.getEncSecName().split("/")[1].equal
    s("GCM");
        int secBytesLen = (isGCM ? 2 : 4) *
    commonCrypto.getSecKeyLen() / 8;
        // DH shared secret: (adv private, B
    public)
        byte[] secretBytesDH =
    KeyAgrDH.genDHSecret(
            commonCrypto.getDhName(),
            advDHKeyPair,
            bobDHPubBytes
        );
        //Na | Nb
        byte[] seed = Utils.concatenate(onceA,
    nonceB);
        GenSessionKeyBytes genKeyBytes = new
    GenSessionKeyBytes(commonCrypto.getAutSecName())
    ;
        SecretKey kdk =
    genKeyBytes.extract(secretBytesDH, seed);
        byte[] randBytes =
    genKeyBytes.expand(kdk, seed, secBytesLen);
        setSessionKeys(randBytes);
        Utils.trace("[AdvKeyEx: " + locId + "]
    Generated session keys with Bob.");
    }
}

```